

A Complete Beginner's Guide to Python Programming

RYUU

© 2026 RYUU. Some Rights Reserved.

This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0).

NEXT WAY TO PYTHON

Complete Table of Contents

PART 1: GETTING STARTED WITH PYTHON

1.1 What is Python? 1.2 Online Python Compilers – No Installation Needed 1.3 Setting Up VS Code (Optional) 1.4 Running Your First Program 1.5 Online Python Playgrounds 1.6 Writing Your First Multi-Line Program

PART 2: VARIABLES AND DATA TYPES

2.1 Variables 2.2 Integers (int) 2.3 Floats (float) 2.4 Strings (str) 2.5 Booleans (bool) 2.6 Type Conversion

PART 3: INPUT, OUTPUT, AND OPERATORS

3.1 print() – Output 3.2 input() – User Input 3.3 Arithmetic Operators (+, -, *, /, //, %, **) 3.4 Comparison Operators (==, !=, >, <, >=, <=) 3.5 Logical Operators (and, or, not) 3.6 Assignment Operators (=, +=, -=, *=, /=)

PART 4: CONTROL FLOW

4.1 if Statements 4.2 if-else 4.3 elif (else if) 4.4 Nested Conditions 4.5 match-case (Python 3.10+) 4.6 Conditional Expressions (Ternary Operator)

PART 5: LOOPS

5.1 for Loops 5.2 while Loops 5.3 range() Function 5.4 break – Exit Loop Early 5.5 continue – Skip Iteration 5.6 pass – Do Nothing

PART 6: FUNCTIONS

6.1 Creating Functions (def) 6.2 Parameters and Arguments 6.3 Return Values 6.4 Scope (local vs global) 6.5 Lambda Functions 6.6 Recursion

PART 7: STRINGS

7.1 String Basics 7.2 Indexing (positive and negative) 7.3 Slicing [start:end:step] 7.4 String Methods (upper, lower, strip, replace, etc.) 7.5 Formatting Strings (f-strings, .format(), %) 7.6 Regular Expressions (re module)

PART 8: LISTS

8.1 Creating Lists 8.2 Accessing Elements 8.3 Modifying Lists 8.4 List Methods (append, insert, remove, pop, sort, etc.) 8.5 Nested Lists 8.6 List Comprehensions

PART 9: TUPLES

9.1 Creating Tuples 9.2 Accessing Tuple Elements 9.3 Tuple Operations 9.4 Packing and Unpacking 9.5 Tuple Methods (count, index)

PART 10: SETS

10.1 Creating Sets 10.2 Set Operations (add, remove, discard) 10.3 Union (|) and Intersection (&) 10.4 Difference (-) and Symmetric Difference (^) 10.5 Frozen Sets (frozenset)

PART 11: DICTIONARIES

11.1 Creating Dictionaries 11.2 Accessing Values 11.3 Modifying Data (add, update, delete) 11.4 Dictionary Methods (keys, values, items, get, etc.) 11.5 Dictionary Comprehensions

PART 12: MODULES AND PACKAGES

12.1 Importing Modules (import, from, as) 12.2 Built-in Modules (math, random, datetime, os, sys) 12.3 Creating Your Own Modules 12.4 Packages (**init**.py) 12.5 pip – Python Package Manager

PART 13: FILE HANDLING

13.1 Reading Files (open, read, readline, readlines) 13.2 Writing Files (write, writelines) 13.3 Appending Data 13.4 Working with CSV Files 13.5 Working with JSON Files

PART 14: ERROR HANDLING

14.1 try and except 14.2 Multiple Exceptions 14.3 else and finally 14.4 raise Keyword 14.5 Custom Exceptions

PART 15: OBJECT-ORIENTED PROGRAMMING

15.1 Classes and Objects 15.2 Constructors (**init**) 15.3 Inheritance 15.4 Polymorphism 15.5 Encapsulation (private attributes)

PART 16: ADVANCED OOP

16.1 Abstraction (ABC module) 16.2 Magic Methods (**str**, **len**, **add**, etc.) 16.3 Class Methods (@classmethod) 16.4 Static Methods (@staticmethod) 16.5 Property Decorators (@property)

PART 17: ADVANCED PYTHON

17.1 Iterators (**iter**, **next**) 17.2 Generators (yield) 17.3 Decorators (@decorator) 17.4 Closures 17.5 Context Managers (with)

PART 18: FUNCTIONAL PROGRAMMING

18.1 map() 18.2 filter() 18.3 reduce() (from functools) 18.4 zip() 18.5 any() and all()

PART 19: DATA STRUCTURES & ALGORITHMS

19.1 Big O Notation 19.2 Arrays and Lists 19.3 Stacks (LIFO) 19.4 Queues (FIFO) 19.5 Trees and Binary Search Trees 19.6 Graphs 19.7 Sorting Algorithms (bubble, quick, merge) 19.8 Searching Algorithms (linear, binary)

PART 20: DATABASES

20.1 SQL Fundamentals 20.2 SQLite (sqlite3 module) 20.3 PostgreSQL (psycopg2) 20.4 CRUD Operations (Create, Read, Update, Delete) 20.5 SQLAlchemy ORM

PART 21: BACKEND DEVELOPMENT

21.1 HTTP Fundamentals 21.2 REST APIs 21.3 Flask (Micro-framework) 21.4 FastAPI (Modern framework) 21.5 Authentication and JWT

PART 22: SUPER APP DEVELOPMENT

22.1 Frontend Basics (HTML, CSS, JavaScript) 22.2 React Integration 22.3 Docker and Containerization 22.4 Cloud Services (AWS, Google Cloud, Azure) 22.5 Building Full-Scale Applications

APPENDIX

A. Python Keywords B. Built-in Functions Reference C. Common Error Messages and Solutions D. Recommended Resources E. Practice Projects

END OF TABLE OF CONTENTS

PART 1: GETTING STARTED WITH PYTHON

CHAPTER 1.1: What is Python?

What You'll Learn

- What Python is (in plain English)
- Who created Python and why
- What Python is used for
- Why Python is so popular
- Python vs other languages

What Is Python?

Python is a high-level, interpreted programming language known for its simplicity and readability.

It was designed to be easy to read and write, with clean syntax that uses indentation instead of curly braces.

```
# This is Python
name = "Ryu"
print(f"Hello, {name}!")
```

Who Created Python?

Guido van Rossum created Python and first released it in **1991**.

He named it after the British comedy group **Monty Python**, not the snake! (Though the snake logo came later.)

What Can You Build With Python?

Field	Examples
Web Development	Django, Flask, FastAPI
Data Science	Pandas, NumPy, Matplotlib
Machine Learning	TensorFlow, PyTorch, scikit-learn
Automation & Scripting	File processing, web scraping
Game Development	Pygame
Desktop Apps	Tkinter, PyQt, Kivy
Backend APIs	RESTful services
Scientific Computing	Research, simulations

Why Is Python So Popular?

Reason	Explanation
Easy to learn	Simple, readable syntax
Versatile	Works for web, data, AI, automation
Large community	Millions of developers worldwide
Rich ecosystem	Thousands of libraries and frameworks
Great for beginners	Feels like writing English
In high demand	Used at Google, NASA, Netflix, Instagram

Python vs Other Languages

Language	Difficulty	Best For
Python	Easy	Data science, web, automation
JavaScript	Medium	Web browsers, frontend
Java	Medium-Hard	Enterprise, Android
C++	Hard	Games, systems programming
Swift	Medium	Apple apps

Is Python Hard to Learn?

Honest answer: Python is one of the **easiest** programming languages to learn.

- Easier than Java, C++, C#
- About the same difficulty as JavaScript (maybe easier)
- Perfect for your **second language** after Swift

What Can You Build After Learning Python?

After Part	You Can Build
Part 5	Calculator, mad libs game
Part 8	Number guessing game, password generator
Part 11	To-do list app, shopping cart
Part 13	Student grade tracker, bank account system
Part 15	Tic-tac-toe, blackjack game
Full Book	Web apps, data visualizations, APIs

Try It Yourself

You don't need to install anything!

Go to:  <https://www.online-python.com> 

Or search "online Python compiler" — there are many free options.

Type this code:

```
print("Hello, Python!")  
print("I'm learning to code!")
```

Click **"Run"** — you just wrote and executed Python code instantly!

Chapter Summary

What	Summary
What Python is	A readable, versatile programming language
Who created it	Guido van Rossum (1991)
What it's used for	Web, data science, AI, automation
Difficulty	Very beginner-friendly
First step	Try online-python.com (no install needed!)

CHAPTER 1.2: Online Python Compilers – No Installation Needed

What You'll Learn

- Why you don't need to install anything
- Best online Python compilers
- How to use them
- Creating an account (optional)

You Don't Need to Install Python!

Here's the secret: **You can start coding in Python right now without installing anything.**

Your browser is all you need.

Best Online Python Compilers

Website	Best For	Free?
online-python.com	Beginners, simple code	✓ Yes
replit.com	Saving projects, collaboration	✓ Yes
trinket.io	Embedding in websites	✓ Yes
programiz.com	Learning with tutorials	✓ Yes
pythontutor.com	Visualizing code step-by-step	✓ Yes

Using online-python.com

Step 1: Open Your Browser

Go to: **<https://www.online-python.com>**

Step 2: Write Your Code

You'll see a code editor with some example code already there.

Delete it and type:

```
print("Python is awesome!")
```

Step 3: Click Run

Click the **"Run"** button (usually green).

Step 4: See the Output

The output appears in a panel below.

```
Python is awesome!
```

That's it! You just ran Python code without installing anything.

Using replit.com (For Saving Projects)

Step 1: Go to replit.com

<https://replit.com>

Step 2: Sign Up (Optional but Recommended)

Click "Sign up" — you can use Google, GitHub, or email.

Step 3: Create a New Repl

Click "+ Create Repl" or "Create".

Choose:

- Language: **Python**
- Title: `my-first-python`

Step 4: Write and Run Code

Type your code and click **"Run"**.

Your code and output appear side by side.

Bonus: Your repl is saved forever! You can come back to it later.

Using pythontutor.com (To See Code Step-by-Step)

This is great for understanding **how** your code runs:

Go to **<https://pythontutor.com>**

Type your code and click **"Visualize Execution"**.

You'll see your code execute line by line, showing how variables change.

Pros and Cons of Online Compilers

Pro	Con
✓ No installation	✗ Requires internet
✓ Works on any computer (even school/chromebooks)	✗ Some advanced features missing
✓ Start coding in seconds	✗ Can't install external packages
✓ Great for learning	✗ Might have file size limits
✓ Save and share code	

When to Install Python Later

Once you're comfortable, you can install Python on your computer (Chapter 1.3 covers it). But for learning the basics, **online compilers are perfect**.

Try It Yourself

Challenge 1: Go to online-python.com and run `print("Hello from the cloud!")`

Challenge 2: Create a free account on replit.com and save a repl called "My First Code"

Challenge 3: Use pythontutor.com to visualize `x = 5; y = 10; z = x + y; print(z)`

Chapter Summary

Website	URL	Best For
online-python.com	online-python.com	Quick testing
replit.com	replit.com	Saving projects
pythontutor.com	pythontutor.com	Learning step-by-step

CHAPTER 1.3: Setting Up VS Code (Optional)

What You'll Learn

- Why VS Code is useful for larger projects
- How to install VS Code (if you want)
- Installing the Python extension
- Running Python in VS Code

When to Use VS Code

Online compilers are great for learning, but when you:

- Build larger projects
- Want offline access
- Need to install external packages
- Want more powerful editing features

...then install VS Code.

For now, you can skip this chapter and come back later!

Why VS Code?

Feature	Benefit
Free	No cost
Cross-platform	Windows, Mac, Linux
Python extension	Syntax highlighting, IntelliSense, debugging
Integrated terminal	Run code without leaving the editor
Git integration	Version control built-in
Offline	No internet needed once installed

Installing VS Code

1. Go to **code.visualstudio.com**
2. Download the installer for your operating system
3. Run the installer
4. Open VS Code

Installing the Python Extension

Step 1: Open Extensions Panel

Click the **Extensions icon** (four squares) in the left sidebar.

Step 2: Search and Install

Type "**Python**" — look for the extension by **Microsoft**. Click **Install**.

Creating Your First Python File in VS Code

Step 1: Create a Folder

Create a folder called `python-beginner` on your computer.

Step 2: Open Folder in VS Code

File → Open Folder → Select your folder.

Step 3: Create a Python File

Click **New File**, save as `hello.py`.

Step 4: Write Code

```
print("Hello, VS Code!")
```

Step 5: Run

Click the **Run** button () or press `Ctrl + F5` (Windows) / `Cmd + F5` (Mac).

When to Use What

You Want To...	Use Online	Use VS Code
Start immediately	✓	⌚ (needs install)
Code on school computer	✓	⌚ (might be blocked)
Build big projects	✗ (limitations)	✓
Work offline	✗	✓
Install packages	✗ (limited)	✓

Chapter Summary

Action	Command/Link
Download VS Code	code.visualstudio.com
Python extension	Search "Python" (by Microsoft)
Run Python file	<code>Ctrl + F5</code> (Win) / <code>Cmd + F5</code> (Mac)

Remember: This chapter is **optional**. You can learn Python perfectly fine using online compilers!

CHAPTER 1.4: Running Your First Program

What You'll Learn

- Writing your first Python program
- Understanding `print()`

- Running code online
- Common first program mistakes

Your First Python Program

Go to **online-python.com** and type:

```
print("Hello, World!")
```

Click **Run**. You should see:

```
Hello, World!
```

Congratulations! You just wrote and ran your first Python program. 🎉

What Just Happened?

Part	What It Is
<code>print()</code>	A built-in function that outputs text
<code>"Hello, World!"</code>	A string (text) inside quotes
<code>()</code>	Parentheses that hold the content to print

Printing Multiple Lines

```
print("Hello, World!")  
print("I am learning Python!")  
print("Python is fun!")
```

Output:

```
Hello, World!  
I am learning Python!  
Python is fun!
```

Printing Without a Newline

```
print("Hello", end=" ")
print("World!")
```

Output:

```
Hello World!
```

The `end=" "` parameter tells Python to add a space instead of a new line.

Printing Numbers and Multiple Items

```
print(42)
print(3.14)
print("The answer is", 42)
```

Output:

```
42
3.14
The answer is 42
```

Common First Program Mistakes

Mistake	Wrong	Correct
Missing quotes	<code>print(Hello)</code>	<code>print("Hello")</code>
Wrong quotes	<code>print('Hello')</code>	<code>print("Hello")</code> (both actually work, but double is common)
Capitalization	<code>Print("Hello")</code>	<code>print("Hello")</code>
Missing parentheses	<code>print "Hello"</code>	<code>print("Hello")</code>

Running in Different Online Compilers

The code is the same everywhere:

Website	Code	Run Button Location
online-python.com	<pre>print("Hi")</pre>	Top (green)
replit.com	<pre>print("Hi")</pre>	Top (▶)
programiz.com	<pre>print("Hi")</pre>	Right side

Try It Yourself

Challenge 1: Print your name on the screen.

Challenge 2: Print three lines: your name, your age, and your favorite color.

Challenge 3: Print "Python" and "Rocks" on the same line with a space between them.

Challenge 4: What happens if you forget the quotes around text? Try it and see.

Chapter Summary

Concept	Syntax	Example
Print text	<pre>print("text")</pre>	<pre>print("Hello")</pre>
Print numbers	<pre>print(number)</pre>	<pre>print(42)</pre>
Print without newline	<pre>print("text", end="")</pre>	<pre>print("Hi", end="")</pre>
Print multiple items	<pre>print(a, b, c)</pre>	<pre>print("Age:", 11)</pre>

CHAPTER 1.5: Online Python Playgrounds

What You'll Learn

- How to experiment with code instantly
- Different online playgrounds
- Saving and sharing code
- Using playgrounds for learning

What Is a Python Playground?

A **playground** is an online coding environment where you can:

- Write code

- Run it instantly
- See the output
- Save and share your work

Best Python Playgrounds

Name	URL	Best Feature
Online-Python	online-python.com	Simple, fast
Replit	replit.com	Saves projects, collaboration
Google Colab	colab.research.google.com	Free GPU, data science
Jupyter Lite	jupyterlite.org	Notebooks in browser
PythonAnywhere	pythonanywhere.com	Host your code online

Using Online-Python as a Playground

Go to **online-python.com**:

1. Write code in the editor
2. Click "**Run**"
3. See output below
4. Click "**Save**" to get a unique URL

You can share the URL with anyone, and they'll see your code!

Using Replit for Saved Projects

Replit is great for building and **saving** projects:

1. Create a free account
2. Click "+ **Create Repl**"
3. Choose Python
4. Write code
5. It saves **automatically**

You can organize your learning into different repls:

```
python-beginner/  
├─ hello_world/  
├─ calculator/  
├─ loops_practice/  
└─ my_first_game/
```

Using Google Colab (For Data Science)

Google Colab is like a free Jupyter notebook in your browser:

1. Go to **colab.research.google.com**
2. Click **"New Notebook"**
3. Write code in "cells"
4. Press `Shift + Enter` to run

Colab is great for:

- Data analysis
- Machine learning
- Sharing notebooks

Sharing Your Code

Method	How
online-python.com	Click "Save" → share URL
Replit	Click "Share" → copy link
Google Colab	File → Share → copy link
Pastebin	Copy code → pastebin.com

Try It Yourself

Challenge 1: Create a free account on replit.com and make a "My First Project" repl.

Challenge 2: Write a program that prints your name and age, then save and share the link.

Challenge 3: Open Google Colab and run `print("Colab is cool!")`

Chapter Summary

Platform	Use For
online-python.com	Quick testing, no account
replit.com	Saving projects, free account
Google Colab	Data science, notebooks

CHAPTER 1.6: Writing Your First Multi-Line Program

What You'll Learn

- Writing longer programs
- Using comments
- Saving your code
- Creating a simple interactive program

Your First Real Program

Let's write something more interesting than just `print("Hello")`.

Go to your online compiler and type:

```
# My first Python program
# This program asks for your name and greets you

print("Welcome to Python!")
print("=" * 30)

name = input("What is your name? ")
age = input("How old are you? ")

print()
print(f"Hello, {name}!")
print(f"You are {age} years old.")
print("Let's learn Python together!")
```

Run it and see what happens!

What Each Line Does

Line	What It Does
<code># My first Python program</code>	Comment (ignored by Python)
<code>print("Welcome...")</code>	Prints a welcome message
<code>print("=" * 30)</code>	Prints 30 equal signs
<code>name = input("What is your name? ")</code>	Asks for input, stores in <code>name</code>
<code>age = input("How old are you? ")</code>	Asks for input, stores in <code>age</code>
<code>print()</code>	Prints a blank line
<code>print(f"Hello, {name}!")</code>	Prints greeting using the name
<code>print(f"You are {age} years old.")</code>	Prints age
<code>print("Let's learn...")</code>	Final message

Comments in Python

Comments are for **you** (and other humans). Python ignores them.

```
# This is a single-line comment

"""
This is a multi-line comment
It can span several lines
Python ignores all of this too
"""
```

The `input()` Function

`input()` waits for the user to type something and press Enter:

```
name = input("What is your name? ")
# Python pauses here until you type something
# Whatever you type gets stored in the variable 'name'
```

f-Strings (Formatted Strings)

The `f` before quotes tells Python to insert variables into the string:

```
name = "Ryu"
print(f"Hello, {name}!") # Hello, Ryu!

age = 11
print(f"You are {age} years old") # You are 11 years old
```

Saving Your Program

On online-python.com:

Click **"Save"** → Copy the URL

On replit.com:

It saves automatically! Just close the tab and come back later.

On your computer (if using VS Code):

Save as `greeting.py`

Try It Yourself

Challenge 1: Modify the greeting program to ask for favorite color too.

Challenge 2: Create a program that asks for two numbers and prints their sum.

Challenge 3: Add comments explaining what each line of your program does.

Challenge 4: Share your saved program with a friend.

Example: Simple Calculator

```
# Simple Calculator
print("=== SIMPLE CALCULATOR ===")

num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print(f"{num1} + {num2} = {num1 + num2}")
print(f"{num1} - {num2} = {num1 - num2}")
print(f"{num1} * {num2} = {num1 * num2}")
print(f"{num1} / {num2} = {num1 / num2}")
```

(We'll learn about `int()` in Part 2!)

Chapter Summary

Concept	Syntax	Example
Comment	<code># text</code>	<code># This is a comment</code>
Input	<code>input("prompt")</code>	<code>name = input("Name? ")</code>
f-string	<code>f"text {variable}"</code>	<code>f"Hello {name}"</code>
Multi-line comment	<code>""" text """</code>	See above

PART 1 SUMMARY

You now know how to start coding in Python **without installing anything**:

Chapter	Topic
1.1	What is Python
1.2	Online Python Compilers (no install!)
1.3	VS Code Setup (optional)
1.4	Running Your First Program
1.5	Online Python Playgrounds
1.6	Writing Your First Multi-Line Program

What's Next?

Next up: **Part 2: Variables and Data Types** — storing and working with data!

Remember: You don't need to install anything. Just keep using online-python.com!

PART 2: VARIABLES AND DATA TYPES

CHAPTER 2.1: Variables – Storing Information

What You'll Learn

- What variables are (named boxes for data)
- How to create variables in Python
- Rules for naming variables
- Best practices for variable names

What Is a Variable?

A **variable** is a named container that stores data. It's called "variable" because its value can **vary** (change).

Think of it like a **labeled box**:

- The label is the variable name (like `score` or `player_name`)
- Inside the box is the value (like `100` or `"Ryu"`)
- You can change what's inside the box

Creating Variables in Python

Unlike Swift, you **don't** need `var` or `let`. Just write the name, an equals sign, and the value:

```
name = "Ryu"
age = 11
score = 95
is_student = True
```

Python automatically figures out the type (more on that in Chapter 2.2-2.5).

Reading Variables

Just use the variable name:

```
name = "Ryu"  
print(name)  # Ryu
```

Changing Variables

Assign a new value (no special keyword needed):

```
score = 100  
print(score)  # 100  
  
score = 250  
print(score)  # 250  
  
score = 500  
print(score)  # 500
```

Multiple Variables in One Line

```
x, y, z = 10, 20, 30  
print(x)  # 10  
print(y)  # 20  
print(z)  # 30
```

Or assign the same value to multiple variables:

```
a = b = c = 100  
print(a, b, c)  # 100 100 100
```

Swapping Variables

Python makes swapping super easy:


```
a = 10
b = 20

a, b = b, a # Swap!

print(a) # 20
print(b) # 10
```

Naming Rules (Must Follow)

Python enforces these rules:

Rule	Good Example	Bad Example
No spaces	<code>player_score</code>	<code>player score</code>
Start with letter or underscore	<code>score</code> , <code>_private</code>	<code>1st_place</code>
Only letters, numbers, underscore	<code>score_2</code>	<code>score!</code>
Case-sensitive	<code>Score</code> and <code>score</code> are different	
No reserved words	<code>player_name</code>	<code>class</code> (reserved)

Reserved Words (Can't Use as Names)

```
# These are reserved by Python:
False, True, None, and, or, not, if, else, elif, for, while, break,
continue, return, def, class, import, from, try, except, finally,
raise, with, as, lambda, global, nonlocal, pass, assert, yield, del
```

Naming Conventions (Should Follow)

These are **optional** but all Python programmers follow them:

Convention	Example	When to Use
<code>snake_case</code>	<code>player_score</code> , <code>user_name</code>	Variables, functions (MOST common)
<code>UPPER_CASE</code>	<code>MAX_SCORE</code> , <code>PI</code>	Constants (never change)
<code>_leading_underscore</code>	<code>_internal_var</code>	"Private" (internal use only)
<code>__double_underscore</code>	<code>__secret</code>	Name mangling (advanced)

Python uses snake_case for almost everything:

```
# ✅ Good Python style
player_name = "Ryu"
high_score = 1000
is_game_over = False
max_lives = 3

# ❌ Not Python style (looks like Swift/Java)
playerName = "Ryu"
highScore = 1000
isGameOver = False
```

Descriptive Names

❌ Bad	✅ Good
x	score
a	age
d	distance_in_km
temp	temperature
flag	is_ready

```
# Bad
x = 5
y = 3
z = x + y

# Good
side_a = 5
side_b = 3
hypotenuse = (side_a ** 2 + side_b ** 2) ** 0.5
```

Variable Types Change Dynamically

Unlike Swift, Python variables can change type:

```
value = 10          # int
print(value)

value = "Hello"     # now a string
print(value)

value = 3.14        # now a float
print(value)

value = True        # now a boolean
print(value)

# This is allowed in Python! (Swift would complain)
```

Checking Variable Type

Use the `type()` function:

```
name = "Ryu"
age = 11
price = 9.99
is_valid = True

print(type(name))    # <class 'str'>
print(type(age))     # <class 'int'>
print(type(price))   # <class 'float'>
print(type(is_valid)) # <class 'bool'>
```

Deleting Variables

Use `del` to delete a variable:

```
score = 100
print(score)    # 100

del score

# print(score)  # ERROR! NameError: name 'score' is not defined
```

Try It Yourself

Challenge 1: Create variables for your name, age, and favorite color. Print each one.

Challenge 2: Swap the values of two variables without creating a third variable.

Challenge 3: Which of these are valid variable names?

- `2fast2furious`
- `player_score`
- `player-score`
- `_hidden`
- `class`

Challenge 4: Create a variable and change its type from int to string to float.

Chapter Summary

Concept	Syntax	Example
Create variable	<code>name = value</code>	<code>score = 100</code>
Multiple variables	<code>a, b = 1, 2</code>	<code>x, y = 10, 20</code>
Same value	<code>a = b = c = 0</code>	All three start at 0
Swap	<code>a, b = b, a</code>	Swap values
Delete	<code>del variable</code>	<code>del score</code>
Check type	<code>type(variable)</code>	<code>type(age)</code>

CHAPTER 2.2: Integers (int) – Whole Numbers

What You'll Learn

- What integers are (whole numbers)
- Creating and using integers
- Integer operations
- Integer limits (there are none in Python!)

What Is an Integer?

An **integer** (int) is a whole number with no decimal point:

Integer Division (Important!)

In Python, `/` always returns a **float**:

```
print(10 / 3)    # 3.3333333333333335 (float)
print(10 / 2)    # 5.0 (float, not 5!)
```

Use `//` for integer division (floor division):

```
print(10 // 3)   # 3
print(10 // 2)   # 5
print(-10 // 3)  # -4 (floor division rounds down)
```

Underscores in Numbers

Use underscores to make large numbers readable:

```
million = 1_000_000
billion = 1_000_000_000
credit_card = 1234_5678_9012_3456

print(million)  # 1000000 (underscores ignored)
```

Binary, Octal, Hexadecimal

Python supports different number bases:

```
# Binary (0b prefix)
binary = 0b1010  # 10 in decimal
print(binary)    # 10

# Octal (0o prefix)
octal = 0o12     # 10 in decimal
print(octal)     # 10

# Hexadecimal (0x prefix)
hexadecimal = 0xA  # 10 in decimal
print(hexadecimal) # 10
```

Converting to Integer

Use `int()` to convert other types to integer:

```
# String to int
num1 = int("123")
print(num1)  # 123

# Float to int (truncates, doesn't round)
num2 = int(3.99)
print(num2)  # 3

# Binary string to int
num3 = int("1010", 2)
print(num3)  # 10
```

Checking if a Value is an Integer

```
value = 42
print(type(value) == int)  # True
print(isinstance(value, int))  # True
```

Real-World Examples

Counter

```
counter = 0
counter += 1
counter += 1
counter += 1
print(counter)  # 3
```

Remainder (Even/Odd Check)

```
number = 7

if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")    # 7 is odd
```

Simple Calculator

```
a = 25
b = 4

print(f"Sum: {a + b}")
print(f"Difference: {a - b}")
print(f"Product: {a * b}")
print(f"Quotient: {a // b} remainder {a % b}")
```

Try It Yourself

Challenge 1: Create an integer for your age, then add 10 to it and print the result.

Challenge 2: Use `//` and `%` to split 127 minutes into hours and minutes.

Challenge 3: Write 1,000,000 with underscores. Print it.

Challenge 4: Use `int()` to convert the string "99" to an integer, then multiply by 3.

Chapter Summary

Operation	Operator	Example	Result
Addition	<code>+</code>	<code>10 + 3</code>	<code>13</code>
Subtraction	<code>-</code>	<code>10 - 3</code>	<code>7</code>
Multiplication	<code>*</code>	<code>10 * 3</code>	<code>30</code>
Division (float)	<code>/</code>	<code>10 / 3</code>	<code>3.333...</code>
Floor division	<code>//</code>	<code>10 // 3</code>	<code>3</code>
Modulo (remainder)	<code>%</code>	<code>10 % 3</code>	<code>1</code>
Exponent	<code>**</code>	<code>10 ** 3</code>	<code>1000</code>

CHAPTER 2.3: Floats (float) – Decimal Numbers

What You'll Learn

- What floats are (numbers with decimal points)
- Creating and using floats
- Float precision issues
- Converting between ints and floats

What Is a Float?

A **float** (floating-point number) is a number with a decimal point:

```
price = 19.99
pi = 3.14159
temperature = -5.5
gpa = 3.75
```

Creating Floats

```
height = 1.75      # meters
weight = 68.5      # kg
distance = 10.5    # km

# Scientific notation
large = 1.5e6       # 1,500,000
small = 2.5e-4      # 0.00025
```

Float Operations

```
a = 5.5
b = 2.0

print(a + b)    # 7.5
print(a - b)    # 3.5
print(a * b)    # 11.0
print(a / b)    # 2.75
print(a ** b)   # 30.25 (5.5 squared)
```

Mixing Integers and Floats

When you mix int and float, Python automatically returns a float:

```
result1 = 5 + 2.5    # 7.5 (float)
result2 = 10 * 1.5    # 15.0 (float)
result3 = 7 // 2.0    # 3.0 (float, floor division with float)
```

Converting Between Int and Float

```
# Int to float
x = 10
print(float(x))    # 10.0

# Float to int (truncates, doesn't round)
y = 3.99
print(int(y))      # 3

# Round instead of truncate
z = 3.99
print(round(z))     # 4
```

The Float Precision Problem

Just like in Swift, Python has floating-point precision issues:

```
x = 0.1 + 0.2
print(x)  # 0.30000000000000004 (not 0.3!)
```

Why? Some decimals can't be represented exactly in binary. This is **not a bug** — it happens in every programming language.

Handling Precision

For display (just round it):

```
x = 0.1 + 0.2
print(round(x, 2))  # 0.3 (rounded to 2 decimal places)
```

For money (use integers to count cents):

```
# Instead of storing $19.99 as float:
price_cents = 1999  # 1999 cents = $19.99
total_cents = price_cents * 3
dollars = total_cents // 100
cents = total_cents % 100
print(f"${dollars}.{cents:02d}")  # $59.97
```

For high precision (use Decimal):

```
from decimal import Decimal

x = Decimal('0.1') + Decimal('0.2')
print(x)  # 0.3 (exact!)
```

Checking if a Value is a Float

```
value = 3.14
print(type(value) == float)  # True
print(isinstance(value, float))  # True
```

Real-World Examples

BMI Calculator

```
weight = 68.5 # kg
height = 1.75 # meters

bmi = weight / (height ** 2)
print(f"Your BMI is: {bmi:.2f}") # Format to 2 decimal places
```

Temperature Converter

```
celsius = 25.0
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}°C = {fahrenheit}°F") # 25.0°C = 77.0°F
```

Circle Area

```
radius = 5.0
pi = 3.14159
area = pi * radius ** 2
print(f"Area: {area:.2f}") # Area: 78.54
```

Try It Yourself

Challenge 1: Create a float for your height in meters. Convert it to centimeters (multiply by 100).

Challenge 2: Add `0.1 + 0.2` and then `0.1 + 0.3`. Which one is exact?

Challenge 3: Convert `3.99` to int, then round it. What's the difference?

Challenge 4: Use `round()` to display `3.14159` with 2 decimal places.

Chapter Summary

Operation	Example	Result
Int → Float	<code>float(10)</code>	<code>10.0</code>
Float → Int	<code>int(3.99)</code>	<code>3</code> (truncates)
Round float	<code>round(3.99)</code>	<code>4</code>
Round with decimals	<code>round(3.14159, 2)</code>	<code>3.14</code>

CHAPTER 2.4: Strings (str) – Text

What You'll Learn

- What strings are (text, words, sentences)
- Creating strings with quotes
- String concatenation
- String methods
- f-strings (formatted strings)

What Is a String?

A **string** (str) is a sequence of characters — text, words, sentences, or even just a single letter.

```
single_letter = "A"
word = "Hello"
sentence = "Python is fun!"
paragraph = "This is a longer piece of text. It can have multiple sentences."
```

Creating Strings

```
# Single quotes
name = 'Ryu'

# Double quotes (most common)
message = "Hello, World!"

# Triple quotes (multi-line)
poem = """
Roses are red,
Violets are blue,
Python is awesome,
And so are you!
"""

# Empty string
empty = ""
```

Quotes Inside Strings

```
# Use double quotes to have single quotes inside
text1 = "It's a beautiful day"

# Use single quotes to have double quotes inside
text2 = 'He said "Hello" to me'

# Or use escape characters
text3 = 'It\'s a beautiful day'
text4 = "He said \"Hello\" to me"
```

String Concatenation

Joining strings with `+`:

```
first_name = "Ryu"
last_name = "Swift"
full_name = first_name + " " + last_name
print(full_name)  # "Ryu Swift"

# Using +=
greeting = "Hello"
greeting += " "
greeting += "World!"
print(greeting)  # "Hello World!"
```

String Multiplication

```
print("-" * 30)      # "-----"
print("Ha" * 3)      # "HaHaHa"
```

String Length

```
name = "Ryu"
print(len(name))     # 4

empty = ""
print(len(empty))    # 0
```

String Methods

```
text = "  Hello, World!  "

# Case conversion
print(text.upper())      # "  HELLO, WORLD!  "
print(text.lower())      # "  hello, world!  "
print(text.title())      # "  Hello, World!  "
print(text.capitalize()) # "  hello, world!  "

# Whitespace removal
print(text.strip())      # "Hello, World!"
print(text.lstrip())     # "Hello, World!  "
print(text.rstrip())     # "  Hello, World!"

# Checking
print(text.startswith("  He")) # True
print(text.endswith("!  "))    # True
print(text.isalpha())          # False (has spaces, punctuation)
print(text.isdigit())          # False
```

String Search

```
sentence = "The quick brown fox jumps over the lazy dog"

# Find position
print(sentence.find("fox"))      # 16 (index where "fox" starts)
print(sentence.find("cat"))     # -1 (not found)

# Check if contains
print("fox" in sentence)        # True
print("cat" in sentence)        # False

# Count occurrences
print(sentence.count("o"))      # 4
```


String Replacement

```
text = "I like cats. Cats are cute."
new_text = text.replace("cats", "dogs")
print(new_text) # "I like dogs. Cats are cute."

# Case-sensitive, so "Cats" stays "Cats"
```

Splitting Strings

```
sentence = "apple,banana,orange"
fruits = sentence.split(",")
print(fruits) # ['apple', 'banana', 'orange']

words = "The quick brown fox".split()
print(words) # ['The', 'quick', 'brown', 'fox']
```

Joining Strings

```
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence) # "Python is awesome"

csv = ",".join(["apple", "banana", "orange"])
print(csv) # "apple,banana,orange"
```

f-Strings (Formatted Strings)

The most modern way to insert variables into strings:

```
name = "Ryu"
age = 11
score = 95.5

print(f"Name: {name}, Age: {age}, Score: {score}")
# "Name: Ryu, Age: 11, Score: 95.5"

# With formatting
print(f"Score: {score:.1f}") # "Score: 95.5"
print(f"Score: {score:.0f}") # "Score: 96" (rounded)

# Expressions inside {}
print(f"Next year I'll be {age + 1}") # "Next year I'll be 12"
print(f"5 + 3 = {5 + 3}") # "5 + 3 = 8"
```

Older String Formatting (Good to Know)

```
name = "Ryu"
age = 11

# .format() method
print("Name: {}, Age: {}".format(name, age))

# % formatting (old style)
print("Name: %s, Age: %d" % (name, age))
```

Escape Characters

```
# New line
print("Hello\nWorld")

# Hello
# World

# Tab
print("Hello\tWorld")    # "Hello    World"

# Backslash
print("C:\Users\Ryuu")   # "C:\Users\Ryuu"

# Quote inside quotes
print("She said \"Hello\"") # She said "Hello"
```

Raw Strings (Ignore Escape Characters)

```
# Regular string (backslashes are escaped)
regular = "C:\Users\Ryuu"

# Raw string (backslashes are literal)
raw = r"C:\Users\Ryuu"

print(regular)    # C:\Users\Ryuu
print(raw)        # C:\Users\Ryuu
```

Try It Yourself

Challenge 1: Create a string with your full name and print its length.

Challenge 2: Use `split()` to break a sentence into words.

Challenge 3: Use an f-string to print "My name is X and I am Y years old".

Challenge 4: Replace all spaces with underscores in a string.

Chapter Summary

Operation	Syntax	Example
Create string	<code>"text"</code> or <code>'text'</code>	<code>name = "Ryu"</code>
Multi-line	<code>"""text"""</code>	Triple quotes
Length	<code>len(s)</code>	<code>len("Hi")</code> → <code>2</code>
Concatenation	<code>s1 + s2</code>	<code>"Hi" + " " + "Ryu"</code>
Repeat	<code>s * n</code>	<code>"Ha" * 3</code> → <code>"HaHaHa"</code>
f-string	<code>f"{var}"</code>	<code>f"Hello {name}"</code>
in check	<code>sub in s</code>	<code>"cat" in "caterpillar"</code> → <code>True</code>

CHAPTER 2.5: Booleans (bool) – True or False

What You'll Learn

- What Booleans are (True or False)
- Creating Boolean values
- Comparison operators return Booleans
- Logical operators (and, or, not)

What Is a Boolean?

A **Boolean** (bool) has only two possible values:

- `True`
- `False`

Note: Capital `T` and capital `F`!

```
is_game_over = False
is_logged_in = True
has_key = False
is_raining = True
```

Creating Booleans

```
# Direct assignment
is_ready = True
is_done = False

# Using bool() function
print(bool(1))    # True
print(bool(0))    # False
print(bool(""))   # False (empty string)
print(bool("Hi")) # True (non-empty string)
print(bool([]))   # False (empty list)
print(bool([1]))  # True (non-empty list)
```

Truthy and Falsy Values

In Python, many values are considered "truthy" or "falsy" in Boolean contexts:

Falsy (act like False)	Truthy (act like True)
False	True
None	Any number except 0
0	Non-empty string
0.0	Non-empty list
"" (empty string)	Non-empty dictionary
[] (empty list)	Non-empty set
{ } (empty dict)	
() (empty tuple)	

```
if 5:
    print("5 is truthy") # This prints

if 0:
    print("0 is truthy") # This does NOT print

if "Hello":
    print("'Hello' is truthy") # This prints

if "":
    print("Empty string is truthy") # This does NOT print
```

Comparison Operators

Comparisons always return Booleans:

```
a = 10
b = 20

print(a == b) # False (equal to)
print(a != b) # True (not equal to)
print(a > b) # False (greater than)
print(a < b) # True (less than)
print(a >= b) # False (greater than or equal)
print(a <= b) # True (less than or equal)
```

Logical Operators

and (both must be True)

```
age = 11
has_permission = True

if age >= 10 and has_permission:
    print("You can join the club") # This runs
```

Truth table:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

or (at least one must be True)

```
is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("No school today!") # This runs
```

Truth table:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

not (reverses the value)

```
is_game_over = False

if not is_game_over:
    print("Keep playing!") # This runs (not False = True)
```

Truth table:

A	not A
True	False
False	True

Combining Conditions

```
age = 16

has_license = True
has_permit = False

# Can drive if (age >= 16 AND has_license) OR (age >= 15 AND has_permit)
if (age >= 16 and has_license) or (age >= 15 and has_permit):
    print("You can drive")
else:
    print("You cannot drive")
```

Short-Circuit Evaluation

Python stops evaluating as soon as it knows the result:

```
def expensive_check():
    print("Doing expensive check...")
    return True

is_fast = False
result = is_fast and expensive_check()

# expensive_check() NEVER runs because is_fast is False
```

Booleans in Numbers

```
# True is 1, False is 0 in numeric contexts
print(int(True))    # 1
print(int(False))   # 0

print(True + True)   # 2
print(True * 10)     # 10
print(False * 100)   # 0
```


Real-World Examples

Login Check

```
username = "ryuu"
password = "swift123"
is_locked = False

if not is_locked and username == "ryuu" and password == "swift123":
    print("Login successful! ✅")
else:
    print("Login failed! ❌")
```

Game State

```
is_game_running = True
has_shield = True
health = 50

if is_game_running:
    if health <= 0:
        is_game_running = False
        print("Game Over!")
    elif has_shield:
        print("Shield is active")
```

Feature Flags

```
debug_mode = True
dark_mode = False
notifications_enabled = True

if debug_mode:
    print("Debugging information shown")

if dark_mode:
    print("🌙 Dark theme active")
else:
    print("☀️ Light theme active")
```

Try It Yourself

- Challenge 1:** Create two Boolean variables and use `and`, `or`, and `not` on them.
- Challenge 2:** Use comparison operators to check if a number is between 10 and 20.
- Challenge 3:** Write an `if` statement that checks if a user is logged in AND has admin privileges.
- Challenge 4:** Use `not` to check if a list is empty.

Chapter Summary

Operator	Name	Example
<code>and</code>	Logical AND	<code>x > 0 and x < 10</code>
<code>or</code>	Logical OR	<code>is_weekend or is_holiday</code>
<code>not</code>	Logical NOT	<code>not is_game_over</code>
<code>==</code>	Equal to	<code>a == b</code>
<code>!=</code>	Not equal	<code>a != b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><</code>	Less than	<code>a < b</code>
<code>>=</code>	Greater or equal	<code>a >= b</code>
<code><=</code>	Less or equal	<code>a <= b</code>

CHAPTER 2.6: Type Conversion – Changing Types

What You'll Learn

- Why you need type conversion
- Converting between int, float, and str
- Converting user input
- Handling conversion errors

Why Type Conversion?

Sometimes you have a value in one type but need it in another:

```
# User input comes as string
age_str = "11"      # string
age_int = 11        # integer

# You can't do math with strings
# age_str + 1 # ERROR! Can't add int to str

# Convert to int first
age = int(age_str)
print(age + 1) # 12
```

Converting to Integer: `int()`

```
# String to int
num = int("123")
print(num)      # 123

# Float to int (truncates)
num = int(3.99)
print(num)      # 3

# Boolean to int
print(int(True))  # 1
print(int(False)) # 0
```

Converting to Float: `float()`

```
# String to float
num = float("3.14")
print(num)      # 3.14

# Int to float
num = float(10)
print(num)      # 10.0

# Boolean to float
print(float(True))  # 1.0
print(float(False)) # 0.0
```

Converting to String: `str()`

```
# Int to string
text = str(42)
print(text)      # "42"

# Float to string
text = str(3.14)
print(text)      # "3.14"

# Boolean to string
print(str(True))  # "True"
print(str(False)) # "False"
```

Converting User Input

`input()` always returns a string:

```
# Without conversion (string math)
age = input("Enter your age: ")
# print(age + 5)  # ERROR! Can't add int to str

# With conversion
age = int(input("Enter your age: "))
print(f"Next year you'll be {age + 1}")
```

Handling Conversion Errors

If conversion fails, Python raises a `ValueError`:

```
# This will cause an error
# num = int("hello") # ValueError!

# Safe way
user_input = input("Enter a number: ")
if user_input.isdigit():
    num = int(user_input)
    print(f"Double is {num * 2}")
else:
    print("That's not a valid number!")
```

Common Conversions

```
# Int to float
x = 10
y = float(x) # 10.0

# Float to int (truncates)
pi = 3.14159
pi_int = int(pi) # 3

# String to int (must be valid number)
num_str = "42"
num = int(num_str) # 42

# String to float
pi_str = "3.14"
pi = float(pi_str) # 3.14

# Number to string
score = 100
message = "Score: " + str(score) # "Score: 100"
```

Automatic Type Conversion (Coercion)

Python automatically converts in some situations:

```
# Int + Float = Float
result = 5 + 3.5    # 8.5 (float)

# Int / Int = Float (unlike Swift!)
result = 10 / 2     # 5.0 (float, not 5!)

# Boolean + Int = Int
result = True + 5   # 6 (True = 1)
```

Type Checking

```
value = 42
print(type(value))    # <class 'int'>

# Check type before converting
if type(value) == int:
    print("It's an integer")
```

Real-World Examples

Simple Calculator

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(f"{num1} + {num2} = {num1 + num2}")
print(f"{num1} * {num2} = {num1 * num2}")
```

Age Validator

```
age_str = input("Enter your age: ")

if age_str.isdigit():
    age = int(age_str)
    if age >= 18:
        print("Adult")
    else:
        print(f"Minor (you can vote in {18 - age} years)")
else:
    print("Please enter a valid number")
```

Temperature Converter

```
celsius_str = input("Enter temperature in Celsius: ")
celsius = float(celsius_str)
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}°C = {fahrenheit}°F")
```

Try It Yourself

- Challenge 1:** Convert the string "99" to an integer and multiply it by 2.
- Challenge 2:** Convert the integer 42 to a float, then to a string.
- Challenge 3:** Ask the user for two numbers, convert to int, and print their sum.
- Challenge 4:** What happens if you try to convert "3.14" to int? What about "hello" to int?

Chapter Summary

Conversion	Syntax	Example
To int	<code>int(value)</code>	<code>int("123")</code> → 123
To float	<code>float(value)</code>	<code>float("3.14")</code> → 3.14
To string	<code>str(value)</code>	<code>str(42)</code> → "42"
Check digit	<code>string.isdigit()</code>	<code>"123".isdigit()</code> → True

PART 2 SUMMARY

You now know all about variables and data types:

Chapter	Topic
2.1	Variables – Storing information
2.2	Integers (int) – Whole numbers
2.3	Floats (float) – Decimal numbers
2.4	Strings (str) – Text
2.5	Booleans (bool) – True or False
2.6	Type Conversion – Changing types

What's Next?

Next up: **Part 3: Input, Output, and Operators** – `print()`, `input()`, and all the operators!

PART 3: INPUT, OUTPUT, AND OPERATORS

CHAPTER 3.1: `print()` – Outputting Information

What You'll Learn

- What `print()` does
- Printing multiple items
- Customizing the separator and end
- Printing to files

What Does `print()` Do?

`print()` outputs text to the console (screen). It's how you see results from your program.


```
print("Hello, World!")  
  
print(42)  
  
print(3.14)  
  
print(True)
```

Printing Multiple Items

Separate items with commas:

```
name = "Ryu"  
age = 11  
  
print("Name:", name, "Age:", age)  
# Output: Name: Ryu Age: 11
```

The `sep` Parameter (Separator)

By default, `print()` puts a space between items. You can change this with `sep`:

```
print("apple", "banana", "orange")  
# Output: apple banana orange  
  
print("apple", "banana", "orange", sep=", ")  
# Output: apple, banana, orange  
  
print("apple", "banana", "orange", sep=" | ")  
# Output: apple | banana | orange  
  
print("a", "b", "c", sep="")  
# Output: abc
```

The `end` Parameter (Line Ending)

By default, `print()` adds a new line at the end. Change it with `end`:

```
print("Hello", end=" ")
print("World!")
# Output: Hello World!

print("Loading", end="...")
print("Done!")
# Output: Loading...Done!

# No new line at all
print("Progress: ", end="")
print("50%")
# Output: Progress: 50%
```

Empty Print (New Line Only)

```
print("First line")
print() # Prints a blank line
print("Third line")

# Output:
# First line
#
# Third line
```

Printing Variables

```
name = "Ryu"
score = 95
is_winner = True

print(name)
print(score)
print(is_winner)
```

Printing with f-strings (Best)

```
name = "Ryu"
age = 11

print(f"My name is {name} and I am {age} years old")
# Output: My name is Ryu and I am 11 years old
```

Formatting Numbers in print

```
pi = 3.14159265359

# 2 decimal places
print(f"Pi: {pi:.2f}")      # Pi: 3.14

# 4 decimal places
print(f"Pi: {pi:.4f}")      # Pi: 3.1416

# Width and alignment
print(f"{'Name':<10} {'Score':>5}")
print(f"{'Ryu':<10} {95:>5}")
print(f"{'Anna':<10} {87:>5}")
# Output:
# Name      Score
# Ryu       95
# Anna      87
```

Printing to a File

```
# Write to a file instead of screen
with open("output.txt", "w") as f:
    print("Hello, file!", file=f)
```

Real-World Examples

Progress Bar

```
import time

print("Loading", end="")

for i in range(5):
    print(".", end="", flush=True)
    time.sleep(0.5)

print(" Done!")
```

Table Output

```
print("-" * 30)

print(f"{'Item':<15} {'Price':>10}")

print("-" * 30)

print(f"{'Apple':<15} {'$1.50':>10}")

print(f"{'Banana':<15} {'$0.99':>10}")

print(f"{'Orange':<15} {'$1.25':>10}")

print("-" * 30)
```

Debugging with Print

```
def calculate(x, y):
    print(f"DEBUG: x={x}, y={y}") # Debug output
    result = x + y
    print(f"DEBUG: result={result}")
    return result

calculate(5, 3)
```

Try It Yourself

Challenge 1: Print your name, age, and city on one line separated by " | ".

Challenge 2: Print numbers 1 to 5 on the same line with spaces between them.

Challenge 3: Create a small table with 3 columns: Name, Score, Grade.

Challenge 4: Use `print()` with `end=""` to create a countdown from 3 to 1 on the same line.

Chapter Summary

Parameter	Default	Use
<code>sep</code>	<code>" "</code>	Separator between items
<code>end</code>	<code>"\n"</code>	What to print at the end
<code>file</code>	<code>sys.stdout</code>	Where to print (file or console)

CHAPTER 3.2: input() – Getting User Input

What You'll Learn

- How to get input from the user
- The `input()` function
- Converting input to numbers
- Handling invalid input

What Does input() Do?

`input()` pauses your program and waits for the user to type something. When they press Enter, it returns what they typed as a **string**.

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

Basic Input

```
# Without prompt
name = input()
print(f"Hello, {name}")

# With prompt
name = input("Enter your name: ")
print(f"Welcome, {name}!")
```

input() Always Returns a String

```
age = input("Enter your age: ")
print(type(age))  # <class 'str'>

# You cannot do math with it directly
# next_age = age + 1  # ERROR!

# Convert to int first
age_int = int(age)
next_age = age_int + 1
print(f"Next year you'll be {next_age}")
```

Converting Input to Numbers

```
# Integer input
age = int(input("Enter your age: "))

# Float input
height = float(input("Enter your height in meters: "))

print(f"Age: {age}, Height: {height}")
```

Handling Invalid Input (Simple)

```
user_input = input("Enter a number: ")

if user_input.isdigit():
    number = int(user_input)
    print(f"Double: {number * 2}")
else:
    print("That's not a valid number!")
```

Handling Invalid Input (try-except)

We'll cover `try-except` in detail in Part 14, but here's a preview:

```
try:
    age = int(input("Enter your age: "))
    print(f"Next year you'll be {age + 1}")
except ValueError:
    print("Please enter a valid number!")
```

Multiple Inputs on One Line

```
# User types: "10 20"
x, y = input("Enter two numbers: ").split()
x = int(x)
y = int(y)
print(f"Sum: {x + y}")

# With custom separator
x, y = input("Enter two numbers (comma-separated): ").split(",")
x = int(x)
y = int(y)
print(f"Product: {x * y}")
```

Real-World Examples

Greeting Program

```
name = input("What's your name? ")
age = input("How old are you? ")

print(f"\nHello, {name}!")
print(f"You are {age} years old.")
print(f"Next year you'll be {int(age) + 1}")
```

Simple Calculator

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(f"{num1} + {num2} = {num1 + num2}")
print(f"{num1} - {num2} = {num1 - num2}")
print(f"{num1} * {num2} = {num1 * num2}")
print(f"{num1} / {num2} = {num1 / num2}")
```

Password Checker (Simple)

```
password = input("Enter password: ")

if password == "swift123":
    print("Access granted! ✅")
else:
    print("Access denied! ❌")
```

Mad Libs Game

```
print("=== MAD LIBS ===")

name = input("Enter a name: ")
place = input("Enter a place: ")
animal = input("Enter an animal: ")
food = input("Enter a food: ")

story = f"""
One day, {name} went to {place}.
Suddenly, a {animal} appeared!
It wanted to eat {name}'s {food}.
But {name} was too fast and escaped!
"""

print(story)
```

Temperature Converter

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}°C = {fahrenheit}°F")
```


Try It Yourself

- Challenge 1:** Ask the user for their name and age, then print a greeting.
- Challenge 2:** Create a program that asks for two numbers and prints their sum.
- Challenge 3:** Ask for a word and print it 5 times.
- Challenge 4:** Create a simple login system that checks username and password.

Chapter Summary

Function	What It Does
<code>input()</code>	Reads a line of user input as a string
<code>input("prompt")</code>	Displays prompt before reading
<code>int(input())</code>	Gets integer input
<code>float(input())</code>	Gets float input
<code>.split()</code>	Splits input into multiple values

CHAPTER 3.3: Arithmetic Operators – Doing Math

What You'll Learn

- All arithmetic operators in Python
- Order of operations
- Integer vs float division
- Modulo (remainder)
- Exponents

Arithmetic Operators

Operator	Name	Example	Result
<code>+</code>	Addition	<code>10 + 3</code>	<code>13</code>
<code>-</code>	Subtraction	<code>10 - 3</code>	<code>7</code>
<code>*</code>	Multiplication	<code>10 * 3</code>	<code>30</code>
<code>/</code>	Division (float)	<code>10 / 3</code>	<code>3.333...</code>
<code>//</code>	Floor division (integer)	<code>10 // 3</code>	<code>3</code>
<code>%</code>	Modulo (remainder)	<code>10 % 3</code>	<code>1</code>
<code>**</code>	Exponentiation (power)	<code>10 ** 3</code>	<code>1000</code>

Basic Operations

```
a = 15
b = 4

print(a + b)    # 19
print(a - b)    # 11
print(a * b)    # 60
print(a / b)    # 3.75
print(a // b)   # 3
print(a % b)    # 3
print(a ** b)   # 50625 (15^4)
```

Division: `/` vs `//`

```
# / always returns a float
print(10 / 3)    # 3.3333333333333335
print(10 / 2)    # 5.0 (float, not 5!)
print(7 / 1)     # 7.0

# // returns floor (integer division)
print(10 // 3)   # 3
print(10 // 2)   # 5
print(7 // 1)    # 7
print(-10 // 3)  # -4 (floor rounds DOWN)
```

Modulo (Remainder)

```
print(10 % 3)    # 1 (10 ÷ 3 = 3 remainder 1)
print(17 % 5)    # 2 (17 ÷ 5 = 3 remainder 2)
print(20 % 4)    # 0 (20 ÷ 4 = 5 remainder 0)
```

Exponentiation

```
print(2 ** 3)    # 8 (2 × 2 × 2)
print(5 ** 2)    # 25 (5 squared)
print(10 ** 4)   # 10000 (10 to the 4th)

# Square root (power of 0.5)
print(16 ** 0.5) # 4.0
print(2 ** 0.5)  # 1.4142135623730951 (√2)
```

Order of Operations (PEMDAS)

Python follows the standard math order:

1. **Parentheses** 
2. **Exponents** 
3. **Multiplication**  and **Division** , , 
4. **Addition**  and **Subtraction** 

```
result = 2 + 3 * 4
print(result) # 14 (3*4=12, then 2+12=14)

result = (2 + 3) * 4
print(result) # 20 (2+3=5, then 5*4=20)

result = 2 + 3 ** 2
print(result) # 11 (3**2=9, then 2+9=11)

result = (2 + 3) ** 2
print(result) # 25 (5**2=25)
```

Combining Operators

```
a = 10
b = 3
c = 5

result = (a + b) * c / 2
print(result)  # (13 * 5) / 2 = 65 / 2 = 32.5
```

Real-World Examples

Circle Area

```
radius = 5
area = 3.14159 * radius ** 2
print(f"Area: {area}")  # Area: 78.53975
```

Convert Seconds

```
total_seconds = 3665
hours = total_seconds // 3600
minutes = (total_seconds % 3600) // 60
seconds = total_seconds % 60

print(f"{hours}:{minutes:02d}:{seconds:02d}")  # 1:01:05
```

Even or Odd

```
number = 17
if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")  # 17 is odd
```

Simple Interest

```
principal = 1000
rate = 0.05
years = 3

interest = principal * rate * years
total = principal + interest
print(f"Interest: ${interest:.2f}")
print(f"Total: ${total:.2f}")
```

Average of Three Numbers

```
a = 85
b = 92
c = 78

average = (a + b + c) / 3
print(f"Average: {average:.2f}") # Average: 85.00
```

Try It Yourself

Challenge 1: Calculate the area of a rectangle (width * height).

Challenge 2: Convert 145 minutes into hours and minutes.

Challenge 3: Check if a number is divisible by 7 using `%`.

Challenge 4: Calculate 2^{10} (2 to the 10th power) two ways: with `**` and with a loop.

Chapter Summary

Operator	Name	Example
<code>+</code>	Addition	<code>5 + 3 = 8</code>
<code>-</code>	Subtraction	<code>10 - 4 = 6</code>
<code>*</code>	Multiplication	<code>6 * 7 = 42</code>
<code>/</code>	Float division	<code>10 / 3 = 3.333</code>
<code>//</code>	Floor division	<code>10 // 3 = 3</code>
<code>%</code>	Modulo	<code>10 % 3 = 1</code>
<code>**</code>	Exponentiation	<code>2 ** 3 = 8</code>

CHAPTER 3.4: Comparison Operators – Comparing Values

What You'll Learn

- How to compare values
- All comparison operators
- Comparison results are Booleans
- Comparing strings

What Are Comparison Operators?

Comparison operators compare two values and return a Boolean (`True` or `False`).

Operator	Name	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>True</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>True</code>
<code><</code>	Less than	<code>3 < 5</code>	<code>True</code>
<code>>=</code>	Greater than or equal	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	Less than or equal	<code>3 <= 5</code>	<code>True</code>

Equal To: ==

```
print(5 == 5)      # True
print(5 == 3)      # False
print("hi" == "hi") # True
print("hi" == "HI") # False (case-sensitive)
```

Not Equal To: !=

```
print(5 != 3)      # True
print(5 != 5)      # False
print("hi" != "hello") # True
```

Greater Than: > and Less Than: <

```
print(10 > 5)      # True
print(10 > 10)     # False
print(5 < 10)      # True
print(5 < 5)      # False
```

Greater/Equal: >= and Less/Equal: <=

```
print(10 >= 10)    # True
print(10 >= 5)     # True
print(5 <= 10)     # True
print(5 <= 5)     # True
```

Comparing Strings

Python compares strings alphabetically (by Unicode values):

```
print("apple" == "apple")    # True
print("apple" != "orange")  # True

print("apple" < "banana")    # True ('a' comes before 'b')
print("cat" < "dog")         # True
print("zebra" > "apple")     # True

print("Apple" < "apple")     # True (uppercase comes before lowercase)
```

Chaining Comparisons

Python allows chaining:

```
x = 5

# These are the same:
print(1 < x < 10)           # True
print(1 < x and x < 10)     # True

print(0 < x < 5)            # False (5 is not < 5)
```

Comparing Different Types

```
# Python 3 allows comparing int and float
print(5 == 5.0)    # True
print(5 > 3.5)     # True

# But comparing int/float with string causes error
# print(5 == "5")  # False? Actually this works but gives False (no error)
print(5 == "5")    # False (different types)
```


Using Comparisons in Conditions

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B") # This runs
elif score >= 70:
    print("C")
else:
    print("F")
```

Real-World Examples

Age Check

```
age = 16

if age >= 18:
    print("You can vote")
else:
    print(f"You can vote in {18 - age} years")
```

Temperature Alert

```
temp = 35

if temp >= 100:
    print("Boiling!")
elif temp <= 0:
    print("Freezing!")
else:
    print("Normal temperature")
```

Login Check

```
username = input("Username: ")
password = input("Password: ")

if username == "ryuu" and password == "swift123":
    print("Login successful!")
else:
    print("Login failed!")
```

Grade Calculator

```
score = 87

if score >= 90:
    print("A")
elif score >= 80:
    print("B")    # B for 87
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

Try It Yourself

Challenge 1: Check if a number is positive, negative, or zero.

Challenge 2: Compare two strings and print if they are the same.

Challenge 3: Write code that prints "Teenager" if age is between 13 and 19 inclusive.

Challenge 4: Create a password strength checker that checks length ≥ 8 .

Chapter Summary

Operator	Meaning	True Example
<code>==</code>	Equal to	<code>5 == 5</code>
<code>!=</code>	Not equal	<code>5 != 3</code>
<code>></code>	Greater than	<code>5 > 3</code>
<code><</code>	Less than	<code>3 < 5</code>
<code>>=</code>	Greater/equal	<code>5 >= 5</code>
<code><=</code>	Less/equal	<code>3 <= 5</code>

CHAPTER 3.5: Logical Operators – Combining Conditions

What You'll Learn

- The `and`, `or`, and `not` operators
- Combining multiple conditions
- Short-circuit evaluation
- Truth tables

What Are Logical Operators?

Logical operators combine Boolean values to produce new Boolean values.

Operator	Name	True When
<code>and</code>	AND	Both are True
<code>or</code>	OR	At least one is True
<code>not</code>	NOT	The value is False

and - Both Must Be True

```
age = 11
has_permission = True

if age >= 10 and has_permission:
    print("You can join")  # This runs (both True)
```

Truth table:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

or - At Least One Must Be True

```
is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("No school!")  # This runs (is_weekend is True)
```

Truth table:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

`not` - Reverses the Value

```
is_game_over = False

if not is_game_over:
    print("Keep playing!") # This runs (not False = True)
```

Truth table:

A	not A
True	False
False	True

Combining Multiple Operators

```
age = 16
has_license = True
has_permit = False

# Can drive if: (age >= 16 AND has_license) OR (age >= 15 AND has_permit)
if (age >= 16 and has_license) or (age >= 15 and has_permit):
    print("You can drive") # This runs
```

Operator Precedence

`not` has highest precedence, then `and`, then `or`:

```
# Without parentheses (not clear)
result = True or False and False
# Step 1: False and False = False
# Step 2: True or False = True

# With parentheses (clear)
result = True or (False and False) # True
```

Use parentheses to be clear!

Short-Circuit Evaluation

Python stops evaluating as soon as it knows the result:

```
def expensive_check():
    print("Doing expensive check...")
    return True

is_fast = False
result = is_fast and expensive_check()
# expensive_check() NEVER runs because is_fast is False
```

Real-World Examples

Login Validation

```
username = "ryuu"
password = "swift123"
is_locked = False

if not is_locked and username == "ryuu" and password == "swift123":
    print("Login successful! ✅")
else:
    print("Login failed! ❌")
```

Game Attack Conditions

```
has_sword = True
has_magic = False
has_stamina = True
is_stunned = False

if (has_sword or has_magic) and has_stamina and not is_stunned:
    print("You can attack! 🗡️")
else:
    print("Cannot attack")
```

Eligibility Check

```
age = 11
is_member = False
has_parental_consent = True

if age >= 18 or (age >= 13 and is_member) or (age >= 10 and has_parental_consent):
    print("You can join the club!")
else:
    print("You cannot join")
```

Weather Decision

```
is_raining = False
is_cold = True
has_umbrella = True
has_coat = False

if (not is_raining or has_umbrella) and (not is_cold or has_coat):
    print("Let's go outside!")
else:
    print("Better stay inside")
```

Try It Yourself

- Challenge 1:** Write a condition that checks if a number is between 10 and 20 (inclusive).
- Challenge 2:** Check if a year is a leap year (divisible by 4, not by 100 unless also by 400).
- Challenge 3:** Create a login system that checks username AND password.
- Challenge 4:** Use `not` to check if a list is empty.

Chapter Summary

Operator	Name	Example
<code>and</code>	AND	<code>x > 0 and x < 10</code>
<code>or</code>	OR	<code>is_weekend or is_holiday</code>
<code>not</code>	NOT	<code>not is_game_over</code>

CHAPTER 3.6: Assignment Operators – Shorthand Math

What You'll Learn

- What assignment operators are
- Shorthand for updating variables
- All assignment operators

What Are Assignment Operators?

Assignment operators combine math operations with assignment. They update a variable in place.

Operator	Meaning	Example	Result
=	Assign	x = 5	x becomes 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 3	x = x - 3
*=	Multiply and assign	x *= 3	x = x * 3
/=	Divide and assign	x /= 3	x = x / 3
//=	Floor divide and assign	x //= 3	x = x // 3
%=	Modulo and assign	x %= 3	x = x % 3
**=	Exponent and assign	x **= 3	x = x ** 3

Basic Examples

```
# Without shorthand (long way)
score = 10
score = score + 5
print(score)  # 15

# With shorthand (shorter!)
score = 10
score += 5
print(score)  # 15
```


Addition Assignment: +=

```
score = 0
score += 10    # 10
score += 25    # 35
score += 5     # 40
print(score)  # 40
```

Subtraction Assignment: -=

```
health = 100
health -= 30   # 70
health -= 15   # 55
print(health) # 55
```

Multiplication Assignment: *=

```
points = 10
points *= 2    # 20
points *= 3    # 60
print(points) # 60
```

Division Assignment: /= and //=

```
value = 100
value /= 4    # 25.0
print(value)

value = 100
value //= 4    # 25
print(value)
```

Modulo Assignment: %=

```
number = 17
number %= 5    # 2
print(number)

counter = 24
counter %= 7   # 3
print(counter)
```

Exponentiation Assignment: **=

```
value = 2
value **= 3    # 8
print(value)

value = 5
value **= 2    # 25
print(value)
```

Real-World Examples

Game Score

```
score = 0

# Player collects coins
score += 10
score += 25
score += 5

# Player loses points
score -= 15

print(f"Final score: {score}") # 25
```

Player Health

```
health = 100

# Take damage
health -= 20
health -= 15

# Heal
health += 30

print(f"Health: {health}") # 95
```

Countdown Timer

```
timer = 10

while timer > 0:
    print(timer)
    timer -= 1
print("Blast off! 🚀")
```

Shopping Cart

```
total = 0.0

total += 19.99 # Book
total += 4.50  # Pen
total += 12.99 # Notebook

# Apply 10% discount
total *= 0.9

print(f"Total: ${total:.2f}") # $33.73
```

Level Progression

```
level = 1
experience = 0

experience += 45
experience += 30
experience += 40

if experience >= 100:
    level += 1
    experience -= 100
    print(f"Level up! Now level {level}")
```

Assignment Operators with Strings

`+=` also works with strings (concatenation):

```
message = "Hello"
message += " "
message += "World!"
print(message)  # "Hello World!"

# Other operators ( -=, *=, etc.) don't work with strings
```

Try It Yourself

Challenge 1: Start with `score = 0`. Add 10, then add 25, then subtract 5. Print the final score.

Challenge 2: Start with `health = 100`. Apply 3 hits of 15 damage each, then heal 25.

Challenge 3: Start with `total = 50.0`. Apply a 20% discount (multiply by 0.8).

Challenge 4: Start with `count = 1`. Double it 3 times (multiply by 2 each time).

Chapter Summary

Operator	Meaning	Example
<code>+=</code>	Add and assign	<code>x += 5</code>
<code>-=</code>	Subtract and assign	<code>x -= 3</code>
<code>*=</code>	Multiply and assign	<code>x *= 2</code>
<code>/=</code>	Divide and assign	<code>x /= 4</code>
<code>//=</code>	Floor divide and assign	<code>x //= 2</code>
<code>%=</code>	Modulo and assign	<code>x %= 5</code>
<code>**=</code>	Exponent and assign	<code>x **= 3</code>

PART 3 SUMMARY

You now know all about input, output, and operators:

Chapter	Topic
3.1	<code>print()</code> – Outputting information
3.2	<code>input()</code> – Getting user input
3.3	Arithmetic Operators – Doing math
3.4	Comparison Operators – Comparing values
3.5	Logical Operators – Combining conditions
3.6	Assignment Operators – Shorthand math

What's Next?

Next up: **Part 4: Control Flow** – `if`, `elif`, `else`, and `match-case` !

PART 4: CONTROL FLOW

CHAPTER 4.1: if Statements – Making Decisions

What You'll Learn

- What an `if` statement does
- How to write conditions
- Indentation matters in Python
- When to use `if`

What Is an if Statement?

An `if` statement lets your code make decisions. It says:

"If this condition is true, run this code. If not, skip it."

```
age = 11

if age < 18:
    print("You are a minor")

print("This always runs")
```

Output:

```
You are a minor
This always runs
```

Basic Syntax

```
if condition:
    # code runs ONLY if condition is True
    print("Condition was true")
```

Important: The colon `:` and the indentation are required!

Indentation Matters

In Python, indentation is **not optional** — it tells Python what code belongs to the `if`:

```
# ✅ Correct
if age >= 18:
    print("You can vote")
    print("This is also part of the if")

print("This is outside the if")

# ❌ Wrong (missing indentation)
if age >= 18:
print("This will cause an IndentationError")
```

Use **4 spaces** for indentation (most Python programmers use 4 spaces, not tabs).

Conditions Must Be Boolean

The condition inside `if` must evaluate to `True` or `False`:

```
score = 100

# ✅ Correct (comparison returns bool)
if score > 50:
    print("Great score!")

# ❌ Wrong (int is not a bool)
# if score:
#     print("This works but is confusing!")
```

Simple Examples

Checking a Number

```
temperature = 30

if temperature > 25:
    print("It's hot outside! 🌞")
```

Checking a Boolean

```
is_logged_in = True

if is_logged_in:
    print("Welcome back!")
```

Using `not`

```
is_game_over = False

if not is_game_over:
    print("Keep playing! 🎮")
```

Multiple Conditions with `and`

```
age = 11
has_permission = True

if age >= 10 and has_permission:
    print("You can join the club")
```

Multiple Conditions with `or`

```
is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("No school today! 🎉")
```

Real-World Examples

Temperature Check

```
temperature = 35

if temperature > 30:
    print("It's hot! Turn on the AC")
```


Login Check

```
username = "ryuu"
password = "swift123"

if username == "ryuu" and password == "swift123":
    print("Login successful!")
```

Age Check

```
age = 11

if age >= 18:
    print("You can vote")
```

Empty Check

```
name = input("Enter your name: ")

if name == "":
    print("You didn't enter a name!")
```

Try It Yourself

- Challenge 1:** Write an `if` statement that prints "You are an adult" if `age` is 18 or greater.
- Challenge 2:** Write an `if` statement that prints "It's freezing!" if `temperature` is 32 or below.
- Challenge 3:** Write an `if` statement that prints "Welcome!" if `is_logged_in` is True.
- Challenge 4:** Write an `if` statement that prints "You qualify" if `score` > 100 and `level` > 5.

Chapter Summary

Concept	Syntax	Example
if statement	<code>if condition:</code>	<code>if age >= 18:</code>
Indentation	4 spaces	Code inside if
Condition	Must be Boolean	<code>score > 50</code>

CHAPTER 4.2: if-else – Two Paths

What You'll Learn

- What `else` does
- How to handle both true and false cases
- The `if-else` syntax

The Problem `else` Solves

With just `if`, you can only run code when a condition is true. What about when it's false?

```
age = 11

# This only handles the true case
if age >= 18:
    print("You can vote")
# Nothing happens when age < 18
```

The `else` Clause

`else` lets you run code when the condition is false:

```
age = 11

if age >= 18:
    print("You can vote")
else:
    print("Too young to vote")

# Output: "Too young to vote"
```

Syntax

```
if condition:
    # Runs when condition is True
    print("True path")
else:
    # Runs when condition is False
    print("False path")
```

Only One Block Runs

Exactly one block runs — never both, never neither.

```
is_raining = True

if is_raining:
    print("Take an umbrella ☔")
else:
    print("Enjoy the sun ☀️")

# Output: "Take an umbrella ☔"
```

Real-World Examples

Age Check

```
age = 16

if age >= 18:
    print("You can vote")
else:
    years_left = 18 - age
    print(f"You can vote in {years_left} years")
```

Login System

```
password = input("Enter password: ")

if password == "swift123":
    print("Access granted ✅")
else:
    print("Access denied ❌")
```

Even or Odd

```
number = 7

if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd") # 7 is odd
```

Game Win/Loss

```
player_score = 85
computer_score = 90

if player_score > computer_score:
    print("You win! 🏆")
else:
    print("You lose! Better luck next time 🍀")
```

Temperature Check

```
temperature = 35

if temperature > 30:
    print("It's hot! Turn on AC 🌡️")
else:
    print("Temperature is comfortable 👍")
```

Multiple Statements Inside else

You can put as much code as you want inside the `else` block:

```
temperature = -5

if temperature > 0:
    print("Above freezing")
else:
    print("Below freezing!")
    print("Wear a heavy coat 🧥")
    print("Watch for ice on roads ⚠️")
```

Nested if-else

You can put an `if-else` inside another `if-else`:

```
score = 85

if score >= 60:
    print("You passed!")
    if score >= 90:
        print("Excellent!")
    else:
        print("Good job!")
else:
    print("You failed.")
    print("Keep practicing!")
```

Try It Yourself

Challenge 1: Write an `if-else` that prints "Good morning!" if hour < 12, else "Good afternoon!".

Challenge 2: Write code that checks if password equals "secret". If yes, print "Access granted". If no, print "Access denied".

Challenge 3: Check if a number is positive or negative.

Challenge 4: Write code that prints "You have enough points" if points >= 100, else prints how many more points needed.

Chapter Summary

Concept	Syntax	Use
if-else	<code>if condition: else:</code>	Two-path decision
True path	<code>if</code> block	Runs when condition is True
False path	<code>else</code> block	Runs when condition is False

CHAPTER 4.3: elif – Multiple Conditions

What You'll Learn

- What `elif` does (else if)
- Chaining multiple conditions
- Order matters
- Creating complex decision trees

The Problem `elif` Solves

With just `if` and `else`, you can only handle two paths. What if you need three or more?

```
score = 75

# This doesn't work well
if score >= 90:
    print("A")
if score >= 80:
    print("B") # Would run for 75? No.
```

The `elif` Syntax

```
if condition1:
    # Runs if condition1 is True
elif condition2:
    # Runs if condition1 is False AND condition2 is True
elif condition3:
    # Runs if all above are False AND condition3 is True
else:
    # Runs if all conditions are False
```

How It Works

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B") # This runs (85 >= 80)
elif score >= 70:
    print("C")
else:
    print("F")

# Output: "B"
```

Once a condition is True, Python stops checking the rest.

Multiple `elif` Blocks

You can have as many `elif` blocks as you want:

```
grade = 87

if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")    # Runs for 87
elif grade >= 70:
    print("C")
elif grade >= 60:
    print("D")
else:
    print("F")
```

Order Matters!

Put the most specific conditions first:

```
score = 95

# ❌ WRONG ORDER
if score >= 80:
    print("B or higher")    # This runs first! (95 >= 80)
elif score >= 90:
    print("A")    # Never runs
# Output: "B or higher"

# ✅ CORRECT ORDER
if score >= 90:
    print("A")    # Runs first
elif score >= 80:
    print("B")
# Output: "A"
```


Real-World Examples

Temperature Categories

```
temp = 25

if temp <= 0:
    print("Freezing ❄️ ")
elif temp <= 10:
    print("Cold 🧊 ")
elif temp <= 20:
    print("Cool 🍃 ")
elif temp <= 30:
    print("Warm ☀️ ") # This runs for 25
else:
    print("Hot 🥵 ")
```

Letter Grade

```
score = 72

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C") # 72 is between 70-79
elif score >= 60:
    print("D")
else:
    print("F")
```

Age Group

```
age = 11

if age < 2:
    print("Baby 🍼")
elif age < 5:
    print("Toddler 🧒")
elif age < 13:
    print("Child 🧒") # This runs
elif age < 20:
    print("Teenager 🧑🎓")
else:
    print("Adult 🧑")
```

BMI Categories

```
weight = 68.0
height = 1.75
bmi = weight / (height ** 2)

if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal weight")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

Time of Day

```
hour = 14 # 2 PM

if hour < 12:
    print("Good morning")
elif hour < 17:
    print("Good afternoon") # This runs
elif hour < 20:
    print("Good evening")
else:
    print("Good night")
```

Try It Yourself

- Challenge 1:** Write an `if-elif-else` that prints "Small" for 1-10, "Medium" for 11-20, "Large" for 21-30, "Too big" for anything else.
- Challenge 2:** Given a speed limit of 65, print "Under limit", "At limit", "Over limit (warning)", or "Over limit (ticket)" for speeds 5, 10, 15+ mph over.
- Challenge 3:** Categorize a restaurant bill: "Cheap" (<\$10), "Moderate" (\$10-25), "Expensive" (\$25-50), "Very Expensive" (>\$50).

Chapter Summary

Concept	Syntax	Use
elif	<code>elif condition:</code>	Check another condition
Order matters	Most specific first	Prevents wrong matches
else	<code>else:</code>	Runs if no conditions True

CHAPTER 4.4: Nested Conditions – Conditions Inside Conditions

What You'll Learn

- What nested conditions are
- Checking multiple levels of conditions

- When nesting is useful
- Avoiding too much nesting

What Is a Nested Condition?

A **nested condition** is an `if` statement inside another `if` or `else` block.

```
age = 11
has_permission = True

if age >= 10:
    print("Age requirement met")

    if has_permission:
        print("Permission granted! You can join")
    else:
        print("Need permission to join")
else:
    print("Too young to join")
```

Why Use Nested Conditions?

Sometimes you only want to check a second condition **if** the first condition is true:

```
is_logged_in = True
is_admin = True

if is_logged_in:
    # Only check admin status if logged in
    if is_admin:
        print("Welcome, admin! 🌟")
    else:
        print("Welcome, user")
else:
    print("Please log in")
```

Multiple Levels of Nesting

You can nest as deeply as needed (but don't go too deep!):

```
is_weekend = True
is_sunny = True
temperature = 28

if is_weekend:
    print("It's the weekend!")

    if is_sunny:
        print("And it's sunny! ☀️")

        if temperature > 25:
            print("Perfect beach weather! 🏖️")
        else:
            print("Nice day for a walk 🚶")
    else:
        print("Too bad it's not sunny 😞")
else:
    print("It's a weekday. Time to work 💼")
```

Nested if with elif

You can combine nesting with `elif`:

```
score = 85
has_bonus = True

if score >= 80:
    print("Great score!")

    if has_bonus:
        print("Bonus round unlocked! 🎮")
        total = score + 20
        print(f"Final score: {total}")
    else:
        print(f"Final score: {score}")
elif score >= 60:
    print("Good effort")
else:
    print("Keep practicing")
```

The Arrow Code Problem

Too much nesting creates "arrow code" — hard to read and maintain:

```
# ❌ Arrow code (too nested)
if condition1:
    if condition2:
        if condition3:
            if condition4:
                # Code buried deep inside
                print("Too deep!")
```

How to Fix Deep Nesting

Solution 1: Combine conditions with `and`

```
# Instead of:
if is_logged_in:
    if has_permission:
        if not is_banned:
            print("Access granted")

# Do this:
if is_logged_in and has_permission and not is_banned:
    print("Access granted")
```

Solution 2: Use `elif` for mutually exclusive conditions

```
# Instead of nested if-else
if a:
    if b:
        # A and B
    else:
        # A but not B

# Use elif
if a and b:
    # A and B
elif a and not b:
    # A but not B
```

Real-World Examples

Order Processing

```
is_member = True
total = 150
years_member = 3

if is_member:
    print("Member discount applied")

    if total > 100:
        print("Extra bulk discount!")

        if years_member > 5:
            print("Loyalty discount added!")
            discount = 0.25
        else:
            discount = 0.20
    else:
        discount = 0.10
else:
    print("Join membership for discounts!")
    discount = 0

final_price = total * (1 - discount)
print(f"Final price: ${final_price:.2f}")
```

Game Attack System

```
has_sword = True
has_shield = False
enemy_health = 30
player_health = 50

if enemy_health > 0:
    print("Battle continues!")

    if has_sword:
        print("Swing sword! 🗡️")
        enemy_health -= 25
    elif has_shield:
        print("Shield bash! 🛡️")
        enemy_health -= 10
    else:
        print("Punch! 👊")
        enemy_health -= 5
else:
    print("Victory! 🏆")
```

Try It Yourself

- Challenge 1:** Write nested if statements that check if a number is positive, then if it's also even.
- Challenge 2:** Write code that checks if a user is logged in, then if they have admin privileges, then print "Admin panel access".
- Challenge 3:** Create a nested decision tree: if it's raining, check if you have an umbrella; if you don't, check if you have a hood.

Chapter Summary

Concept	Syntax	Use
Nested if	<code>if a: if b:</code>	Check second condition only after first
Too deep	More than 3 levels	Hard to read
Flatten	Use <code>and</code>	Combine related conditions

CHAPTER 4.5: match-case – Pattern Matching (Python 3.10+)

What You'll Learn

- What `match-case` is (Python's version of switch)
- How to match single values
- Matching with guards
- Matching multiple patterns

What Is match-case?

`match-case` was introduced in Python 3.10. It's like a more powerful version of `if-elif-else` for comparing one value against many patterns.

```
grade = "B"

match grade:
    case "A":
        print("Excellent!")
    case "B":
        print("Good job!") # This runs
    case "C":
        print("Fair")
    case "D":
        print("Needs improvement")
    case _:
        print("Invalid grade")
```

Basic Syntax

```
match value:
    case pattern1:
        # code
    case pattern2:
        # code
    case _:
        # default (like else)
```

The underscore `_` is a wildcard that matches anything (like `default`).

Matching Single Values

```
day = "Tuesday"

match day:
    case "Monday":
        print("Start of week 🤝")
    case "Tuesday":
        print("Second day")
    case "Wednesday":
        print("Halfway there 🎯")
    case "Thursday":
        print("Almost Friday")
    case "Friday":
        print("TGIF! 🍷")
    case "Saturday" | "Sunday":
        print("Weekend! 🏖️")
    case _:
        print("Invalid day")
```

Matching Multiple Patterns with `|`

```
character = "a"

match character:
    case "a" | "e" | "i" | "o" | "u":
        print("Vowel")
    case _:
        print("Consonant")
```

Matching with Guards (`if`)

You can add extra conditions using `if`:

```
number = 42

match number:
    case x if x < 0:
        print(f"Negative: {x}")
    case 0:
        print("Zero")
    case x if x % 2 == 0:
        print(f"Positive even: {x}")
    case x:
        print(f"Positive odd: {x}")
```

Matching Sequences (Lists/Tuples)

```
point = (3, 5)

match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"On y-axis at y={y}")
    case (x, 0):
        print(f"On x-axis at x={x}")
    case (x, y):
        print(f"Point at ({x}, {y})")
```

Matching with Variables

You can capture values into variables:

```
command = "move 10 20"

match command.split():
    case ["move", x, y]:
        print(f"Moving to ({x}, {y})")
    case ["attack", target]:
        print(f"Attacking {target}")
    case ["quit"]:
        print("Goodbye!")
    case _:
        print("Unknown command")
```

Real-World Examples

HTTP Status Codes

```
status = 404

match status:
    case 200:
        print("OK ✅")
    case 201:
        print("Created")
    case 301 | 302:
        print("Redirected")
    case 400:
        print("Bad Request")
    case 401:
        print("Unauthorized")
    case 403:
        print("Forbidden")
    case 404:
        print("Not Found 🔍")
    case 500:
        print("Server Error")
    case _:
        print(f"Unknown status: {status}")
```

Calculator Operation

```
operation = "+"

a = 10
b = 5

match operation:
    case "+":
        print(f"{a} + {b} = {a + b}")
    case "-":
        print(f"{a} - {b} = {a - b}")
    case "*":
        print(f"{a} * {b} = {a * b}")
    case "/":
        if b != 0:
            print(f"{a} / {b} = {a / b}")
        else:
            print("Cannot divide by zero")
    case _:
        print("Unknown operation")
```

Shape Matching

```
def describe(shape):
    match shape:
        case []:
            return "Empty list"
        case [x]:
            return f"Single element: {x}"
        case [x, y]:
            return f"Pair: ({x}, {y})"
        case [x, y, z]:
            return f"Triple: ({x}, {y}, {z})"
        case _:
            return f"List with {len(shape)} elements"

print(describe([1, 2, 3])) # Triple: (1, 2, 3)
print(describe([1, 2]))   # Pair: (1, 2)
print(describe([1]))      # Single element: 1
print(describe([1, 2, 3, 4])) # List with 4 elements
```

match-case vs if-elif

Feature	match-case	if-elif
Readability	Clean for many cases	Clean for complex conditions
Pattern matching	✔ Powerful	✗ Limited
Available from	Python 3.10+	All Python versions
Best for	Value matching	Complex boolean conditions

Try It Yourself

- Challenge 1:** Write a `match-case` for numbers 1-7 that prints the day of the week.
- Challenge 2:** Write a `match-case` for a traffic light color ("red", "yellow", "green") that prints the action.
- Challenge 3:** Use `match-case` to handle different pet types ("dog", "cat", "bird", "fish") with different messages.

Chapter Summary

Pattern	Syntax	Matches
Single value	<code>case value:</code>	Exact match
Multiple	<code>case a b:</code>	Either a or b
Wildcard	<code>case _:</code>	Anything (default)
Guard	<code>case x if condition:</code>	With extra condition

CHAPTER 4.6: Conditional Expressions – Ternary Operator

What You'll Learn

- What conditional expressions are (one-line if-else)
- The ternary operator syntax
- When to use it (and when not to)

What Is a Conditional Expression?

A **conditional expression** (ternary operator) is a one-line shorthand for `if-else` that returns a value.

```
# Long way
if age >= 18:
    status = "Adult"
else:
    status = "Minor"

# Short way (ternary)
status = "Adult" if age >= 18 else "Minor"
```

Syntax

```
value_if_true if condition else value_if_false
```

- If `condition` is `True`, use `value_if_true`
- If `condition` is `False`, use `value_if_false`

Basic Examples

```
age = 11
result = "Adult" if age >= 18 else "Minor"
print(result)  # "Minor"

score = 85
grade = "Pass" if score >= 60 else "Fail"
print(grade)  # "Pass"

temperature = 30
message = "Hot" if temperature > 25 else "Cool"
print(message)  # "Hot"
```

Using in print Statements

```
age = 11

print("Adult" if age >= 18 else "Minor")

score = 95

print("You passed!" if score >= 60 else "You failed!")
```

Using in f-strings

```
score = 85

message = f"You {'passed' if score >= 60 else 'failed'} the test!"

print(message)  # "You passed the test!"
```

Finding Maximum/Minimum

```
a = 10

b = 20

max_value = a if a > b else b
min_value = a if a < b else b

print(f"Max: {max_value}")  # 20
print(f"Min: {min_value}")  # 10
```

Absolute Value

```
number = -7

absolute = number if number >= 0 else -number

print(absolute)  # 7
```


Real-World Examples

Discount Calculator

```
is_member = True
price = 100
final_price = price * 0.9 if is_member else price
print(f"${final_price:.2f}") # $90.00
```

Button Text

```
is_logged_in = True
button_text = "Logout" if is_logged_in else "Login"
print(button_text) # "Logout"
```

Even or Odd

```
number = 7
result = "even" if number % 2 == 0 else "odd"
print(f"{number} is {result}") # "7 is odd"
```

List Default

```
items = []
first_item = items[0] if items else "Empty"
print(first_item) # "Empty"
```

When NOT to Use Ternary

✗ Too Complex

```
# Don't do this (hard to read)
result = a if condition1 else (b if condition2 else c)

# Do this instead
if condition1:
    result = a
elif condition2:
    result = b
else:
    result = c
```

✗ Multiple Statements

Ternary only works for **single expressions**, not multiple statements:

```
# ✗ Can't do this
x = print("True") if condition else print("False")

# ✓ Do this instead
if condition:
    print("True")
else:
    print("False")
```

Ternary vs if-else

Feature	Ternary	if-else
Length	One line	Multiple lines
Returns value	✓ Yes	No (assigns inside)
Readability (simple)	✓ Clear	✓ Clear
Readability (complex)	✗ Unreadable	✓ Clear
Best for	Simple value selection	Complex logic

Try It Yourself

- Challenge 1:** Use ternary to set `message` to "Adult" if age $\geq$ 18 else "Minor".
- Challenge 2:** Use ternary to find the larger of two numbers.
- Challenge 3:** Use ternary inside a print to say "Even" or "Odd" for a number.
- Challenge 4:** Use ternary to set a price: \$10 if `is_member` else \$15.

Chapter Summary

Part	Syntax	Example
Condition	<code>value_if_true if condition</code>	<code>"Adult" if age >= 18</code>
Else	<code>else value_if_false</code>	<code>else "Minor"</code>
Full	<code>true_value if condition else false_value</code>	<code>"Adult" if age >= 18 else "Minor"</code>

PART 4 SUMMARY

You now know all about control flow:

Chapter	Topic
4.1	<code>if</code> – Basic decision making
4.2	<code>if-else</code> – Two paths
4.3	<code>elif</code> – Multiple conditions
4.4	Nested conditions
4.5	<code>match-case</code> – Pattern matching (Python 3.10+)
4.6	Conditional expressions (ternary operator)

What's Next?

Next up: **Part 5: Loops** – `for` loops, `while` loops, `break`, `continue`, and `pass` !

PART 5: LOOPS

CHAPTER 5.1: for Loops – Iterating Over Sequences

What You'll Learn

- What `for` loops do (repeat code for each item in a sequence)
- Looping over strings, lists, and ranges
- The loop variable
- Nested for loops

What Is a for Loop?

A `for` loop repeats code for each item in a sequence (string, list, range, etc.).

```
# Loop through a list
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(fruit)

# Output:
# apple
# banana
# orange
```

Syntax

```
for variable in sequence:
    # Code to repeat for each item
    print(variable)
```

The loop variable takes each value from the sequence, one at a time.

Looping Over Strings

```
name = "Ryu"

for letter in name:
    print(letter)

# Output:
# R
# y
# u
# u
```

Looping Over Lists

```
scores = [85, 92, 78, 96, 88]

for score in scores:
    if score >= 90:
        print(f"{score} - A")
    elif score >= 80:
        print(f"{score} - B")
    else:
        print(f"{score} - C")
```

Looping Over Ranges with `range()`

We'll cover `range()` in detail in Chapter 5.3, but here's a preview:

```
# Loop 5 times (0 to 4)
for i in range(5):
    print(i)
# Output: 0, 1, 2, 3, 4

# Loop from 1 to 5
for i in range(1, 6):
    print(i)
# Output: 1, 2, 3, 4, 5

# Loop with step
for i in range(0, 10, 2):
    print(i)
# Output: 0, 2, 4, 6, 8
```

Looping Over Dictionaries

```
scores = {"Ryu": 95, "Anna": 87, "Max": 92}

# Loop through keys
for name in scores:
    print(f"{name}: {scores[name]}")

# Loop through key-value pairs
for name, score in scores.items():
    print(f"{name} scored {score}")
```

Using `enumerate()` to Get Index and Value

```
fruits = ["apple", "banana", "orange"]

for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Output:
# 0: apple
# 1: banana
# 2: orange

# Start at 1
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")

# Output:
# 1: apple
# 2: banana
# 3: orange
```

Nested for Loops

Loops inside loops:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"({i},{j})", end=" ")
    print() # New line after each row

# Output:
# (1,1) (1,2) (1,3)
# (2,1) (2,2) (2,3)
# (3,1) (3,2) (3,3)
```

Real-World Examples

Sum of List

```
numbers = [10, 20, 30, 40, 50]

total = 0

for num in numbers:
    total += num

print(f"Sum: {total}") # 150
```

Find Maximum

```
numbers = [45, 23, 89, 12, 67, 34]

max_value = numbers[0]

for num in numbers:
    if num > max_value:
        max_value = num

print(f"Maximum: {max_value}") # 89
```

Multiplication Table

```
number = 7

for i in range(1, 13):
    print(f"{number} x {i} = {number * i}")
```


Triangle Pattern

```
for i in range(1, 6):
    print("*" * i)

# Output:
# *
# **
# ***
# ****
# *****
```

Filtering a List

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = []

for num in numbers:
    if num % 2 == 0:
        evens.append(num)

print(evens)  # [2, 4, 6, 8, 10]
```

Try It Yourself

Challenge 1: Loop through a list of your 5 favorite movies and print each one.

Challenge 2: Use a `for` loop to calculate the sum of numbers 1 to 100.

Challenge 3: Loop through a string and count how many vowels it has.

Challenge 4: Use nested loops to print a 5x5 grid of asterisks.

Chapter Summary

Pattern	Syntax	Use
Loop over list	<code>for item in list:</code>	Process each item
Loop over string	<code>for char in string:</code>	Process each character
Loop with index	<code>for i, item in enumerate(list):</code>	Need index and value
Nested loops	<code>for i in range(): for j in range():</code>	Multi-dimensional

CHAPTER 5.2: while Loops – Condition-Based Looping

What You'll Learn

- What `while` loops do (repeat while condition is true)
- The syntax of `while` loops
- Avoiding infinite loops
- When to use `while` vs `for`

What Is a while Loop?

A `while` loop repeats a block of code **as long as** a condition is true. It checks the condition before each iteration.

```
count = 1

while count <= 5:
    print(f"Count: {count}")
    count += 1

# Output:
# Count: 1
# Count: 2
# Count: 3
# Count: 4
# Count: 5
```

Syntax

```
while condition:
    # Code runs repeatedly while condition is True
    print("Still looping")
    # Make sure condition eventually becomes False!
```

Countdown Example

```
timer = 5

while timer > 0:
    print(f"{timer}...")
    timer -= 1

print("Blast off! 🚀")

# Output:
# 5...
# 4...
# 3...
# 2...
# 1...
# Blast off! 🚀
```

while vs for

for loop	while loop
Best for known number of iterations	Best for unknown number of iterations
Example: <code>for i in range(10):</code>	Example: <code>while not game_over:</code>
Loops through a sequence	Loops based on a condition

Infinite Loops (Be Careful!)

```
# ⚠️ DANGER: This loop never ends!
x = 5
while x > 0:
    print(x)
    # Forgot to change x!
    # This will run forever!

# ✅ Safe: Condition will become False
x = 5
while x > 0:
    print(x)
    x -= 1 # This changes x!
```

Always ensure your loop condition eventually becomes `False`.

Real-World Examples

Game Loop

```
player_health = 100
enemy_health = 50

while player_health > 0 and enemy_health > 0:
    print("Battle continues...")
    player_health -= 10
    enemy_health -= 15
    print(f"Player: {player_health} | Enemy: {enemy_health}")

print("Battle ended!")
```

Password Input (Simulated)

```
correct_password = "swift123"

attempts = 0
max_attempts = 3

while attempts < max_attempts:
    password = input("Enter password: ")
    attempts += 1

    if password == correct_password:
        print("Access granted! ✅")
        break
    else:
        print(f"Wrong password. Attempts left: {max_attempts - attempts}")

else:
    print("Account locked! ❌")
```

Bank Balance

```
balance = 100.0
withdraw_amount = 15.0

while balance >= withdraw_amount:
    print(f"Withdrawing ${withdraw_amount}")
    balance -= withdraw_amount
    print(f"Remaining: ${balance}")

print(f"Insufficient funds. Balance: ${balance}")
```

Number Guessing Game

```
import random

secret = random.randint(1, 100)
guess = 0
attempts = 0

while guess != secret:
    guess = int(input("Guess a number (1-100): "))
    attempts += 1

    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print(f"Correct! You got it in {attempts} attempts!")
```

Validation Loop

```
while True:
    age = input("Enter your age: ")
    if age.isdigit():
        age = int(age)
        break
    print("Please enter a valid number!")

print(f"Your age is {age}")
```

Try It Yourself

Challenge 1: Write a `while` loop that prints numbers from 10 down to 1.

Challenge 2: Keep doubling a number starting at 1 until it exceeds 1000.

Challenge 3: Simulate a countdown timer starting at 60 seconds.

Challenge 4: Write a loop that asks for a number until the user enters 0.

Chapter Summary

Concept	Syntax	Use
while loop	<code>while condition:</code>	Repeat while condition True
Update variable	Must change condition	Prevent infinite loops
Infinite loop	<code>while True:</code>	Run forever (use with <code>break</code>)

CHAPTER 5.3: range() – Generating Number Sequences

What You'll Learn

- What `range()` does (generate sequences of numbers)
- Different ways to use `range()`
- Converting `range` to list
- Using `range()` in loops

What Is range()?

`range()` generates a sequence of numbers. It's most commonly used in `for` loops.

```
# range(stop) – from 0 to stop-1
for i in range(5):
    print(i)
# Output: 0, 1, 2, 3, 4

# range(start, stop) – from start to stop-1
for i in range(2, 6):
    print(i)
# Output: 2, 3, 4, 5

# range(start, stop, step) – with step size
for i in range(0, 10, 2):
    print(i)
# Output: 0, 2, 4, 6, 8
```

range() Syntax

Form	Example	Output
<code>range(stop)</code>	<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(start, stop)</code>	<code>range(2, 5)</code>	2, 3, 4
<code>range(start, stop, step)</code>	<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9

Converting range to List

```
# Convert range to list to see all values
numbers = list(range(5))
print(numbers)  # [0, 1, 2, 3, 4]

evens = list(range(0, 10, 2))
print(evens)   # [0, 2, 4, 6, 8]

odds = list(range(1, 10, 2))
print(odds)    # [1, 3, 5, 7, 9]
```

Counting Down with range()

Use a negative step to count down:

```
# Count down from 5 to 1
for i in range(5, 0, -1):
    print(i)
# Output: 5, 4, 3, 2, 1

# Count down from 10 to 0 by 2s
for i in range(10, -1, -2):
    print(i)
# Output: 10, 8, 6, 4, 2, 0
```


Looping Backwards Through a List

```
fruits = ["apple", "banana", "orange", "mango"]

# Using range with negative step
for i in range(len(fruits) - 1, -1, -1):
    print(fruits[i])

# Output: mango, orange, banana, apple

# Simpler way using reversed()
for fruit in reversed(fruits):
    print(fruit)

# Output: mango, orange, banana, apple
```

Real-World Examples

Times Table

```
number = 7

for i in range(1, 13):
    print(f"{number} x {i} = {number * i}")
```

Sum of Even Numbers

```
total = 0

for i in range(0, 101, 2):
    total += i

print(f"Sum of evens 0-100: {total}") # 2550
```

Factorial

```
n = 5

factorial = 1

for i in range(1, n + 1):
    factorial *= i

print(f"{n}! = {factorial}")  # 5! = 120
```

List Indices

```
names = ["Ryu", "Anna", "Max", "Lily"]

for i in range(len(names)):
    print(f"{i}: {names[i]}")

# Output:
# 0: Ryu
# 1: Anna
# 2: Max
# 3: Lily
```

Star Pattern

```
# Right triangle
for i in range(1, 6):
    print(" " * i)

# Inverted triangle
for i in range(5, 0, -1):
    print(" " * i)
```

range() vs Manual List

Method	Code	Memory
<code>range(1000000)</code>	Efficient	Almost none
<code>list(range(1000000))</code>	Uses memory	8 million+ bytes

`range()` is **lazy** — it generates numbers on demand instead of storing them all.

Try It Yourself

Challenge 1: Use `range()` to print numbers 1 to 20.

Challenge 2: Use `range()` to print even numbers from 2 to 20.

Challenge 3: Use `range()` to count down from 10 to 1.

Challenge 4: Create a list of squares of numbers 1-10 using a loop and `range()`.

Chapter Summary

Function	Output
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(2, 5)</code>	2, 3, 4
<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1

CHAPTER 5.4: break – Exiting Loops Early

What You'll Learn

- What `break` does (immediately exit a loop)
- Using `break` in `for` and `while` loops
- Finding the first match
- Breaking out of nested loops

What Is break?

The `break` statement **immediately exits** the current loop. No more iterations run.

```
for i in range(1, 11):
    if i == 5:
        break # Stop at 5
    print(i)

# Output: 1, 2, 3, 4
```

break in for Loops

```
numbers = [1, 3, 5, 7, 9, 11, 13]

for num in numbers:
    if num == 9:
        print("Found 9! Stopping...")
        break
    print(f"Checking {num}")

# Output:
# Checking 1
# Checking 3
# Checking 5
# Checking 7
# Found 9! Stopping...
```

break in while Loops

```
# Infinite loop with break
secret = 42
guess = 0

while True: # Run forever until break
    guess = int(input("Guess a number: "))
    if guess == secret:
        print("Correct!")
        break
    print("Try again!")
```

Finding First Match

```
names = ["Anna", "Bob", "Charlie", "Diana", "Eve"]
search = "Charlie"

for name in names:
    if name == search:
        print(f"Found {search}!")
        break
    print(f"Checking {name}...")

# Output:
# Checking Anna...
# Checking Bob...
# Found Charlie!
```

break in Nested Loops

`break` only exits the **innermost** loop:

```
for i in range(3):
    for j in range(3):
        if j == 2:
            break # Only breaks inner loop
        print(f"({i},{j})", end=" ")
    print() # New line

# Output:
# (0,0) (0,1)
# (1,0) (1,1)
# (2,0) (2,1)
```

Breaking Out of Nested Loops (Flag Method)

```
found = False
target = 7

for i in range(1, 4):
    for j in range(1, 4):
        product = i * j
        print(f"Checking {i} x {j} = {product}")
        if product == target:
            print(f"Found! {i} x {j} = {target}")
            found = True
            break
    if found:
        break
```

Real-World Examples

Search Until Found

```
import random

secret = random.randint(1, 100)
attempts = 0

while True:
    guess = random.randint(1, 100)
    attempts += 1

    if guess == secret:
        print(f"Found {secret} in {attempts} attempts!")
        break
```

Input Validation with Break

```
while True:
    age = input("Enter your age: ")
    if age.isdigit():
        age = int(age)
        if 0 <= age <= 120:
            break
    print("Invalid input. Please enter a valid age.")

print(f"Age: {age}")
```

Menu System

```
while True:
    print("\n1. Play Game")
    print("2. View Scores")
    print("3. Quit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("Playing game...")
    elif choice == "2":
        print("Showing scores...")
    elif choice == "3":
        print("Goodbye!")
        break
    else:
        print("Invalid choice")
```

Try It Yourself

Challenge 1: Loop through `[4, 7, 2, 9, 1, 5]` and stop when you find a number greater than 8.

Challenge 2: Write a loop that asks for a number until the user enters 0 (use break).

Challenge 3: Find the first prime number greater than 100 (stop once found).

Chapter Summary

Keyword	Effect
<code>break</code>	Exits current loop immediately
In nested loops	Only exits innermost loop

CHAPTER 5.5: continue – Skipping to the Next Iteration

What You'll Learn

- What `continue` does (skip current iteration, go to next)
- Difference between `break` and `continue`
- Filtering items with `continue`
- When to use `continue`

What Is continue?

The `continue` statement **skips the rest of the current iteration** and moves to the next one.

```
for i in range(1, 11):
    if i % 2 == 0:
        continue # Skip even numbers
    print(i)

# Output: 1, 3, 5, 7, 9
```

break vs continue

break	continue
Exits loop completely	Skips only current iteration
Loop stops	Loop continues with next value
No more iterations	Other iterations still run

continue in for Loops

```
for num in range(1, 21):  
    if num % 3 == 0:  
        continue # Skip multiples of 3  
    print(num)  
  
# Output: 1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20
```

continue in while Loops

```
num = 0  
while num < 10:  
    num += 1  
  
    if num % 2 == 0:  
        continue # Skip even numbers  
    print(f"Odd: {num}")  
  
# Output:  
# Odd: 1  
# Odd: 3  
# Odd: 5  
# Odd: 7  
# Odd: 9
```

Real-World Examples

Skip Empty Strings

```
words = ["Hello", "", "World", "", "Python", ""]

for word in words:
    if not word: # if word is empty
        continue
    print(f"Word: {word}")

# Output:
# Word: Hello
# Word: World
# Word: Python
```

Filter Negative Numbers

```
numbers = [5, -2, 8, -1, 0, 3, -4, 7]
total = 0

for num in numbers:
    if num < 0:
        continue # Skip negatives
    total += num

print(f"Sum of non-negative numbers: {total}") # 23 (5+8+0+3+7)
```

Skip Weekends

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

for day in days:
    if day == "Sat" or day == "Sun":
        continue # Skip weekends
    print(f"Work day: {day}")

# Output:
# Work day: Mon
# Work day: Tue
# Work day: Wed
# Work day: Thu
# Work day: Fri
```

Skip Vowels

```
text = "Hello World"
consonants = ""

for char in text:
    if char.lower() in "aeiou":
        continue # Skip vowels
    consonants += char

print(f"Consonants only: {consonants}") # "Hll Wrld"
```

Process Only Valid Data

```
data = ["apple", "", "banana", None, "cherry", "", "date"]

for item in data:
    if not item: # Skip None or empty
        continue
    print(f"Processing: {item}")

# Output:
# Processing: apple
# Processing: banana
# Processing: cherry
# Processing: date
```

continue in Nested Loops

Like `break`, `continue` only affects the **innermost** loop:

```
for i in range(3):
    for j in range(3):
        if j == 1:
            continue # Only skips to next j
        print(f"({i},{j})", end=" ")
    print()

# Output:
# (0,0) (0,2)
# (1,0) (1,2)
# (2,0) (2,2)
```

Try It Yourself

Challenge 1: Print all numbers from 1 to 20 except multiples of 3.

Challenge 2: Given a list with empty strings, print only non-empty strings.

Challenge 3: Sum all positive numbers in a list (skip negatives).

Challenge 4: Print only the uppercase letters from a string.

Chapter Summary

Keyword	Effect	Loop Continues?
<code>break</code>	Exits loop completely	No
<code>continue</code>	Skips to next iteration	Yes

CHAPTER 5.6: pass – Do Nothing Placeholder

What You'll Learn

- What `pass` does (absolutely nothing)
- When to use `pass` as a placeholder
- Why `pass` is useful for incomplete code

What Is pass?

`pass` is a statement that does **nothing**. It's a placeholder for when you need a statement syntactically but don't want any action.

```
if True:
    pass # Does nothing
```

Why Use pass?

Python requires indented code blocks after `if`, `for`, `while`, `def`, `class`, etc. If you haven't written the code yet, `pass` keeps it valid.

```
# Without pass (SyntaxError!)
# if age >= 18:
#     # TODO: Add adult code

# With pass (works)
if age >= 18:
    pass # I'll write this later
```

Common Uses of pass

Placeholder for Future Code

```
def calculate_score():
    pass # Will implement later

class GameCharacter:
    pass # Will add methods later

if condition:
    pass # Handle this case later
```

Empty Loops

```
# Waiting for something (not recommended for production)
while not game_ready():
    pass # Do nothing until ready
```

Empty Exception Handling

```
try:
    risky_operation()
except ValueError:
    pass # Ignore this error for now
```

Creating Minimal Classes/Structs

```
class Person:
    pass

class Student(Person):
    pass

ryuu = Person()
ryuu.name = "Ryuu" # Dynamically add attributes
```

pass in Action

```
def process_user(user):  
    # Work in progress  
    pass  
  
def validate_email(email):  
    # TODO: Add validation  
    pass  
  
def save_to_database(data):  
    # Coming soon  
    pass  
  
# Your code still runs (does nothing)  
process_user("ryuu")  
validate_email("test@test.com")  
save_to_database({"name": "Ryu"})
```

pass vs ... (Ellipsis)

Python also has `...` (Ellipsis) which can be used similarly:

```
def future_function():  
    ...  
  
class FutureClass:  
    ...
```

But `pass` is more common and explicit.

Real-World Examples

Stub Functions

```
def authenticate(username, password):  
    """Will implement authentication later"""  
    pass  
  
def calculate_tax(amount):  
    """Will add tax calculation later"""  
    pass  
  
def send_email(to, subject, body):  
    """Will implement email sending later"""  
    pass
```

Conditional Placeholders

```
age = 11  
  
if age < 13:  
    pass # Child logic (not implemented yet)  
elif age < 20:  
    pass # Teen logic (not implemented yet)  
else:  
    pass # Adult logic (not implemented yet)
```

Loop Placeholders

```
# Will add processing later  
for item in large_dataset:  
    pass # Process each item  
  
# Will add condition later  
for i in range(100):  
    if i % 10 == 0:  
        pass # Handle multiples of 10  
    else:  
        pass # Handle others
```


Try It Yourself

- Challenge 1:** Write a function called `calculate_average` that does nothing yet.
- Challenge 2:** Create a class called `Game` that is empty (use `pass`).
- Challenge 3:** Write an `if` statement with `pass` in the `else` block.
- Challenge 4:** Why would you use `pass` instead of just leaving the block empty?

Chapter Summary

Keyword	Effect	Use
<code>pass</code>	Does nothing	Placeholder for future code

PART 5 SUMMARY

You now know all about loops:

Chapter	Topic
5.1	<code>for</code> loops – Iterating over sequences
5.2	<code>while</code> loops – Condition-based looping
5.3	<code>range()</code> – Generating number sequences
5.4	<code>break</code> – Exiting loops early
5.5	<code>continue</code> – Skipping to next iteration
5.6	<code>pass</code> – Do nothing placeholder

What's Next?

Next up: **Part 6: Functions** – Creating reusable code blocks!

PART 6: FUNCTIONS

CHAPTER 6.1: Creating Functions – Reusable

Code Blocks

What You'll Learn

- What functions are (reusable blocks of code)
- How to define a function with `def`
- How to call a function
- Why functions make your code better

What Is a Function?

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, you write it once inside a function, then **call** the function whenever you need it.

```
# Define the function
def say_hello():
    print("Hello, world!")

# Call the function
say_hello()
say_hello()
say_hello()

# Output:
# Hello, world!
# Hello, world!
# Hello, world!
```

Function Syntax

```
def function_name():
    # Code goes here
    print("This is inside the function")
```

Part	What It Means
<code>def</code>	Keyword that tells Python "this is a function"
<code>function_name</code>	What you name your function (use snake_case)
<code>()</code>	Parentheses for parameters (empty for now)
<code>:</code>	Colon ends the definition line
Indented block	The code that runs when called

Calling a Function

To run a function, write its name followed by parentheses:

```
def show_message():  
    print("Python is awesome!")  
  
show_message() # Calls the function
```

Why Use Functions?

Benefit	Example
Don't repeat yourself	Write once, use many times
Organize code	Group related tasks together
Easier to fix bugs	Fix in one place, not everywhere
Reusable	Use same function in different programs
Testable	Test each function independently

Multiple Statements in a Function

A function can contain as much code as you want:

```
def show_welcome():  
    print("Welcome to my program!")  
    print("Let's learn Python")  
    print("You'll be a coder soon!")  
    print("Keep going! 🤖")
```

```
show_welcome()
```

```
# Output:
```

```
# Welcome to my program!
```

```
# Let's learn Python
```

```
# You'll be a coder soon!
```

```
# Keep going! 🤖
```

Functions Can Call Other Functions

```
def step_one():  
    print("Step 1: Gather ingredients")  
  
def step_two():  
    print("Step 2: Mix ingredients")  
  
def step_three():  
    print("Step 3: Bake for 30 minutes")  
  
def make_cake():  
    step_one()  
    step_two()  
    step_three()  
    print("Cake is ready! 🍰")
```

```
make_cake()
```

```
# Output:
```

```
# Step 1: Gather ingredients
```

```
# Step 2: Mix ingredients
```

```
# Step 3: Bake for 30 minutes
```

```
# Cake is ready! 🍰
```

Naming Functions

Function names should describe what they **do**:

✔ Good Name	✗ Bad Name
<code>show_menu()</code>	<code>do_thing()</code>
<code>calculate_score()</code>	<code>calc()</code>
<code>print_greeting()</code>	<code>func1()</code>
<code>start_game()</code>	<code>go()</code>

Use **snake_case**: all lowercase, words separated by underscores.

```
def display_high_scores(): # ✔ Good
    pass

def displayhighscores(): # ✗ Hard to read
    pass
```

Real-World Examples

Start Game Function

```
def start_game():
    print("Game starting...")
    print("Loading levels...")
    print("Ready? GO! 🎮")

start_game()
```

Show Menu Function

```
def show_main_menu():
    print("=" * 30)
    print("          MAIN MENU")
    print("=" * 30)
    print("1. Start Game")
    print("2. Load Game")
    print("3. Settings")
    print("4. Quit")
    print("=" * 30)

show_main_menu()
```

Print Border Function

```
def print_border():
    print("*" * 40)

print_border()
print("Hello")
print_border()
print("World")
print_border()

# Output:
# *****
# Hello
# *****
# World
# *****
```

Error Message Function

```
def show_error():
    print("❌ ERROR: Something went wrong!")
    print("Please try again.")

def show_success():
    print("✅ Success! Operation completed.")

show_error()
show_success()
```

Try It Yourself

Challenge 1: Create a function called `say_goodbye` that prints "Goodbye! See you later!"

Challenge 2: Create a function called `print_star_line` that prints 10 stars (`*****`).

Challenge 3: Create a function called `show_instructions` that prints 3 instructions for a game.

Challenge 4: Create a function called `celebrate` that prints "🎉 Hooray! 🎉" and call it three times.

Chapter Summary

Concept	Syntax	Example
Define function	<code>def name():</code>	<code>def say_hi():</code>
Call function	<code>name()</code>	<code>say_hi()</code>
Multiple calls	<code>name(); name()</code>	Run the function multiple times

CHAPTER 6.2: Parameters – Passing Data to Functions

What You'll Learn

- What parameters are (inputs to functions)
- How to define parameters
- How to pass arguments when calling
- Using parameters inside functions

What Are Parameters?

Parameters let you pass data into a function. They make functions flexible and reusable.

Without parameters, a function always does the exact same thing. With parameters, it can work with different values.

```
# Without parameter (always says Hello to the same person)
def say_hello():
    print("Hello, World")

# With parameter (can say Hello to anyone!)
def say_hello(name):
    print(f"Hello, {name}")

say_hello("Ryu")    # Hello, Ryu
say_hello("Anna")   # Hello, Anna
say_hello("Max")    # Hello, Max
```

Parameter Syntax

```
def function_name(parameter_name):
    # Use parameter_name inside
    print(parameter_name)
```

Single Parameter

```
def square(number):
    result = number * number
    print(f"{number} squared is {result}")

square(5)    # 5 squared is 25
square(8)    # 8 squared is 64
square(12)   # 12 squared is 144
```

Multiple Parameters

Separate parameters with commas:


```
def greet(first_name, last_name):  
    print(f"Hello, {first_name} {last_name}!")  
  
greet("Ryu", "Swift")  
# Output: "Hello, Ryu Swift!"  
  
def calculate_bmi(weight, height):  
    bmi = weight / (height ** 2)  
    print(f"Your BMI is: {bmi:.1f}")  
  
calculate_bmi(68, 1.75)
```

Using Parameters Inside Functions

Parameters act like variables inside the function:

```
def describe_pet(name, species, age):  
    print(f"{name} is a {species} who is {age} years old")  
  
describe_pet("Fluffy", "cat", 3)  
# Output: "Fluffy is a cat who is 3 years old"
```

Positional Arguments

The order of arguments matters:

```
def introduce(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}")  
  
# Arguments in correct order  
introduce("Ryu", 11, "Tokyo")  
# Output: Ryu is 11 years old and lives in Tokyo  
  
# Wrong order (still runs but makes no sense!)  
introduce("Tokyo", "Ryu", 11)  
# Output: Tokyo is Ryu years old and lives in 11
```

Keyword Arguments

You can specify which parameter gets which value by name:

```
def introduce(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}")  
  
    # Keyword arguments (order doesn't matter)  
    introduce(age=11, name="Ryu", city="Tokyo")  
  
    # Output: Ryu is 11 years old and lives in Tokyo
```

Mixing Positional and Keyword Arguments

Positional arguments must come first:

```
def describe_product(name, price, in_stock=True):  
    print(f"Product: {name}")  
    print(f"Price: ${price}")  
    print(f"In stock: {in_stock}")  
  
    # Positional + keyword  
    describe_product("Book", 19.99, in_stock=False)
```

Real-World Examples

Calculate Rectangle Area

```
def rectangle_area(width, height):  
    area = width * height  
    print(f"Area: {area}")  
  
rectangle_area(5, 3)    # Area: 15  
rectangle_area(7.5, 4) # Area: 30.0
```

Order Food

```
def order_food(item, quantity):  
    print(f"Ordered {quantity} x {item}")  
  
order_food("pizza", 2)    # Ordered 2 x pizza  
order_food("burger", 1)  # Ordered 1 x burger
```

Character Creation

```
def create_character(name, character_class, level):  
    print("Character created!")  
    print(f"Name: {name}")  
    print(f"Class: {character_class}")  
    print(f"Level: {level}")  
  
create_character("Ryu", "Warrior", 1)
```

Temperature Conversion

```
def convert_to_fahrenheit(celsius):  
    fahrenheit = (celsius * 9/5) + 32  
    print(f"{celsius}°C = {fahrenheit}°F")  
  
convert_to_fahrenheit(0)    # 0°C = 32.0°F  
convert_to_fahrenheit(25)  # 25°C = 77.0°F  
convert_to_fahrenheit(100) # 100°C = 212.0°F
```

Greeting with Title

```
def greet(title, name):  
    print(f"Welcome, {title}. {name}!")  
  
greet("Mr", "Ryu")        # Welcome, Mr. Ryu!  
greet("Ms", "Anna")       # Welcome, Ms. Anna!
```

Try It Yourself

Challenge 1: Create a function `multiply(a, b)` that prints the product of a and b.

Challenge 2: Create a function `introduce(name, age, city)` that prints a sentence introducing someone.

Challenge 3: Create a function `calculate_speed(distance, time)` that prints speed = distance / time.

Challenge 4: Create a function `make_pizza(size, toppings)` that prints the pizza order.

Chapter Summary

Concept	Syntax	Example
Single parameter	<code>def name(p):</code>	<code>def square(n):</code>
Multiple parameters	<code>def name(p1, p2):</code>	<code>def add(a, b):</code>
Keyword arguments	<code>name(p1=v1, p2=v2)</code>	<code>greet(name="Ryu")</code>

CHAPTER 6.3: Return Values – Getting Results Back

What You'll Learn

- What return values are (functions that give back results)
- How to use `return` to send a value back
- Storing return values in variables
- Functions that return multiple values

Functions That Return Values

Some functions just do work (like printing). Others **calculate and return** a value you can use.

```
# This function returns nothing (just prints)
def say_hello():
    print("Hello")

# This function returns a value
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # 8
```

Return Syntax

```
def function_name(parameters):
    # Do some work
    return value
```

The `return` keyword sends a value back to the caller.

Basic Return Examples

```
# Returns an integer
def double(number):
    return number * 2

doubled = double(7)
print(doubled)  # 14

# Returns a string
def greet(name):
    return f"Hello, {name}!"

message = greet("Ryu")
print(message)  # "Hello, Ryu!"

# Returns a boolean
def is_even(number):
    return number % 2 == 0

print(is_even(4))  # True
print(is_even(7))  # False
```

Storing Return Values

You can store the return value in a variable:

```
def get_pi():
    return 3.14159

pi = get_pi()
area = pi * 5 * 5
print(area)  # 78.53975
```

Using Return Values Directly

```
def square(n):  
    return n * n  
  
# Use directly in print  
print(square(5)) # 25  
  
# Use in calculations  
result = square(4) + square(3)  
print(result) # 16 + 9 = 25
```

Multiple Return Statements

A function can have multiple `return` statements, but only one will run:

```
def check_age(age):  
    if age >= 18:  
        return "Adult"  
    else:  
        return "Minor"  
  
print(check_age(20)) # "Adult"  
print(check_age(15)) # "Minor"
```

Functions Must Always Return

If a function doesn't have a `return`, it returns `None`:

```
def say_hello(name):  
    print(f"Hello, {name}")  
  
result = say_hello("Ryu")  
print(result) # None
```

Returning Multiple Values

Use tuples to return multiple values:

```
def divide(a, b):  
    quotient = a // b  
    remainder = a % b  
    return quotient, remainder  
  
q, r = divide(10, 3)  
print(f"Quotient: {q}, Remainder: {r}") # Quotient: 3, Remainder: 1
```

Real-World Examples

Calculate Total Price

```
def total_price(price, quantity, tax_rate=0.08):  
    subtotal = price * quantity  
    tax = subtotal * tax_rate  
    return subtotal + tax  
  
total = total_price(19.99, 3)  
print(f"${total:.2f}") # $64.77
```

Get Grade Letter

```
def get_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    elif score >= 60:  
        return "D"  
    else:  
        return "F"  
  
grade = get_grade(85)  
print(f"Your grade: {grade}") # "Your grade: B"
```


Maximum of Two Numbers

```
def max_of_two(a, b):  
    if a > b:  
        return a  
    else:  
        return b  
  
print(max_of_two(10, 20)) # 20  
print(max_of_two(42, 7)) # 42
```

Format Time

```
def format_time(hours, minutes):  
    return f"{hours:02d}:{minutes:02d}"  
  
time = format_time(3, 5)  
print(time) # "03:05"
```

Validate Email (Simple)

```
def is_valid_email(email):  
    return "@" in email and "." in email  
  
print(is_valid_email("test@example.com")) # True  
print(is_valid_email("invalid")) # False
```

Try It Yourself

Challenge 1: Create a function `cube(n)` that returns $n \times n \times n$.

Challenge 2: Create a function `is_adult(age)` that returns True if age 18 or older.

Challenge 3: Create a function `get_full_name(first, last)` that returns the full name with a space.

Challenge 4: Create a function `circle_area(radius)` that returns the area ($\pi \times r^2$).

Chapter Summary

Concept	Syntax	Example
Return type	No type annotation needed	<code>return value</code>
Return value	<code>return value</code>	<code>return a + b</code>
Store result	<code>x = function()</code>	<code>total = add(5, 3)</code>
Multiple returns	<code>return a, b</code>	<code>return quotient, remainder</code>
No return	Implicit <code>None</code>	<code>def f(): pass</code>

CHAPTER 6.4: Scope – Where Variables Live

What You'll Learn

- What scope is (where variables are accessible)
- Local scope vs global scope
- The `global` keyword
- Why scope matters

What Is Scope?

Scope determines where a variable can be accessed. Variables created inside a function are **local** — they only exist inside that function.

```
def my_function():
    x = 10  # Local variable
    print(x)

my_function()
print(x)  # ERROR! NameError: name 'x' is not defined
```

Local Scope

Variables created inside a function are local to that function:

```
def greet():  
    message = "Hello" # Local variable  
    print(message)  
  
greet()  
# print(message) # ERROR! message doesn't exist outside
```

Global Scope

Variables created outside any function are **global** — accessible anywhere:

```
name = "Ryu" # Global variable  
  
def greet():  
    print(f"Hello, {name}") # Can access global  
  
greet() # Hello, Ryu
```

Modifying Global Variables

To modify a global variable inside a function, use the `global` keyword:

```
count = 0 # Global  
  
def increment():  
    global count # Tell Python we're using the global count  
    count += 1  
  
increment()  
increment()  
print(count) # 2
```

Without `global`, Python creates a **new local variable**:

```
count = 0

def increment():
    count = 5 # Creates a NEW local variable
    print(f"Inside: {count}")

increment()    # Inside: 5
print(count)   # 0 (global unchanged)
```

Local Variables Hide Globals

Local variables with the same name as a global will "shadow" (hide) the global:

```
name = "Global Ryuu"

def greet():
    name = "Local Ryuu" # Shadows the global
    print(f"Inside: {name}")

greet()    # Inside: Local Ryuu
print(f"Outside: {name}") # Outside: Global Ryuu
```

Function Parameters Are Local

Function parameters are local to the function:

```
def double(num): # num is local to this function
    result = num * 2 # result is also local
    return result

# print(num) # ERROR! num doesn't exist here
# print(result) # ERROR! result doesn't exist here
```

Accessing Global vs Local

Python looks for variables in this order:

1. **Local** (inside the current function)
2. **Enclosing** (if nested functions)

3. **Global** (top level of the module)

4. **Built-in** (Python's built-in functions)

```
value = 100 # Global

def outer():
    value = 50 # Enclosing

    def inner():
        value = 25 # Local
        print(value) # Finds local (25)

    inner()
    print(value) # Finds enclosing (50)

outer()
print(value) # Finds global (100)
```

Real-World Examples

Score Tracker

```
total_score = 0 # Global

def add_points(points):
    global total_score
    total_score += points
    print(f"Added {points}. Total: {total_score}")

def reset_score():
    global total_score
    total_score = 0
    print("Score reset")

add_points(10) # Added 10. Total: 10
add_points(25) # Added 25. Total: 35
reset_score() # Score reset
print(total_score) # 0
```

Counter Function

```
counter = 0

def increment():
    global counter
    counter += 1
    return counter

def decrement():
    global counter
    counter -= 1
    return counter

print(increment()) # 1
print(increment()) # 2
print(decrement()) # 1
```

Configuration Settings

```
# Global settings
debug_mode = True
theme = "dark"

def log(message):
    if debug_mode:
        print(f"[DEBUG] {message}")

def set_theme(new_theme):
    global theme
    theme = new_theme
    print(f"Theme changed to {theme}")

log("Starting app")          # [DEBUG] Starting app
set_theme("light")           # Theme changed to light
print(theme)                 # light
```

When to Avoid Global Variables

Global variables can make code hard to debug:

```
# ❌ Avoid: Too many globals

x = 1
y = 2
z = 3

def do_something():
    global x, y, z
    x = x + y
    y = y + z
    z = x + y

# ✅ Better: Pass parameters and return values
def do_something(x, y, z):
    x = x + y
    y = y + z
    z = x + y
    return x, y, z
```

Try It Yourself

Challenge 1: Create a global variable and access it inside a function.

Challenge 2: Try to modify a global variable inside a function without `global`. What happens?

Challenge 3: Create a local variable with the same name as a global. Which one gets used inside the function?

Challenge 4: Create a counter that increments using a global variable.

Chapter Summary

Scope	Where	Accessible
Local	Inside a function	Only inside that function
Global	Outside any function	Everywhere
Parameter	Function definition	Inside that function

CHAPTER 6.5: Lambda Functions – Small Anonymous Functions

What You'll Learn

- What lambda functions are (small one-line functions)
- How to write lambda functions
- When to use lambda vs regular functions
- Using lambda with `map`, `filter`, `sorted`

What Is a Lambda Function?

A **lambda function** is a small, anonymous function defined in one line. It's called "lambda" after the Greek letter λ .

```
# Regular function
def square(x):
    return x * x

# Lambda function (same thing)
square = lambda x: x * x

print(square(5)) # 25
```

Lambda Syntax

```
lambda parameters: expression
```

- `lambda` keyword
- Parameters (comma-separated)
- `:` colon
- Expression (single expression, no statements)

Lambda vs Regular Function

Feature	Regular Function	Lambda
Name	Yes	No (anonymous)
Lines	Multiple	One line only
Statements	Any	Single expression only
Return	<code>return</code> keyword	Implicitly returns expression
Use	Complex logic	Simple operations

Basic Lambda Examples

```
# Add two numbers
add = lambda a, b: a + b
print(add(5, 3))  # 8

# Check if even
is_even = lambda n: n % 2 == 0
print(is_even(4))  # True

# Get string length
length = lambda s: len(s)
print(length("Python"))  # 6

# Maximum of two
maximum = lambda a, b: a if a > b else b
print(maximum(10, 20))  # 20
```

Lambda with map()

`map()` applies a function to each item in an iterable:

```
numbers = [1, 2, 3, 4, 5]

# Double each number
doubled = list(map(lambda x: x * 2, numbers))
print(doubled)  # [2, 4, 6, 8, 10]

# Square each number
squares = list(map(lambda x: x ** 2, numbers))
print(squares)  # [1, 4, 9, 16, 25]

# Convert to string
strings = list(map(lambda x: f"Number {x}", numbers))
print(strings)  # ["Number 1", "Number 2", ...]
```

Lambda with filter()

`filter()` keeps items where the function returns True:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Keep even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4, 6, 8, 10]

# Keep numbers greater than 5
greater = list(filter(lambda x: x > 5, numbers))
print(greater)  # [6, 7, 8, 9, 10]

# Keep strings longer than 3
words = ["cat", "elephant", "dog", "bird", "ant"]
long = list(filter(lambda w: len(w) > 3, words))
print(long)  # ["elephant", "bird"]
```

Lambda with sorted()

```
# Sort by length
words = ["apple", "banana", "cherry", "date", "elderberry"]
by_length = sorted(words, key=lambda w: len(w))
print(by_length)  # ["date", "apple", "banana", "cherry", "elderberry"]

# Sort by last character
by_last = sorted(words, key=lambda w: w[-1])
print(by_last)

# Sort list of tuples by second value
pairs = [(1, "one"), (3, "three"), (2, "two")]
by_value = sorted(pairs, key=lambda p: p[1])
print(by_value)  # [(1, 'one'), (3, 'three'), (2, 'two')]
```

Lambda with list comprehensions (alternative)

Often list comprehensions are clearer than `map()` and `filter()`:

```
numbers = [1, 2, 3, 4, 5]

# Using map/lambda
doubled_map = list(map(lambda x: x * 2, numbers))

# Using list comprehension (often preferred)
doubled_comp = [x * 2 for x in numbers]

# Both give the same result: [2, 4, 6, 8, 10]

# Filter with list comprehension
evens_filter = list(filter(lambda x: x % 2 == 0, numbers))
evens_comp = [x for x in numbers if x % 2 == 0]
```

Real-World Examples

Data Transformation

```
prices = [19.99, 4.50, 12.99, 3.25]

# Add tax (10%)
with_tax = list(map(lambda p: p * 1.10, prices))
print(with_tax) # [21.989, 4.95, 14.289, 3.575]

# Format as currency
formatted = list(map(lambda p: f"${p:.2f}", prices))
print(formatted) # ["$19.99", "$4.50", "$12.99", "$3.25"]
```

User Input Processing

```
names = [" ryuu ", " ANNA ", " max "]

# Clean names: strip, lowercase, capitalize
cleaned = list(map(lambda n: n.strip().lower().capitalize(), names))
print(cleaned) # ["Ryu", "Anna", "Max"]
```

Filtering Data

```
scores = [85, 92, 58, 76, 44, 95, 67]

# Passing scores (>= 60)
passing = list(filter(lambda s: s >= 60, scores))
print(passing) # [85, 92, 76, 95, 67]

# Failing scores (< 60)
failing = list(filter(lambda s: s < 60, scores))
print(failing) # [58, 44]
```

Sorting with Custom Key

```
products = [
    {"name": "Book", "price": 19.99},
    {"name": "Pen", "price": 2.50},
    {"name": "Laptop", "price": 999.99},
]

# Sort by price
by_price = sorted(products, key=lambda p: p["price"])
print([p["name"] for p in by_price])  # ["Pen", "Book", "Laptop"]

# Sort by name
by_name = sorted(products, key=lambda p: p["name"])
```

When to Use Lambda

Use Lambda When...	Use Regular Function When...
Simple one-line operation	Complex logic
Used once (for map/filter/sorted)	Used multiple times
No side effects	Has side effects
Expression only	Multiple statements

Try It Yourself

- Challenge 1:** Create a lambda that returns the square of a number.
- Challenge 2:** Use `map()` with a lambda to convert a list of temperatures from Celsius to Fahrenheit.
- Challenge 3:** Use `filter()` with a lambda to keep only words that start with "a".
- Challenge 4:** Use `sorted()` with a lambda to sort a list of tuples by the second element.

Chapter Summary

Concept	Syntax	Example
Lambda	<code>lambda p: expression</code>	<code>lambda x: x * 2</code>
With map	<code>map(lambda x: ..., list)</code>	<code>map(lambda x: x*2, [1,2,3])</code>
With filter	<code>filter(lambda x: ..., list)</code>	<code>filter(lambda x: x>5, [1,2,3])</code>
With sorted	<code>sorted(list, key=lambda x: ...)</code>	<code>sorted(words, key=len)</code>

CHAPTER 6.6: Recursion – Functions Calling Themselves

What You'll Learn

- What recursion is (a function that calls itself)
- Base case vs recursive case
- Common recursion examples
- When to use recursion (and when not to)

What Is Recursion?

Recursion is when a function calls itself. It's a way to solve problems by breaking them down into smaller versions of the same problem.

```
def countdown(n):
    if n <= 0:
        print("Blast off!")
    else:
        print(n)
        countdown(n - 1) # Calls itself

countdown(5)

# Output:
# 5
# 4
# 3
# 2
# 1
# Blast off!
```

Base Case vs Recursive Case

Every recursive function needs two parts:

Part	What It Does
Base case	Stops the recursion (no more calls)
Recursive case	Calls itself with a smaller problem

```
def factorial(n):
    # Base case: stop when n <= 1
    if n <= 1:
        return 1
    # Recursive case: call itself with n-1
    return n * factorial(n - 1)

print(factorial(5)) # 120 (5 × 4 × 3 × 2 × 1)
```

How Recursion Works

```
def factorial(n):  
    print(f"Calling factorial({n})")  
    if n <= 1:  
        print(f"Base case reached: factorial({n}) = 1")  
        return 1  
    result = n * factorial(n - 1)  
    print(f"factorial({n}) = {result}")  
    return result  
  
factorial(4)  
# Output:  
# Calling factorial(4)  
# Calling factorial(3)  
# Calling factorial(2)  
# Calling factorial(1)  
# Base case reached: factorial(1) = 1  
# factorial(2) = 2  
# factorial(3) = 6  
# factorial(4) = 24
```

Common Recursion Examples

Factorial

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5))    # 120  
print(factorial(10))   # 3628800
```


Sum of Numbers 1 to n

```
def sum_to(n):  
    if n <= 0:  
        return 0  
    return n + sum_to(n - 1)  
  
print(sum_to(5))    # 15 (5+4+3+2+1)  
print(sum_to(10))   # 55
```

Fibonacci Sequence

```
def fibonacci(n):  
    if n <= 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
# First 10 Fibonacci numbers  
for i in range(10):  
    print(fibonacci(i), end=" ")  
  
# Output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Power Function

```
def power(base, exp):  
    if exp == 0:  
        return 1  
    return base * power(base, exp - 1)  
  
print(power(2, 10))    # 1024  
print(power(5, 3))     # 125
```

Reverse String

```
def reverse_string(s):  
    if len(s) <= 1:  
        return s  
    return reverse_string(s[1:]) + s[0]  
  
print(reverse_string("hello"))    # "olleh"  
print(reverse_string("Python"))  # "nohtyP"
```

Real-World Examples

Directory Tree Walking (Simplified)

```
def list_files(path, indent=""):  
    # Simulated file system  
    files = ["file1.txt", "file2.txt"]  
    folders = ["subfolder1", "subfolder2"]  
  
    for file in files:  
        print(f"{indent}📄 {file}")  
  
    for folder in folders:  
        print(f"{indent}📁 {folder}/")  
        list_files(f"{path}/{folder}", indent + "  ")  
  
# Would recursively list all files in a directory
```

Check Palindrome

```
def is_palindrome(s):
    # Remove spaces and convert to lowercase
    s = s.replace(" ", "").lower()

    # Base case: 0 or 1 character is a palindrome
    if len(s) <= 1:
        return True

    # Check first and last characters
    if s[0] != s[-1]:
        return False

    # Recursively check the middle
    return is_palindrome(s[1:-1])

print(is_palindrome("racecar"))          # True
print(is_palindrome("hello"))            # False
print(is_palindrome("A man a plan a canal panama")) # True
```

Tower of Hanoi

```
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
    else:
        hanoi(n - 1, source, auxiliary, target)
        print(f"Move disk {n} from {source} to {target}")
        hanoi(n - 1, auxiliary, target, source)

hanoi(3, "A", "C", "B")
# Output shows how to solve Tower of Hanoi with 3 disks
```

Recursion vs Iteration

Recursion	Iteration (Loop)
Elegant for certain problems	Usually faster
Uses more memory (call stack)	Uses less memory
Risk of stack overflow	No stack overflow risk
Natural for tree/graph problems	Natural for simple loops

```
# Iterative factorial (loop)
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Recursive factorial
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)
```

The Danger: Infinite Recursion

Without a proper base case, recursion never ends:

```
# ⚠ DANGER: This will crash with RecursionError
def infinite_recursion():
    return infinite_recursion()

# infinite_recursion() # RecursionError: maximum recursion depth exceeded
```

Python has a recursion limit (usually around 1000). You can check it:

```
import sys
print(sys.getrecursionlimit()) # Usually 1000
```

When to Use Recursion

✔ Good for Recursion	✗ Avoid Recursion
Tree traversal	Simple counting
Divide-and-conquer	Large loops (use iteration)
Mathematical recursion (factorial, fibonacci)	When depth > 1000
Backtracking problems	Performance-critical code

Try It Yourself

- Challenge 1:** Write a recursive function to compute the sum of digits of a number.
- Challenge 2:** Write a recursive function to find the greatest common divisor (GCD) of two numbers.
- Challenge 3:** Write a recursive function to count vowels in a string.
- Challenge 4:** Convert a recursive function to an iterative version (or vice versa).

Chapter Summary

Concept	Syntax	Example
Recursive function	<code>def f(): ... f()</code>	Function calls itself
Base case	<code>if condition: return value</code>	Stops recursion
Recursive case	<code>return ... f(modified)</code>	Calls with smaller problem

PART 6 SUMMARY

You now know all about functions:

Chapter	Topic
6.1	Creating functions (<code>def</code>)
6.2	Parameters and arguments
6.3	Return values
6.4	Scope (local vs global)
6.5	Lambda functions (anonymous)
6.6	Recursion (functions calling themselves)

What's Next?

Next up: **Part 7: Strings** – String indexing, slicing, methods, and formatting!

PART 7: STRINGS

CHAPTER 7.1: String Basics – Working with Text

What You'll Learn

- What strings are (sequences of characters)
- Creating strings with quotes
- Escape characters
- Raw strings
- Multi-line strings

What Is a String?

A **string** is a sequence of characters — text, words, sentences, or even just a single letter.

```
single_letter = "A"  
word = "Hello"  
sentence = "Python is fun!"  
paragraph = "This is a longer piece of text. It can have multiple sentences."
```

Creating Strings

```
# Single quotes
name = 'Ryu'

# Double quotes (most common)
message = "Hello, World!"

# Triple quotes (multi-line)
poem = """
Roses are red,
Violets are blue,
Python is awesome,
And so are you!
"""

# Empty string
empty = ""
```

Quotes Inside Strings

```
# Use double quotes to have single quotes inside
text1 = "It's a beautiful day"

# Use single quotes to have double quotes inside
text2 = 'He said "Hello" to me'

# Or use escape characters
text3 = 'It\'s a beautiful day'
text4 = "He said \"Hello\" to me"

# Triple quotes handle both easily
text5 = """She said "It's great!" """
```

Escape Characters

Escape	Meaning	Example
<code>\n</code>	New line	<code>"Hello\nWorld"</code>
<code>\t</code>	Tab	<code>"Hello\tWorld"</code>
<code>\</code>	Backslash	<code>"C:\Users\Ryuu"</code>
<code>\'</code>	Single quote	<code>'It\'s cool'</code>
<code>\"</code>	Double quote	<code>"He said \"Hi\""</code>

```
# New line
print("Hello\nWorld")

# Output:
# Hello
# World

# Tab
print("Hello\tWorld")
# Output: Hello      World

# Backslash
print("C:\Users\Ryuu\Documents")
# Output: C:\Users\Ryuu\Documents
```

Raw Strings (Ignore Escape Characters)

Use `r` before the quotes to treat backslashes as literal characters:

```
# Regular string (backslashes are escaped)
regular = "C:\Users\Ryuu"

# Raw string (backslashes are literal)
raw = r"C:\Users\Ryuu"

print(regular)  # C:\Users\Ryuu
print(raw)     # C:\Users\Ryuu

# Useful for file paths and regex
file_path = r"C:\Program Files\Python"
regex = r"\d{3}-\d{2}-\d{4}"  # Social Security pattern
```


Multi-Line Strings

Using triple quotes `"""` or `'''`:

```
# Triple double quotes
message = """
This is a multi-line string.
It can span several lines.
All whitespace is preserved.
"""

# Triple single quotes
another = '''
This also works
with single quotes.
'''

print(message)
```

String Length

Use `len()` to get the number of characters:

```
name = "Ryu"
print(len(name))      # 4

empty = ""
print(len(empty))     # 0

sentence = "Python is fun"
print(len(sentence))  # 15 (includes spaces)
```

String Concatenation

Joining strings with `+`:

```
first_name = "Ryu"
last_name = "Swift"
full_name = first_name + " " + last_name
print(full_name) # "Ryu Swift"

# Using +=
greeting = "Hello"
greeting += " "
greeting += "World!"
print(greeting) # "Hello World!"
```

String Repetition

Use `*` to repeat strings:

```
print("-" * 30) # "-----"
print("Ha" * 3) # "HaHaHa"
print("Python! " * 3) # "Python! Python! Python! "
```

Checking String Content

```
text = "Hello123"

# Check if all characters are letters
print(text.isalpha())      # False (has numbers)

# Check if all characters are digits
print(text.isdigit())      # False (has letters)

# Check if all characters are alphanumeric (letters or numbers)
print(text.isalnum())      # True

# Check if all characters are lowercase
print(text.islower())      # False (has uppercase H)

# Check if all characters are uppercase
print(text.isupper())      # False

# Check if string starts with
print(text.startswith("Hell")) # True
print(text.startswith("hell")) # False (case-sensitive)

# Check if string ends with
print(text.endswith("123"))    # True
```

Real-World Examples

User Input Validation

```
username = input("Enter username: ")

if username.isalnum() and len(username) >= 3:
    print(f"Username '{username}' is valid")
else:
    print("Username must be alphanumeric and at least 3 characters")
```

Formatting Output

```
def print_box(message):
    width = len(message) + 4
    print "+" + "-" * width + "+"
    print "| " + message + " |"
    print "+" + "-" * width + "+"

print_box("Welcome")

# Output:
# +-----+
# |  Welcome  |
# +-----+
```

Building a Path

```
import os

folder = "documents"
filename = "report.txt"

# Using concatenation
path = folder + "/" + filename
print(path)  # documents/report.txt

# Using os.path.join (better for cross-platform)
path = os.path.join(folder, filename)
print(path)  # documents/report.txt (Windows uses )
```

Try It Yourself

Challenge 1: Create a string with your full name and print its length.

Challenge 2: Create a multi-line string that introduces yourself (name, age, favorite thing).

Challenge 3: Use `+` to build a sentence from 3 different string variables.

Challenge 4: Use `*` to create a border of 50 asterisks.

Chapter Summary

Operation	Syntax	Example
Create string	<code>"text"</code> or <code>'text'</code>	<code>name = "Ryu"</code>
Multi-line	<code>"""text"""</code>	Triple quotes
Raw string	<code>r"text"</code>	<code>r"C:\Users"</code>
Length	<code>len(s)</code>	<code>len("Hi")</code> → <code>2</code>
Concatenation	<code>s1 + s2</code>	<code>"Hi" + " " + "Ryu"</code>
Repeat	<code>s * n</code>	<code>"Ha" * 3</code> → <code>"HaHaHa"</code>

CHAPTER 7.2: String Indexing – Accessing Characters by Position

What You'll Learn

- What string indexing is (accessing characters by position)
- Positive indexing (starts at 0)
- Negative indexing (starts at -1)
- Accessing individual characters

What Is Indexing?

Every character in a string has a position number called an **index**. You can use that index to access the character.

```
text = "Python"

# Index:  0 1 2 3 4 5
# Char:   P y t h o n

print(text[0]) # P
print(text[1]) # y
print(text[2]) # t
print(text[3]) # h
print(text[4]) # o
print(text[5]) # n
```

Positive Indexing

Starts at **0** (first character) and goes up to **length - 1** (last character):

```
name = "Ryu"

print(name[0]) # R
print(name[1]) # y
print(name[2]) # u
print(name[3]) # u

# print(name[4]) # ERROR! Index out of range
```

Negative Indexing

Starts at **-1** (last character) and goes backward:

```
text = "Python"

# Index:  -6 -5 -4 -3 -2 -1
# Char:   P  y  t  h  o  n

print(text[-1]) # n (last character)
print(text[-2]) # o
print(text[-3]) # h
print(text[-4]) # t
print(text[-5]) # y
print(text[-6]) # P
```

IndexError (Out of Range)

Trying to access an index that doesn't exist causes an error:

```
name = "Ryu"

# These work
print(name[0]) # R
print(name[3]) # u

# These cause IndexError
# print(name[4]) # IndexError: string index out of range
# print(name[-5]) # IndexError: string index out of range
```

Getting the First and Last Characters

```
text = "Python Programming"

# First character (always index 0)
first = text[0]
print(f"First: {first}") # P

# Last character (index -1 or len-1)
last = text[-1]
print(f"Last: {last}") # g

# Safe way (check length first)
if len(text) > 0:
    first = text[0]
    last = text[-1]
```

Accessing Characters in a Loop

```
text = "Python"

# Using positive index
for i in range(len(text)):
    print(f"Index {i}: {text[i]}")

# Using negative index
for i in range(-len(text), 0):
    print(f"Index {i}: {text[i]}")

# Simplest way (no index needed)
for char in text:
    print(char)
```

Real-World Examples

Get Initials

```
def get_initials(name):
    # Get first character of each word
    words = name.split()
    initials = ""
    for word in words:
        initials += word[0].upper()
    return initials

print(get_initials("Ryu Swift"))      # RS
print(get_initials("Anna Marie Smith")) # AMS
```


Validate First Character

```
def is_capitalized(word):
    if len(word) == 0:
        return False
    return word[0].isupper()

print(is_capitalized("Python"))    # True
print(is_capitalized("python"))    # False
```

Mask Credit Card Number

```
def mask_card(card_number):
    # Show only last 4 digits
    if len(card_number) < 4:
        return card_number
    return "*" * (len(card_number) - 4) + card_number[-4:]

print(mask_card("1234567890123456"))    # *****3456
```

Check File Extension

```
def get_extension(filename):
    # Find the last dot and get everything after it
    dot_index = filename.rfind(".")
    if dot_index == -1:
        return ""
    return filename[dot_index + 1:]

print(get_extension("document.pdf"))    # pdf
print(get_extension("image.jpg"))       # jpg
print(get_extension("file"))            # ""
```

Try It Yourself

Challenge 1: Print the first, middle, and last character of a string.

Challenge 2: Check if a string starts and ends with the same character.

Challenge 3: Use negative indexing to print a string backwards (character by character).

Challenge 4: Create a function that returns the first character of a string, or `None` if empty.

Chapter Summary

Index Type	Start	End	Example
Positive	0 (first)	<code>len-1</code> (last)	<code>text[0]</code>
Negative	-1 (last)	<code>-len</code> (first)	<code>text[-1]</code>
Out of range			<code>IndexError</code>

CHAPTER 7.3: String Slicing – Extracting Parts of Strings

What You'll Learn

- What string slicing is (extracting a portion of a string)
- Slice syntax: `[start:end:step]`
- Omitting start and end
- Negative step for reversing

What Is Slicing?

Slicing extracts a portion (substring) from a string.

```
text = "Python Programming"
#      0123456789...

# Extract from index 0 to 5 (not including 6)
substring = text[0:6]
print(substring)  # "Python"
```

Slice Syntax

```
string[start:end:step]
```

Part	Meaning	Default
<code>start</code>	Starting index (included)	0
<code>end</code>	Ending index (excluded)	length
<code>step</code>	Step size (how many to skip)	1

Basic Slicing

```
text = "Hello World"

# From index 0 to 4 (not including 5)
print(text[0:5])    # "Hello"

# From index 6 to end
print(text[6:11])   # "World"

# From index 0 to 5 with step 2
print(text[0:6:2])  # "Hlo"
```

Omitting Start and End

```
text = "Python Programming"

# From start to index 5 (not including 6)
print(text[:6])     # "Python"

# From index 7 to end
print(text[7:])      # "Programming"

# Whole string (copy)
print(text[:])       # "Python Programming"
```

Using Negative Indices in Slices

```
text = "Python Programming"

# Last 3 characters
print(text[-3:])    # "ing"

# All except last 3
print(text[:-3])    # "Python Programm"

# Characters from index 1 to -3
print(text[1:-3])   # "ython Programm"
```

Step Parameter

```
text = "Python Programming"

# Every 2nd character
print(text[::2])    # "Pto rgamn"

# Every 3rd character
print(text[::3])    # "PhPamn"

# Reverse the string (step = -1)
print(text[::-1])   # "gnimmargorP nohtyP"
```

Reversing a String

```
text = "hello"

reversed_text = text[::-1]
print(reversed_text)  # "olleh"

# Using reversed() function (another way)
reversed_text = "".join(reversed(text))
print(reversed_text)  # "olleh"
```

Practical Slicing Examples

Get First N Characters

```
def first_n(text, n):  
    return text[:n]  
  
print(first_n("Hello World", 5))    # "Hello"  
print(first_n("Python", 3))         # "Pyt"
```

Get Last N Characters

```
def last_n(text, n):  
    if n > len(text):  
        return text  
    return text[-n:]  
  
print(last_n("Hello World", 5))     # "World"  
print(last_n("Hello World", 3))     # "rld"
```

Remove First and Last Character

```
def remove_ends(text):  
    if len(text) <= 2:  
        return ""  
    return text[1:-1]  
  
print(remove_ends("Hello"))         # "ell"  
print(remove_ends("Hi"))            # ""
```

Get Middle Characters

```
def get_middle(text):
    length = len(text)
    if length % 2 == 0:
        # Even length: return two middle characters
        mid = length // 2
        return text[mid-1:mid+1]
    else:
        # Odd length: return single middle character
        mid = length // 2
        return text[mid]

print(get_middle("Hello"))      # "l" (odd)
print(get_middle("Python"))    # "th" (even)
```

Check Palindrome (Using Slicing)

```
def is_palindrome(text):
    # Remove spaces and convert to lowercase
    cleaned = text.replace(" ", "").lower()
    return cleaned == cleaned[::-1]

print(is_palindrome("racecar"))      # True
print(is_palindrome("A man a plan a canal panama")) # True
print(is_palindrome("hello"))        # False
```

Extract Domain from Email

```
def get_domain(email):
    # Find the @ symbol
    at_index = email.find("@")
    if at_index == -1:
        return None
    return email[at_index + 1:]

print(get_domain("ryuu@example.com")) # "example.com"
print(get_domain("invalid"))          # None
```

Format Phone Number

```
def format_phone(phone):
    # Remove any non-digits
    digits = "".join(c for c in phone if c.isdigit())

    if len(digits) == 10:
        return f"({digits[0:3]}) {digits[3:6]}-{digits[6:10]}"
    return phone

print(format_phone("1234567890"))          # (123) 456-7890
print(format_phone("(123)456-7890"))       # (123) 456-7890
```

Slicing vs Indexing

Operation	Syntax	Returns
Indexing	<code>text[5]</code>	Single character
Slicing	<code>text[5:10]</code>	Substring (could be 0 or more chars)
Slicing	<code>text[100:200]</code>	Empty string (no error)

```
text = "Hello"

# Indexing out of range → ERROR
# print(text[10])  # IndexError

# Slicing out of range → empty string
print(text[10:20])  # "" (no error)
```

Try It Yourself

- Challenge 1:** Extract the first 3 characters of a string.
- Challenge 2:** Extract the last 3 characters of a string.
- Challenge 3:** Reverse a string using slicing.
- Challenge 4:** Extract every other character from a string.
- Challenge 5:** Check if a string is a palindrome (reads the same forwards and backwards).

Chapter Summary

Slice	Meaning	Example ("Hello")
[:n]	First n characters	[:2] → "He"
[n:]	From n to end	[2:] → "llo"
[m:n]	From m to n-1	[1:4] → "ell"
[::-1]	Reverse	[::-1] → "olleH"
[::2]	Every other	[::2] → "Hlo"

CHAPTER 7.4: String Methods – Built-in String Functions

What You'll Learn

- Common string methods for manipulation
- Case conversion methods
- Search and replace methods
- Split and join methods
- Strip methods for whitespace

Case Conversion Methods

```
text = "  Hello World  "

# Convert to uppercase
print(text.upper())      # "  HELLO WORLD  "

# Convert to lowercase
print(text.lower())      # "  hello world  "

# Capitalize first letter only
print(text.capitalize()) # "  Hello world  "

# Capitalize first letter of each word
print(text.title())      # "  Hello World  "

# Swap case (upper <-> lower)
print(text.swapcase())   # "  hELLO wORLD  "
```

Whitespace Removal Methods

```
messy = "  Hello  World  "

# Remove whitespace from both ends
print(messy.strip())      # "Hello  World"

# Remove whitespace from left only
print(messy.lstrip())     # "Hello  World  "

# Remove whitespace from right only
print(messy.rstrip())     # "  Hello  World"

# Remove specific characters
text = "xxHelloxx"
print(text.strip("x"))    # "Hello"
```

Search Methods

```
text = "The quick brown fox jumps over the lazy dog"

# Find first occurrence (returns index)
print(text.find("fox"))      # 16
print(text.find("cat"))     # -1 (not found)

# Find last occurrence
print(text.rfind("the"))    # 31 (case-sensitive)

# Index (like find but raises error if not found)
print(text.index("fox"))    # 16
# print(text.index("cat"))  # ValueError

# Count occurrences
print(text.count("o"))      # 4
```

Check Methods (Boolean)

```
text = "Hello123"

print(text.isalpha())      # False (has numbers)
print(text.isdigit())      # False (has letters)
print(text.isalnum())      # True (letters and numbers only)
print(text.islower())      # False (has capital H)
print(text.isupper())      # False
print(text.isspace())      # False
print(text.startswith("He")) # True
print(text.endswith("23"))  # True
```

Replace Methods

```
text = "I like cats. Cats are cute."

# Replace all occurrences
new = text.replace("cats", "dogs")
print(new)  # "I like dogs. Cats are cute."

# Replace with limit
new = text.replace("cats", "dogs", 1)
print(new)  # "I like dogs. Cats are cute."

# Remove characters by replacing with empty
no_spaces = text.replace(" ", "")
print(no_spaces)  # "Ilikecats.Catsarecute."
```

Split and Join Methods

```
# split() - string to list
sentence = "apple,banana,orange"
fruits = sentence.split(",")
print(fruits)  # ['apple', 'banana', 'orange']

words = "The quick brown fox".split()
print(words)  # ['The', 'quick', 'brown', 'fox']

# splitlines() - split by newlines
multiline = "Line1\nLine2\nLine3"
lines = multiline.splitlines()
print(lines)  # ['Line1', 'Line2', 'Line3']

# join() - list to string
words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence)  # "Python is awesome"

csv = ",".join(["apple", "banana", "orange"])
print(csv)  # "apple,banana,orange"
```

Other Useful Methods

```
text = "Hello World"

# Center in string of width 20
print(text.center(20, "*")) # "*****Hello World*****"

# Left justify
print(text.ljust(20, "-")) # "Hello World-----"

# Right justify
print(text.rjust(20, "-")) # "-----Hello World"

# Fill with zeros (for numbers as strings)
print("42".zfill(5)) # "00042"

# Expand tabs
tabbed = "Hello\tWorld"
print(tabbed.expandtabs(10)) # "Hello      World"
```

Real-World Examples

Clean User Input

```
def clean_username(username):
    # Remove whitespace, convert to lowercase
    return username.strip().lower()

username = input("Enter username: ")
clean = clean_username(username)
print(f"Cleaned: '{clean}'")
```

Extract Hashtags

```
def extract_hashtags(text):
    words = text.split()
    hashtags = []
    for word in words:
        if word.startswith("#"):
            hashtags.append(word)
    return hashtags

post = "Loving #Python and #coding! #100DaysOfCode"
print(extract_hashtags(post))  # ['#Python', '#coding', '#100DaysOfCode']
```

Slug for URLs

```
def create_slug(title):
    # Convert to lowercase, replace spaces with hyphens, remove punctuation
    slug = title.lower()
    slug = slug.replace(" ", "-")
    slug = "".join(c for c in slug if c.isalnum() or c == "-")
    return slug

title = "My Awesome Blog Post!"
slug = create_slug(title)
print(slug)  # "my-awesome-blog-post"
```

Truncate Long Text

```
def truncate(text, max_length, suffix="..."):
    if len(text) <= max_length:
        return text
    return text[:max_length].rstrip() + suffix

long_text = "This is a very long sentence that needs to be truncated"
print(truncate(long_text, 20))  # "This is a very long..."
print(truncate(long_text, 30))  # "This is a very long sentence..."
```

Try It Yourself

Challenge 1: Take `" swift programming "`, trim whitespace, then capitalize it.

Challenge 2: Check if a string starts with `"http://"` or `"https://"`.

Challenge 3: Split `"one|two|three"` using `"|"` as the separator.

Challenge 4: Replace all spaces with underscores in a string.

Challenge 5: Count how many vowels are in a string.

Chapter Summary

Category	Methods
Case	<code>upper()</code> , <code>lower()</code> , <code>capitalize()</code> , <code>title()</code>
Strip	<code>strip()</code> , <code>lstrip()</code> , <code>rstrip()</code>
Search	<code>find()</code> , <code>rfind()</code> , <code>count()</code> , <code>index()</code>
Check	<code>isalpha()</code> , <code>isdigit()</code> , <code>isalnum()</code> , <code>startswith()</code> , <code>endswith()</code>
Replace	<code>replace()</code>
Split/Join	<code>split()</code> , <code>splitlines()</code> , <code>join()</code>

CHAPTER 7.5: Formatting Strings – Controlling Output Appearance

What You'll Learn

- f-strings (Python 3.6+)
- The `.format()` method
- Old `%` formatting
- Format specifiers for numbers, alignment, and padding

f-Strings (Recommended)

f-strings are the most modern and readable way to format strings in Python.

```
name = "Ryu"
age = 11
score = 95.5

# Basic f-string
print(f"Name: {name}, Age: {age}") # Name: Ryu, Age: 11

# With expressions
print(f"Next year: {age + 1}")      # Next year: 12
print(f"Sum: {5 + 3}")              # Sum: 8

# With method calls
print(f"Uppercase: {name.upper()}") # Uppercase: RYU
```

Number Formatting in f-strings

```
pi = 3.14159265359

# Decimal places
print(f"Pi: {pi:.2f}") # Pi: 3.14
print(f"Pi: {pi:.4f}") # Pi: 3.1416

# Percentage
rate = 0.25
print(f"Rate: {rate:.1%}") # Rate: 25.0%

# Thousands separator
big = 1234567
print(f"Big: {big:,}") # Big: 1,234,567

# Width and alignment
name = "Ryu"
print(f"{name:10}") # "Ryu      " (left align, width 10)
print(f"{name:>10}") # "          Ryu" (right align)
print(f"{name:^10}") # "    Ryu    " (center)

# Zero padding
number = 42
print(f"{number:05d}") # "00042"
```

Alignment Specifiers

```
# Left align (<)
print(f"{'Left':<10}")    # "Left      "

# Right align (>)
print(f"{'Right':>10}")   # "          Right"

# Center (^)
print(f"{'Center':^10}")  # "   Center   "

# With fill character
print(f"{'Left':*<10}")    # "Left*****"
print(f"{'Right':*>10}")   # "*****Right"
```

Creating Tables with f-strings

```
print(f"{'Name':<10} {'Score':>5} {'Grade':>5}")
print("-" * 25)
print(f"{'Ryu':<10} {95:>5} {'A':>5}")
print(f"{'Anna':<10} {87:>5} {'B':>5}")
print(f"{'Max':<10} {92:>5} {'A':>5}")

# Output:
# Name           Score Grade
# -----
# Ryu            95      A
# Anna           87      B
# Max            92      A
```

The `.format()` Method

Older but still widely used:


```
name = "Ryu"
age = 11

# Positional arguments
print("Name: {}, Age: {}".format(name, age))

# Indexed arguments
print("Name: {0}, Age: {1}".format(name, age))

# Named arguments
print("Name: {n}, Age: {a}".format(n=name, a=age))

# With formatting
pi = 3.14159
print("Pi: {:.2f}".format(pi))      # Pi: 3.14
print("{:>10}".format("right"))    # "      right"
```

Old Formatting (Legacy)

You might see this in older code:

```
name = "Ryu"
age = 11
pi = 3.14159

# %s for string, %d for integer, %f for float
print("Name: %s, Age: %d" % (name, age))
print("Pi: %.2f" % pi)    # Pi: 3.14

# Multiple
print("%s is %d years old" % (name, age))
```

Format Specifiers Reference

Specifier	Meaning	Example
<code>{:.2f}</code>	2 decimal places	<code>3.14</code>
<code>{:.0f}</code>	0 decimal places	<code>3</code>
<code>{:,}</code>	Thousands separator	<code>1,234,567</code>
<code>{:.1%}</code>	Percentage (1 decimal)	<code>25.0%</code>
<code>{:>10}</code>	Right align width 10	<code>" text"</code>
<code>{:<10}</code>	Left align width 10	<code>"text "</code>
<code>{:^10}</code>	Center width 10	<code>" text "</code>
<code>{:05d}</code>	Zero pad to width 5	<code>"00042"</code>

Real-World Examples

Currency Formatter

```
def format_money(amount):
    return f"${amount:,.2f}"

print(format_money(1234.56))    # $1,234.56
print(format_money(99.9))      # $99.90
```

Progress Percentage

```
def show_progress(current, total):
    percent = current / total * 100
    bar_length = 20
    filled = int(bar_length * current / total)
    bar = "█" * filled + "░" * (bar_length - filled)
    print(f"\r[{bar}] {current}/{total} ({percent:.1f}%)", end="")

# Simulate progress
for i in range(1, 101):
    show_progress(i, 100)
    import time
    time.sleep(0.02)
```

Aligned Columns

```
def print_inventory(items):
    print(f"{'Item':<15} {'Quantity':>8} {'Price':>10}")
    print("-" * 35)
    for item, qty, price in items:
        print(f"{'item':<15} {'qty':>8} ${price:>9.2f}")

inventory = [
    ("Book", 10, 19.99),
    ("Pen", 50, 2.50),
    ("Laptop", 3, 999.99),
]

print_inventory(inventory)
```

f-strings vs .format() vs %

Method	Syntax	Best For
f-strings	<code>f"{var}"</code>	Python 3.6+ (recommended)
<code>.format()</code>	<code>"{}".format(var)</code>	Compatibility with older Python
<code>%</code>	<code>"%s" % var</code>	Legacy code (avoid for new code)

Try It Yourself

- Challenge 1:** Format the number 42.5678 to show only 2 decimal places.
- Challenge 2:** Format the number 1234567 with thousands separators.
- Challenge 3:** Create a table with 3 columns (Name, Age, City) with proper alignment.
- Challenge 4:** Format 0.75 as a percentage.
- Challenge 5:** Pad the number 7 to display as `"007"` (3 digits with leading zeros).

Chapter Summary

Format	Syntax	Example
f-string	<code>f"{var}"</code>	<code>f"Hello {name}"</code>
2 decimals	<code>{:.2f}</code>	<code>f"{pi:.2f}"</code>
Thousands	<code>{:,}</code>	<code>f"{num:,}"</code>
Percentage	<code>{:.1%}</code>	<code>f"{rate:.1%}"</code>
Right align	<code>{>10}</code>	<code>f"{text:>10}"</code>
Zero pad	<code>{:05d}</code>	<code>f"{num:05d}"</code>

CHAPTER 7.6: Regular Expressions – Pattern Matching

What You'll Learn

- What regular expressions (regex) are
- Basic regex patterns
- Using the `re` module
- Common regex patterns for validation

What Are Regular Expressions?

Regular expressions (regex) are patterns used to match, search, and manipulate text. They are powerful but can look complex.

```
import re

# Check if a string is a valid email
email = "user@example.com"
pattern = r"^[\\w.-]+@[\\w.-]+\\.\\w+$"

if re.match(pattern, email):
    print("Valid email!")
```

The re Module

```
import re

text = "The quick brown fox jumps over the lazy dog"

# Search for a pattern
match = re.search(r"fox", text)

if match:
    print(f"Found at index {match.start()}") # Found at index 16

# Find all matches
words = re.findall(r"\b\w{3}\b", text)
print(words) # ['The', 'fox', 'the', 'dog']

# Replace patterns
new_text = re.sub(r"fox", "cat", text)
print(new_text) # "The quick brown cat jumps over the lazy dog"
```

Basic Regex Patterns

Pattern	Meaning	Example Match
.	Any character (except newline)	"a" , "b" , "1"
\d	Any digit (0-9)	"5"
\D	Any non-digit	"a"
\w	Any letter, digit, or underscore	"a" , "1" , "_"
\W	Any non-word character	"@" , "#"
\s	Any whitespace (space, tab, newline)	" "
\S	Any non-whitespace	"a"
^	Start of string	
\$	End of string	
[abc]	Any of a, b, or c	"a" , "b" , "c"
[a-z]	Any lowercase letter	"a" to "z"
[0-9]	Any digit	"0" to "9"

Quantifiers

Quantifier	Meaning	Example
<code>*</code>	0 or more times	<code>\d*</code> matches <code>"123"</code> , <code>""</code>
<code>+</code>	1 or more times	<code>\d+</code> matches <code>"123"</code> , not <code>""</code>
<code>?</code>	0 or 1 time	<code>colou?r</code> matches <code>"color"</code> , <code>"colour"</code>
<code>{n}</code>	Exactly n times	<code>\d{3}</code> matches <code>"123"</code>
<code>{n,}</code>	n or more times	<code>\d{3,}</code> matches <code>"123"</code> , <code>"12345"</code>
<code>{n,m}</code>	Between n and m times	<code>\d{2,4}</code> matches <code>"12"</code> , <code>"123"</code> , <code>"1234"</code>

Common Regex Examples

Email Validation

```
import re

def is_valid_email(email):
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    return bool(re.match(pattern, email))

print(is_valid_email("user@example.com"))      # True
print(is_valid_email("user.name@domain.co"))   # True
print(is_valid_email("invalid-email"))         # False
```

Phone Number Validation

```
def is_valid_phone(phone):
    # Matches: 123-456-7890, (123)456-7890, 1234567890
    pattern = r"^(? \d{3})?[-.]? \d{3}[-.]? \d{4}$"
    return bool(re.match(pattern, phone))

print(is_valid_phone("123-456-7890"))          # True
print(is_valid_phone("1234567890"))           # True
print(is_valid_phone("(123) 456-7890"))        # True
print(is_valid_phone("123-45-6789"))          # False
```

URL Validation

```
def is_valid_url(url):  
    pattern = r"^https?://(www.)?[a-zA-Z0-9-]+\.[a-zA-Z]{2,}(/\S*)?$"  
    return bool(re.match(pattern, url))  
  
print(is_valid_url("https://google.com"))          # True  
print(is_valid_url("http://www.example.com/path")) # True  
print(is_valid_url("invalid-url"))                 # False
```

Extract Numbers from Text

```
def extract_numbers(text):  
    # Find all integers and decimals  
    pattern = r"\d+(?:\.\d+)?"  
    return re.findall(pattern, text)  
  
text = "The price is $19.99 for 3 items and $5.50 for 2 more"  
print(extract_numbers(text)) # ['19.99', '3', '5.50', '2']
```

Extract Dates

```
def extract_dates(text):  
    # Matches YYYY-MM-DD or MM/DD/YYYY  
    pattern = r"\d{4}-\d{2}-\d{2}|\d{2}/\d{2}/\d{4}"  
    return re.findall(pattern, text)  
  
text = "Today is 2024-01-15 and tomorrow is 01/16/2024"  
print(extract_dates(text)) # ['2024-01-15', '01/16/2024']
```

Extract Hashtags

```
def extract_hashtags(text):  
    pattern = r"#\w+"  
    return re.findall(pattern, text)  
  
text = "I love #Python and #coding! #100DaysOfCode"  
print(extract_hashtags(text)) # ['#Python', '#coding', '#100DaysOfCode']
```

Regex with Groups

```
# Extracting parts of a match using parentheses
def parse_name(full_name):
    pattern = r"(\w+), (\w+)" # Last, First
    match = re.match(pattern, full_name)
    if match:
        return match.group(2), match.group(1) # First, Last
    return None, None

first, last = parse_name("Swift, Ryu")
print(f"{first} {last}") # Ryu Swift
```

Compiling Regex for Performance

```
# Compile once, use many times
email_pattern = re.compile(r"^[\\w.-]+@[\\w.-]+\\.\\w+$")

def is_valid_email(email):
    return bool(email_pattern.match(email))

# Much faster if called many times
for email in emails:
    is_valid_email(email)
```

Raw Strings for Regex

Always use **raw strings** (`r"..."`) for regex patterns to avoid escaping issues:

```
# Without raw string (need double backslashes)
pattern1 = "\\d{3}-\\d{2}-\\d{4}"

# With raw string (cleaner)
pattern2 = r"\d{3}-\d{2}-\d{4}"

# Both work, but raw is clearer
```


Try It Yourself

- Challenge 1: Write a regex that matches a 5-digit ZIP code.
- Challenge 2: Write a regex that matches a username (letters, numbers, underscore, 3-16 chars).
- Challenge 3: Extract all email addresses from a block of text.
- Challenge 4: Validate a password (at least 8 chars, 1 uppercase, 1 number).

Chapter Summary

Pattern	Matches
<code>\d</code>	Digit (0-9)
<code>\w</code>	Word character (a-z, A-Z, 0-9, _)
<code>\s</code>	Whitespace
<code>+</code>	1 or more
<code>*</code>	0 or more
<code>{n,m}</code>	n to m times
<code>^</code>	Start of string
<code>\$</code>	End of string

PART 7 SUMMARY

You now know all about strings:

Chapter	Topic
7.1	String basics (creation, quotes, escape)
7.2	String indexing (positive and negative)
7.3	String slicing (<code>[start:end:step]</code>)
7.4	String methods (<code>upper()</code> , <code>split()</code> , etc.)
7.5	String formatting (f-strings, <code>.format()</code>)
7.6	Regular expressions (pattern matching)

What's Next?

Next up: **Part 8: Lists** – Working with ordered collections!

PART 8: LISTS

CHAPTER 8.1: Creating Lists – Ordered Collections

What You'll Learn

- What lists are (ordered, mutable collections)
- Creating lists with square brackets
- Lists can hold different types
- Empty lists

What Is a List?

A **list** is an ordered collection of items. Lists are **mutable** (you can change them after creation).

```
# List of numbers
numbers = [1, 2, 3, 4, 5]

# List of strings
fruits = ["apple", "banana", "orange"]

# Mixed types (allowed but not common)
mixed = [1, "hello", 3.14, True]

print(numbers)  # [1, 2, 3, 4, 5]
print(fruits)   # ["apple", "banana", "orange"]
```

Creating Lists

```
# Empty list (two ways)
empty1 = []
empty2 = list()

# List with values
colors = ["red", "green", "blue"]

# Using list() constructor
numbers = list(range(5)) # [0, 1, 2, 3, 4]
```

Lists Can Hold Any Type

```
# Integers
ages = [11, 12, 10, 13]

# Strings
names = ["Ryu", "Anna", "Max"]

# Floats
prices = [19.99, 4.50, 12.99]

# Booleans
flags = [True, False, True]

# Mixed (generally avoid)
mixed = [1, "hello", 3.14, True, None]
```

Lists of Lists (Nested Lists)

```
# 2D list (matrix)
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[0])      # [1, 2, 3]
print(matrix[1][2])   # 6

# List of different length lists
jagged = [
    [1, 2],
    [3, 4, 5],
    [6]
]
```

List Length

```
fruits = ["apple", "banana", "orange", "mango"]
print(len(fruits))   # 4

empty = []
print(len(empty))    # 0
```

Checking if an Item Exists

```
fruits = ["apple", "banana", "orange"]

print("apple" in fruits)    # True
print("grape" in fruits)    # False
print("apple" not in fruits) # False
```

Real-World Examples

Shopping List

```
shopping_list = ["milk", "eggs", "bread", "butter"]  
print(f"Items to buy: {len(shopping_list)}")  
print(shopping_list)
```

High Scores

```
high_scores = [950, 800, 750, 600, 500]  
print(f"Top score: {high_scores[0]}")
```

Student Roster

```
students = ["Ryuu", "Anna", "Max", "Lily", "John"]  
print(f"Class has {len(students)} students")
```

Coordinates

```
point = [10, 20]  
print(f"X: {point[0]}, Y: {point[1]}")
```

Try It Yourself

Challenge 1: Create a list of your 5 favorite movies.

Challenge 2: Create a list with mixed types (int, string, float, bool).

Challenge 3: Create a 2D list representing a 3x3 tic-tac-toe board.

Challenge 4: Check if "Python" is in a list of programming languages.

Chapter Summary

Operation	Syntax	Example
Create list	<code>[item1, item2]</code>	<code>fruits = ["apple", "banana"]</code>
Empty list	<code>[]</code> or <code>list()</code>	<code>empty = []</code>
List length	<code>len(list)</code>	<code>len(fruits)</code>
Check membership	<code>item in list</code>	<code>"apple" in fruits</code>

CHAPTER 8.2: Accessing Elements – Reading from Lists

What You'll Learn

- Accessing elements by index (positive and negative)
- Getting first and last elements
- Slicing lists
- Copying lists

Accessing by Index

List indices start at **0**:

```
fruits = ["apple", "banana", "orange", "mango", "grape"]
# Index:    0         1         2         3         4

print(fruits[0]) # apple
print(fruits[1]) # banana
print(fruits[2]) # orange
print(fruits[3]) # mango
print(fruits[4]) # grape

# print(fruits[5]) # IndexError!
```

Negative Indexing

Negative indices count from the end:

```
fruits = ["apple", "banana", "orange", "mango", "grape"]

# Index:    -5      -4      -3      -2      -1

print(fruits[-1]) # grape (last)
print(fruits[-2]) # mango
print(fruits[-3]) # orange
print(fruits[-4]) # banana
print(fruits[-5]) # apple
```

Getting First and Last Elements

```
fruits = ["apple", "banana", "orange"]

# Using index
first = fruits[0]
last = fruits[-1]

# Using built-in functions (returns None if empty)
first = fruits[0] if fruits else None
last = fruits[-1] if fruits else None

# Using list methods (returns None if empty)
first = fruits and fruits[0]
last = fruits and fruits[-1]
```

Slicing Lists

Slicing extracts a portion of a list:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# From index 2 to 5 (not including 5)
print(numbers[2:5])    # [2, 3, 4]

# From start to index 5
print(numbers[:5])     # [0, 1, 2, 3, 4]

# From index 5 to end
print(numbers[5:])     # [5, 6, 7, 8, 9]

# Whole list (copy)
print(numbers[:])      # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Slicing with Step

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Every other element
print(numbers[::2])    # [0, 2, 4, 6, 8]

# Every third element
print(numbers[::3])    # [0, 3, 6, 9]

# Reverse list
print(numbers[::-1])   # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```


Slicing with Negative Indices

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Last 3 elements
print(numbers[-3:])    # [7, 8, 9]

# All except last 3
print(numbers[:-3])    # [0, 1, 2, 3, 4, 5, 6]

# From index 2 to -2
print(numbers[2:-2])    # [2, 3, 4, 5, 6, 7]
```

Copying Lists

```
original = [1, 2, 3]

# Method 1: slicing
copy1 = original[:]

# Method 2: list() constructor
copy2 = list(original)

# Method 3: copy() method
copy3 = original.copy()

# Method 4: list comprehension
copy4 = [x for x in original]

# All create independent copies
copy1[0] = 99
print(original)    # [1, 2, 3] (unchanged)
```

Nested List Access

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
# Access element at row 1, column 2  
print(matrix[1][2]) # 6  
  
# Access entire row  
print(matrix[0])    # [1, 2, 3]  
  
# Access column (list comprehension)  
column = [row[1] for row in matrix]  
print(column)       # [2, 5, 8]
```

Real-World Examples

First and Last Elements

```
def first_last(lst):  
    if not lst:  
        return None, None  
    return lst[0], lst[-1]  
  
scores = [85, 92, 78, 96, 88]  
first, last = first_last(scores)  
print(f"First: {first}, Last: {last}") # First: 85, Last: 88
```

Middle Element

```
def middle(lst):
    n = len(lst)
    if n == 0:
        return None
    return lst[n // 2]

print(middle([1, 2, 3, 4, 5]))    # 3
print(middle([1, 2, 3, 4]))       # 3 (left-middle)
```

Every Other Element

```
def every_other(lst):
    return lst[::2]

print(every_other([1, 2, 3, 4, 5, 6])) # [1, 3, 5]
```

Safe Access (with default)

```
def safe_get(lst, index, default=None):
    if 0 <= index < len(lst):
        return lst[index]
    return default

colors = ["red", "green", "blue"]
print(safe_get(colors, 1))    # green
print(safe_get(colors, 10))   # None
print(safe_get(colors, 10, "unknown")) # unknown
```

Try It Yourself

Challenge 1: Print the first, middle, and last element of a list.

Challenge 2: Get the last 3 elements of a list using negative slicing.

Challenge 3: Reverse a list using slicing (not the `reverse()` method).

Challenge 4: Create a copy of a list and verify that modifying the copy doesn't affect the original.

Chapter Summary

Operation	Syntax	Example
Access by index	<code>lst[i]</code>	<code>fruits[0]</code>
Negative index	<code>lst[-i]</code>	<code>fruits[-1]</code>
Slice	<code>lst[start:end]</code>	<code>numbers[2:5]</code>
Slice with step	<code>lst[start:end:step]</code>	<code>numbers[:2]</code>
Copy list	<code>lst[:]</code> or <code>lst.copy()</code>	<code>copy = original[:]</code>

CHAPTER 8.3: Modifying Lists – Adding, Changing, Removing

What You'll Learn

- Changing elements by index
- Adding elements (`append`, `insert`, `extend`)
- Removing elements (`remove`, `pop`, `clear`)
- Other modification methods

Changing Elements

Lists are mutable — you can change individual elements:

```
fruits = ["apple", "banana", "orange"]
print(fruits) # ['apple', 'banana', 'orange']

# Change an element
fruits[1] = "blueberry"
print(fruits) # ['apple', 'blueberry', 'orange']

# Change multiple elements with slice
fruits[0:2] = ["grape", "kiwi"]
print(fruits) # ['grape', 'kiwi', 'orange']
```

Adding Elements

`append()` – Add to End

```
fruits = ["apple", "banana"]
fruits.append("orange")
print(fruits)  # ['apple', 'banana', 'orange']
```

`insert()` – Add at Position

```
fruits = ["apple", "banana", "orange"]
fruits.insert(1, "grape")  # Insert at index 1
print(fruits)  # ['apple', 'grape', 'banana', 'orange']

fruits.insert(0, "kiwi")  # Insert at beginning
print(fruits)  # ['kiwi', 'apple', 'grape', 'banana', 'orange']
```

`extend()` – Add Another List

```
fruits = ["apple", "banana"]
more_fruits = ["orange", "mango"]

fruits.extend(more_fruits)
print(fruits)  # ['apple', 'banana', 'orange', 'mango']

# Using += (same as extend)
fruits += ["grape", "kiwi"]
print(fruits)  # ['apple', 'banana', 'orange', 'mango', 'grape', 'kiwi']
```

Removing Elements

`remove()` – Remove by Value

```
fruits = ["apple", "banana", "orange", "banana"]
fruits.remove("banana")  # Removes first occurrence only
print(fruits)  # ['apple', 'orange', 'banana']
```

pop() - Remove by Index (returns value)

```
fruits = ["apple", "banana", "orange"]

# Remove last element
last = fruits.pop()
print(last)      # orange
print(fruits)    # ['apple', 'banana']

# Remove at specific index
second = fruits.pop(0)
print(second)    # apple
print(fruits)    # ['banana']
```

del - Delete by Index or Slice

```
fruits = ["apple", "banana", "orange", "mango", "grape"]

# Delete single element
del fruits[1]
print(fruits)    # ['apple', 'orange', 'mango', 'grape']

# Delete slice
del fruits[1:3]
print(fruits)    # ['apple', 'grape']

# Delete entire list
del fruits
# print(fruits) # NameError: name 'fruits' is not defined
```

clear() - Remove All Elements

```
fruits = ["apple", "banana", "orange"]

fruits.clear()
print(fruits)    # []
```

Other Modification Methods

`reverse()` – Reverse in Place

```
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(numbers)  # [5, 4, 3, 2, 1]
```

`sort()` – Sort in Place

```
numbers = [5, 2, 8, 1, 9]
numbers.sort()
print(numbers)  # [1, 2, 5, 8, 9]

numbers.sort(reverse=True)
print(numbers)  # [9, 8, 5, 2, 1]

# Sort strings alphabetically
words = ["banana", "apple", "cherry"]
words.sort()
print(words)  # ['apple', 'banana', 'cherry']
```

Real-World Examples

Todo List

```
tasks = []

# Add tasks
tasks.append("Buy groceries")
tasks.append("Write code")
tasks.append("Exercise")

print(f"Tasks: {tasks}")

# Complete a task (remove)
completed = tasks.pop(1)
print(f"Completed: {completed}")
print(f"Remaining: {tasks}")

# Add at specific position
tasks.insert(0, "Call mom")
print(f"Updated: {tasks}")
```

Score Tracker

```
scores = [85, 92, 78, 96, 88]

# Add new score
new_score = 95
scores.append(new_score)
print(f"Scores: {scores}")

# Sort descending
scores.sort(reverse=True)
print(f"Sorted: {scores}")

# Remove lowest score
lowest = scores.pop()
print(f"Removed lowest: {lowest}")
print(f"Top scores: {scores}")
```


Shopping Cart

```
cart = []

def add_item(item):
    cart.append(item)
    print(f"Added: {item}")

def remove_item(item):
    if item in cart:
        cart.remove(item)
        print(f"Removed: {item}")
    else:
        print(f"{item} not in cart")

def show_cart():
    print(f"Cart ({len(cart)} items): {cart}")

add_item("Book")
add_item("Pen")
add_item("Notebook")
show_cart()

remove_item("Pen")
show_cart()
```

History Tracker (Undo/Redo)

```
history = []
current_state = []

def action(operation):
    # Save current state before change
    history.append(current_state.copy())
    # Perform operation
    if operation == "add":
        current_state.append("new item")
    elif operation == "remove" and current_state:
        current_state.pop()
    print(f"After {operation}: {current_state}")

def undo():
    if history:
        previous = history.pop()
        current_state.clear()
        current_state.extend(previous)
        print(f"Undo: {current_state}")
    else:
        print("Nothing to undo")

action("add")      # After add: ['new item']
action("add")      # After add: ['new item', 'new item']
undo()             # Undo: ['new item']
```

Try It Yourself

Challenge 1: Start with `[1, 2, 3, 4, 5]`. Replace the middle element (2) with 99.

Challenge 2: Create a shopping list: add 3 items, remove one, insert one at the beginning.

Challenge 3: Sort a list of numbers in descending order.

Challenge 4: Remove all occurrences of 5 from `[5, 1, 5, 2, 5, 3, 5, 4]`.

Chapter Summary

Operation	Method	Example
Add to end	<code>append(item)</code>	<code>lst.append(5)</code>
Add at position	<code>insert(i, item)</code>	<code>lst.insert(0, 5)</code>
Add another list	<code>extend(list)</code>	<code>lst.extend([1,2])</code>
Remove by value	<code>remove(item)</code>	<code>lst.remove(5)</code>
Remove by index	<code>pop(i)</code>	<code>lst.pop(0)</code>
Remove all	<code>clear()</code>	<code>lst.clear()</code>
Reverse	<code>reverse()</code>	<code>lst.reverse()</code>
Sort	<code>sort()</code>	<code>lst.sort()</code>

CHAPTER 8.4: List Methods – Built-in List Operations

What You'll Learn

- All important list methods
- `index()` – find position
- `count()` – count occurrences
- `copy()` – shallow copy
- List methods summary

Finding Elements

`index()` – Find Position

```
fruits = ["apple", "banana", "orange", "banana"]

# Find first occurrence
pos = fruits.index("banana")
print(pos)  # 1

# Find with start position
pos = fruits.index("banana", 2)  # Start searching from index 2
print(pos)  # 3

# ValueError if not found
# fruits.index("grape")  # ValueError
```

`count()` – Count Occurrences

```
fruits = ["apple", "banana", "orange", "banana", "apple"]

print(fruits.count("apple"))  # 2
print(fruits.count("banana"))  # 2
print(fruits.count("grape"))  # 0
```

Copying Lists

`copy()` – Shallow Copy

```
original = [1, 2, 3]
copy = original.copy()

copy[0] = 99
print(original)  # [1, 2, 3] (unchanged)
print(copy)      # [99, 2, 3]

# Alternative ways to copy
copy2 = original[:]
copy3 = list(original)
```

Shallow vs Deep Copy (for nested lists)

```
import copy

nested = [[1, 2], [3, 4]]

# Shallow copy (inner lists are still shared)
shallow = nested.copy()
shallow[0][0] = 99
print(nested)  # [[99, 2], [3, 4]] (changed!)

# Deep copy (everything is copied)
deep = copy.deepcopy(nested)
deep[0][0] = 100
print(nested)  # [[99, 2], [3, 4]] (unchanged)
print(deep)    # [[100, 2], [3, 4]]
```

Other Useful Methods

`sort()` with Key

```
words = ["apple", "Banana", "cherry", "DATE"]

# Case-sensitive sort (uppercase first)
words.sort()
print(words)  # ['Banana', 'DATE', 'apple', 'cherry']

# Case-insensitive sort
words.sort(key=str.lower)
print(words)  # ['apple', 'Banana', 'cherry', 'DATE']

# Sort by length
words.sort(key=len)
print(words)  # ['DATE', 'apple', 'Banana', 'cherry']
```

`reverse()` – Reverse Order

```
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(numbers)  # [5, 4, 3, 2, 1]
```

List Methods Summary Table

Method	Description	Example
<code>append(x)</code>	Add x to end	<code>lst.append(5)</code>
<code>extend(iter)</code>	Add all items from iterable	<code>lst.extend([1,2])</code>
<code>insert(i, x)</code>	Insert x at index i	<code>lst.insert(0, 5)</code>
<code>remove(x)</code>	Remove first x	<code>lst.remove(5)</code>
<code>pop(i)</code>	Remove and return item at i	<code>lst.pop(0)</code>
<code>clear()</code>	Remove all items	<code>lst.clear()</code>
<code>index(x)</code>	Return index of first x	<code>lst.index(5)</code>
<code>count(x)</code>	Count occurrences of x	<code>lst.count(5)</code>
<code>sort()</code>	Sort in place	<code>lst.sort()</code>
<code>reverse()</code>	Reverse in place	<code>lst.reverse()</code>
<code>copy()</code>	Return shallow copy	<code>lst.copy()</code>

Real-World Examples

Finding Duplicates

```
def find_duplicates(lst):
    seen = []
    duplicates = []
    for item in lst:
        if lst.count(item) > 1 and item not in seen:
            duplicates.append(item)
            seen.append(item)
    return duplicates

numbers = [1, 2, 3, 2, 4, 5, 3, 6, 1]
print(find_duplicates(numbers))  # [1, 2, 3]
```

Most Common Element

```
def most_common(lst):
    if not lst:
        return None
    return max(set(lst), key=lst.count)

colors = ["red", "blue", "red", "green", "red", "blue"]
print(most_common(colors))  # red
```

Remove Duplicates (Preserve Order)

```
def remove_duplicates(lst):
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

numbers = [1, 2, 3, 2, 4, 5, 3, 6, 1]
print(remove_duplicates(numbers))  # [1, 2, 3, 4, 5, 6]
```

Check if List is Sorted

```
def is_sorted(lst):
    return lst == sorted(lst)

print(is_sorted([1, 2, 3, 4, 5]))  # True
print(is_sorted([1, 3, 2, 4, 5]))  # False
```

Try It Yourself

Challenge 1: Find the index of the number 42 in a list.

Challenge 2: Count how many times "apple" appears in a list.

Challenge 3: Create a copy of a list and verify it's independent.

Challenge 4: Sort a list of strings by length (shortest to longest).

Chapter Summary

Method	Returns	Modifies Original
<code>index()</code>	Integer (position)	No
<code>count()</code>	Integer (count)	No
<code>copy()</code>	New list	No
<code>sort()</code>	None	Yes
<code>reverse()</code>	None	Yes

CHAPTER 8.5: Nested Lists – Lists Within Lists

What You'll Learn

- Creating nested lists
- Accessing elements in nested lists
- Modifying nested lists
- Iterating over nested lists
- Practical applications (matrices, grids)

What Are Nested Lists?

A **nested list** is a list that contains other lists as elements.

```
# Simple nested list
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix)
# [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Accessing Nested List Elements

Use multiple indices:


```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
# Access entire row  
print(matrix[0])    # [1, 2, 3]  
print(matrix[1])    # [4, 5, 6]  
  
# Access single element (row, column)  
print(matrix[0][0]) # 1 (row 0, col 0)  
print(matrix[1][2]) # 6 (row 1, col 2)  
print(matrix[2][1]) # 8 (row 2, col 1)
```

Modifying Nested Lists

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
# Change a single element  
matrix[1][1] = 99  
print(matrix) # [[1, 2, 3], [4, 99, 6], [7, 8, 9]]  
  
# Change an entire row  
matrix[2] = [0, 0, 0]  
print(matrix) # [[1, 2, 3], [4, 99, 6], [0, 0, 0]]  
  
# Add a new row  
matrix.append([10, 11, 12])  
print(matrix) # [[1, 2, 3], [4, 99, 6], [0, 0, 0], [10, 11, 12]]
```

Creating Nested Lists

```
# Method 1: Direct
matrix = [[1, 2], [3, 4]]

# Method 2: Using list comprehension
rows, cols = 3, 4
matrix = [[0] * cols for _ in range(rows)]
print(matrix) # [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]

# ⚠️ Wrong way (creates references to same row!)
wrong = [[0] * 3] * 3
wrong[0][0] = 1
print(wrong) # [[1, 0, 0], [1, 0, 0], [1, 0, 0]] (all rows changed!)
```

Iterating Over Nested Lists

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Nested loops
for row in matrix:
    for item in row:
        print(item, end=" ")
    print()

# Output:
# 1 2 3
# 4 5 6
# 7 8 9

# With indices
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        print(f"matrix[{i}][{j}] = {matrix[i][j]}")
```

Flattening Nested Lists

```
def flatten(nested):
    result = []
    for row in nested:
        result.extend(row)
    return result

matrix = [[1, 2], [3, 4], [5, 6]]
flat = flatten(matrix)
print(flat)  # [1, 2, 3, 4, 5, 6]

# Using list comprehension
flat = [item for row in matrix for item in row]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

Real-World Examples

2D Grid (Game Board)

```
# Tic-Tac-Toe board
board = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "]
]

def print_board(board):
    for row in board:
        print("|".join(row))
        print("-" * 5)

def make_move(board, row, col, player):
    if board[row][col] == " ":
        board[row][col] = player
        return True
    return False

make_move(board, 1, 1, "X")
make_move(board, 0, 0, "O")
make_move(board, 2, 2, "X")
print_board(board)

# X| |
# ---
#  |X|
# ---
#  | |X
```

Student Grades

```
# Student grades: [name, [scores]]

students = [
    ["Ryu", [95, 88, 92]],
    ["Anna", [87, 90, 85]],
    ["Max", [92, 94, 89]]
]

# Calculate averages
for student in students:
    name = student[0]
    scores = student[1]
    average = sum(scores) / len(scores)
    print(f"{name}: {average:.1f}")

# Ryu: 91.7
# Anna: 87.3
# Max: 91.7
```

Matrix Operations

```
def add_matrices(a, b):
    """Add two matrices of same size"""
    rows = len(a)
    cols = len(a[0])
    result = [[0] * cols for _ in range(rows)]

    for i in range(rows):
        for j in range(cols):
            result[i][j] = a[i][j] + b[i][j]
    return result

matrix_a = [[1, 2], [3, 4]]
matrix_b = [[5, 6], [7, 8]]
result = add_matrices(matrix_a, matrix_b)
print(result)  # [[6, 8], [10, 12]]
```

Finding Maximum in Nested List

```
def max_in_nested(nested):
    max_val = float('-inf')
    for row in nested:
        for item in row:
            if item > max_val:
                max_val = item
    return max_val

matrix = [[5, 2, 8], [1, 9, 3], [7, 4, 6]]
print(max_in_nested(matrix))  # 9
```

Try It Yourself

- Challenge 1:** Create a 3x3 matrix with numbers 1-9.
- Challenge 2:** Print all elements of a 2D list row by row.
- Challenge 3:** Create a 4x4 identity matrix (1s on diagonal, 0s elsewhere).
- Challenge 4:** Find the sum of all elements in a 2D list.

Chapter Summary

Operation	Example	Result
Access element	<code>matrix[row][col]</code>	Single element
Access row	<code>matrix[row]</code>	List (row)
Iterate	<code>for row in matrix:</code>	Each row
Create	<code>[[0]*cols for _ in range(rows)]</code>	New matrix

CHAPTER 8.6: List Comprehensions – Concise List Creation

What You'll Learn

- What list comprehensions are (one-line list creation)
- Basic syntax: `[expr for item in iterable]`

- Adding conditions: `[expr for item in iterable if condition]`
- Nested list comprehensions

What Are List Comprehensions?

List comprehensions provide a concise way to create lists. They are often more readable and faster than traditional loops.

```
# Traditional loop
squares = []
for x in range(10):
    squares.append(x ** 2)

# List comprehension (same result)
squares = [x ** 2 for x in range(10)]

print(squares)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Basic Syntax

```
[expression for item in iterable]
```

```
# List of numbers
numbers = [x for x in range(5)]
print(numbers)  # [0, 1, 2, 3, 4]

# Double each number
doubled = [x * 2 for x in range(5)]
print(doubled)  # [0, 2, 4, 6, 8]

# Convert to string
strings = [str(x) for x in range(5)]
print(strings)  # ['0', '1', '2', '3', '4']
```

List Comprehensions with Condition

```
[expression for item in iterable if condition]
```

```
numbers = range(10)

# Even numbers only
evens = [x for x in numbers if x % 2 == 0]
print(evens)  # [0, 2, 4, 6, 8]

# Numbers greater than 5
greater = [x for x in numbers if x > 5]
print(greater)  # [6, 7, 8, 9]

# Even numbers, squared
even_squares = [x ** 2 for x in numbers if x % 2 == 0]
print(even_squares)  # [0, 4, 16, 36, 64]
```

if-else in List Comprehension

```
[true_value if condition else false_value for item in iterable]
```

```
numbers = range(5)

# Even -> "even", Odd -> "odd"
labels = ["even" if x % 2 == 0 else "odd" for x in numbers]
print(labels)  # ['even', 'odd', 'even', 'odd', 'even']

# Positive -> number, Negative -> 0
values = [-3, -1, 0, 2, 5]
processed = [x if x > 0 else 0 for x in values]
print(processed)  # [0, 0, 0, 2, 5]
```


Nested List Comprehensions

```
# Create a matrix
matrix = [[x for x in range(3)] for y in range(3)]
print(matrix) # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]

# Flatten a matrix
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [item for row in matrix for item in row]
print(flat) # [1, 2, 3, 4, 5, 6]
```

Real-World Examples

Filtering a List

```
scores = [85, 92, 58, 76, 44, 95, 67, 88]

# Passing scores (>= 60)
passing = [s for s in scores if s >= 60]
print(passing) # [85, 92, 76, 95, 67, 88]

# Failing scores (< 60)
failing = [s for s in scores if s < 60]
print(failing) # [58, 44]
```

Transforming Data

```
names = ["ryuu", "anna", "max"]

# Capitalize
capitalized = [name.capitalize() for name in names]
print(capitalized) # ['Ryu', 'Anna', 'Max']

# Add prefix
prefixed = [f"User: {name}" for name in names]
print(prefixed) # ['User: ryuu', 'User: anna', 'User: max']
```

Working with Strings

```
text = "Hello World"

vowels = [c for c in text if c.lower() in "aeiou"]
print(vowels)  # ['e', 'o', 'o']

# Uppercase consonants
consonants = [c.upper() for c in text if c.isalpha() and c.lower() not in "aeiou"]
print(consonants)  # ['H', 'L', 'L', 'W', 'R', 'L', 'D']
```

Data Cleaning

```
# Remove empty strings
words = ["hello", "", "world", "", "python"]
cleaned = [w for w in words if w]
print(cleaned)  # ['hello', 'world', 'python']

# Remove None values
data = [1, None, 2, None, 3, None, 4]
cleaned = [x for x in data if x is not None]
print(cleaned)  # [1, 2, 3, 4]

# Strip whitespace
messy = [" hello ", "world ", " python"]
stripped = [s.strip() for s in messy]
print(stripped)  # ['hello', 'world', 'python']
```

Set and Dictionary Comprehensions

```
# Set comprehension (curly braces)
squares_set = {x ** 2 for x in range(5)}
print(squares_set)  # {0, 1, 4, 9, 16}

# Dictionary comprehension
squares_dict = {x: x ** 2 for x in range(5)}
print(squares_dict)  # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

List Comprehension vs Traditional Loop

Aspect	List Comprehension	Traditional Loop
Lines of code	1 line	3+ lines
Readability (simple)	✔ Excellent	✔ Good
Readability (complex)	✘ Poor	✔ Good
Performance	✔ Faster	⚠ Slower
When to use	Simple transformations	Complex logic, side effects

When NOT to use list comprehensions

```
# ✘ Too complex (hard to read)
result = [x for x in range(100) if x % 2 == 0 if x % 3 == 0 if x % 5 == 0]

# ✔ Use loop for complex logic
result = []
for x in range(100):
    if x % 2 == 0 and x % 3 == 0 and x % 5 == 0:
        result.append(x)

# ✘ Side effects (printing)
# [print(x) for x in range(5)] # Don't do this

# ✔ Use regular loop
for x in range(5):
    print(x)
```

Try It Yourself

- Challenge 1:** Use a list comprehension to create a list of squares for numbers 1-10.
- Challenge 2:** Use a list comprehension to filter a list of numbers, keeping only those divisible by 3.
- Challenge 3:** Use a list comprehension to convert a list of temperatures from Celsius to Fahrenheit.
- Challenge 4:** Use a nested list comprehension to create a 5x5 multiplication table.

Chapter Summary

Pattern	Syntax	Example
Basic	<code>[expr for item in iter]</code>	<code>[x*2 for x in range(5)]</code>
With condition	<code>[expr for item in iter if cond]</code>	<code>[x for x in range(10) if x%2==0]</code>
if-else	<code>[if_true if cond else if_false for item in iter]</code>	<code>['even' if x%2==0 else 'odd' for x in range(5)]</code>
Nested	<code>[expr for row in matrix for item in row]</code>	<code>[item for row in matrix for item in row]</code>

PART 8 SUMMARY

You now know all about lists:

Chapter	Topic
8.1	Creating lists
8.2	Accessing elements
8.3	Modifying lists
8.4	List methods
8.5	Nested lists
8.6	List comprehensions

What's Next?

Next up: **Part 9: Tuples** – Immutable sequences!

PART 9: TUPLES

CHAPTER 9.1: Creating Tuples – Immutable Sequences

What You'll Learn

- What tuples are (immutable, ordered collections)
- Creating tuples with parentheses
- Tuples vs lists
- When to use tuples

What Is a Tuple?

A **tuple** is an ordered, immutable collection. Once created, you cannot change, add, or remove elements.

```
# Tuple with parentheses
colors = ("red", "green", "blue")

# Tuple without parentheses (tuple packing)
colors = "red", "green", "blue"

# Single element tuple (note the comma!)
single = (42,) # Without comma, it's just a number

print(colors)    # ('red', 'green', 'blue')
print(type(colors)) # <class 'tuple'>
```

Creating Tuples

```
# Empty tuple
empty1 = ()
empty2 = tuple()

# Tuple with values
numbers = (1, 2, 3, 4, 5)
fruits = ("apple", "banana", "orange")

# Using tuple() constructor
numbers = tuple([1, 2, 3]) # Convert list to tuple
string_tuple = tuple("hello") # ('h', 'e', 'l', 'l', 'o')

# Mixed types (allowed)
mixed = (1, "hello", 3.14, True)
```

Tuples vs Lists

Feature	Tuple	List
Mutable	✗ No (cannot change)	✓ Yes
Syntax	(1, 2, 3)	[1, 2, 3]
Speed	✓ Faster	⚠ Slower
Memory	✓ Uses less	⚠ Uses more
Use for	Fixed data	Changing data

```
# List (mutable)
my_list = [1, 2, 3]
my_list[0] = 99 # Works
my_list.append(4) # Works

# Tuple (immutable)
my_tuple = (1, 2, 3)
# my_tuple[0] = 99 # TypeError! Can't modify tuple
# my_tuple.append(4) # AttributeError! No append method
```

Tuple Packing and Unpacking

```
# Packing (creating tuple)
point = 10, 20 # Automatically packed into tuple
print(point)   # (10, 20)

# Unpacking (extracting values)
x, y = point
print(x) # 10
print(y) # 20

# Swap variables (uses tuple unpacking)
a, b = 5, 10
a, b = b, a # Swaps!
print(a, b) # 10, 5
```

Real-World Examples

Days of Week (Fixed)

```
DAYS = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")

def is_weekend(day):
    return day == "Saturday" or day == "Sunday"

print(DAYS[0]) # Monday
print(is_weekend("Saturday")) # True
```

RGB Colors

```
# RGB color as tuple (red, green, blue)

RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

def mix_colors(color1, color2):
    return tuple((c1 + c2) // 2 for c1, c2 in zip(color1, color2))

print(mix_colors(RED, BLUE)) # (127, 0, 127) - purple
```

Coordinates

```
# Points in 2D space

origin = (0, 0)
point = (10, 20)

def distance(p1, p2):
    return ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5

print(distance(origin, point)) # ~22.36
```

Constants

```
# Application settings (should never change)

APP_CONFIG = {
    "VERSION": (1, 0, 0),
    "SUPPORTED_LANGUAGES": ("en", "es", "fr", "ja"),
    "DEFAULT_THEME": ("dark", "blue"),
}

print(APP_CONFIG["VERSION"]) # (1, 0, 0)
```

Try It Yourself

Challenge 1: Create a tuple of your 5 favorite movies.

Challenge 2: Create a single-element tuple (note the comma).

Challenge 3: Pack three values into a tuple, then unpack them into three variables.

Challenge 4: Try to modify a tuple. What error do you get?

Chapter Summary

Operation	Syntax	Example
Create tuple	<code>(a, b, c)</code>	<code>colors = ("red", "green")</code>
Single element	<code>(value,)</code>	<code>single = (42,)</code>
Empty tuple	<code>()</code>	<code>empty = ()</code>
Packing	<code>var = a, b</code>	<code>point = 10, 20</code>
Unpacking	<code>a, b = tuple</code>	<code>x, y = point</code>

CHAPTER 9.2: Accessing Tuple Elements – Reading from Tuples

What You'll Learn

- Accessing elements by index (positive and negative)
- Slicing tuples
- Iterating through tuples
- Checking membership

Accessing by Index

Like lists, tuple indices start at **0**:

```
fruits = ("apple", "banana", "orange", "mango", "grape")  
  
# Index:      0          1          2          3          4  
  
print(fruits[0]) # apple  
print(fruits[1]) # banana  
print(fruits[2]) # orange  
print(fruits[3]) # mango  
print(fruits[4]) # grape  
  
# print(fruits[5]) # IndexError!
```

Negative Indexing

Negative indices count from the end:

```
fruits = ("apple", "banana", "orange", "mango", "grape")  
  
# Index:    -5         -4         -3         -2         -1  
  
print(fruits[-1]) # grape (last)  
print(fruits[-2]) # mango  
print(fruits[-3]) # orange  
print(fruits[-4]) # banana  
print(fruits[-5]) # apple
```

Slicing Tuples

Slicing extracts a portion of a tuple (returns a new tuple):

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

# From index 2 to 5 (not including 5)
print(numbers[2:5])    # (2, 3, 4)

# From start to index 5
print(numbers[:5])     # (0, 1, 2, 3, 4)

# From index 5 to end
print(numbers[5:])     # (5, 6, 7, 8, 9)

# Whole tuple (copy)
print(numbers[:])      # (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

# With step
print(numbers[::2])    # (0, 2, 4, 6, 8)
print(numbers[::-1])  # (9, 8, 7, 6, 5, 4, 3, 2, 1, 0)
```

Iterating Over Tuples

```
colors = ("red", "green", "blue")

# Simple iteration
for color in colors:
    print(color)

# With index
for i, color in enumerate(colors):
    print(f"{i}: {color}")

# Using range
for i in range(len(colors)):
    print(colors[i])
```

Checking Membership

```
fruits = ("apple", "banana", "orange")

print("apple" in fruits)    # True
print("grape" in fruits)   # False
print("apple" not in fruits) # False

# Using in with if
if "banana" in fruits:
    print("Found banana!")
```

Tuple Methods

Tuples have only two methods (since they're immutable):

```
numbers = (1, 2, 3, 2, 4, 2, 5)

# count() - count occurrences
print(numbers.count(2))  # 3
print(numbers.count(9))  # 0

# index() - find first occurrence
print(numbers.index(2))  # 1
print(numbers.index(4))  # 4

# index() with start/end
print(numbers.index(2, 2))  # 3 (search from index 2)
```

Real-World Examples

Using Tuples as Dictionary Keys

```
# Tuples can be dictionary keys (lists cannot!)
coordinates = {
    (0, 0): "Origin",
    (1, 0): "Right",
    (0, 1): "Up",
    (-1, 0): "Left",
    (0, -1): "Down"
}

print(coordinates[(0, 0)]) # Origin

# Get value with default
print(coordinates.get((2, 2), "Unknown")) # Unknown
```

Function Return Values

```
def get_min_max(numbers):
    return min(numbers), max(numbers)

scores = [85, 92, 78, 96, 88]
lowest, highest = get_min_max(scores)
print(f"Lowest: {lowest}, Highest: {highest}") # Lowest: 78, Highest: 96
```

Multiple Assignment

```
# Unpacking tuple into variables
person = ("Ryuu", 11, "Tokyo")
name, age, city = person

print(name)  # Ryuu
print(age)   # 11
print(city)  # Tokyo

# Ignoring values with _
data = (1, 2, 3, 4, 5)
first, second, *rest = data
print(first)  # 1
print(second) # 2
print(rest)   # [3, 4, 5]
```

Try It Yourself

- Challenge 1:** Print the first and last element of a tuple using positive and negative indexing.
- Challenge 2:** Get a slice containing the middle three elements of a 7-element tuple.
- Challenge 3:** Iterate through a tuple and print each element.
- Challenge 4:** Use `in` to check if a value exists in a tuple.

Chapter Summary

Operation	Syntax	Example
Access by index	<code>tup[i]</code>	<code>fruits[0]</code>
Negative index	<code>tup[-i]</code>	<code>fruits[-1]</code>
Slice	<code>tup[start:end]</code>	<code>numbers[2:5]</code>
Slice with step	<code>tup[start:end:step]</code>	<code>numbers[:2]</code>
Count	<code>tup.count(x)</code>	<code>numbers.count(2)</code>
Index	<code>tup.index(x)</code>	<code>numbers.index(2)</code>

CHAPTER 9.3: Tuple Operations – Working with Tuples

What You'll Learn

- Concatenating tuples (`+`)
- Repeating tuples (`*`)
- Comparing tuples
- Tuple methods (`count` , `index`)

Concatenation (`+`)

Combine two or more tuples:

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)

result = tuple1 + tuple2
print(result)  # (1, 2, 3, 4, 5, 6)

# Multiple concatenation
result = tuple1 + tuple2 + (7, 8, 9)
print(result)  # (1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Repetition (`*`)

Repeat a tuple multiple times:

```
base = (1, 2)
result = base * 3
print(result)  # (1, 2, 1, 2, 1, 2)

# Create a tuple with repeated values
zeros = (0,) * 5
print(zeros)  # (0, 0, 0, 0, 0)

# Create a pattern
pattern = ("x", "y") * 4
print(pattern)  # ('x', 'y', 'x', 'y', 'x', 'y', 'x', 'y')
```

Comparing Tuples

Tuples are compared lexicographically (element by element):

```
# Equal tuples
print((1, 2) == (1, 2))  # True
print((1, 2) == (2, 1))  # False (order matters)

# Less than / greater than
print((1, 2) < (1, 3))    # True (1 == 1, then 2 < 3)
print((1, 2) < (2, 1))    # True (1 < 2)
print((1, 2, 3) < (1, 2)) # False (3 > 2)

# Different lengths
print((1, 2) < (1, 2, 3)) # True (first two equal, second tuple longer)
```


Finding Elements

```
fruits = ("apple", "banana", "orange", "banana")

# count() - how many times?
print(fruits.count("banana")) # 2
print(fruits.count("grape")) # 0

# index() - where is it?
print(fruits.index("banana")) # 1
print(fruits.index("orange")) # 2

# index() with start parameter
print(fruits.index("banana", 2)) # 3 (search from index 2)

# ValueError if not found
# fruits.index("grape") # ValueError
```

Tuple Length

```
fruits = ("apple", "banana", "orange")
print(len(fruits)) # 3

empty = ()
print(len(empty)) # 0
```

Membership Testing

```
fruits = ("apple", "banana", "orange")

print("apple" in fruits) # True
print("grape" in fruits) # False
print("apple" not in fruits) # False

if "banana" in fruits:
    print("Found banana!")
```

Real-World Examples

Building a Menu System

```
MENU = (  
    ("1", "Start Game"),  
    ("2", "Load Game"),  
    ("3", "Settings"),  
    ("4", "Quit")  
)  
  
def show_menu():  
    for code, text in MENU:  
        print(f"{code}. {text}")  
  
show_menu()  
  
# 1. Start Game  
# 2. Load Game  
# 3. Settings  
# 4. Quit
```

Data Validation

```
def validate_coordinates(point):  
    VALID_RANGE = (-100, 100)  
    x, y = point  
    return VALID_RANGE[0] <= x <= VALID_RANGE[1] and \  
        VALID_RANGE[0] <= y <= VALID_RANGE[1]  
  
print(validate_coordinates((50, 75)))    # True  
print(validate_coordinates((150, 50)))   # False
```

Combining Data

```
# Combine user data from multiple sources
name = ("Ryuu", "Swift")
age = (11,)
city = ("Tokyo",)

user = name + age + city
print(user)  # ('Ryuu', 'Swift', 11, 'Tokyo')
```

Try It Yourself

- Challenge 1:** Concatenate two tuples `(1, 2, 3)` and `(4, 5, 6)`.
- Challenge 2:** Create a tuple of five zeros using repetition.
- Challenge 3:** Compare `(1, 2, 3)` and `(1, 2, 4)`. Which is larger?
- Challenge 4:** Count how many times "a" appears in `("a", "b", "a", "c", "a")`.

Chapter Summary

Operation	Operator	Example
Concatenation	<code>+</code>	<code>(1,2) + (3,4) = (1,2,3,4)</code>
Repetition	<code>*</code>	<code>(1,2) * 3 = (1,2,1,2,1,2)</code>
Equality	<code>==</code>	<code>(1,2) == (1,2)</code>
Length	<code>len()</code>	<code>len((1,2,3)) = 3</code>
Count	<code>.count()</code>	<code>(1,2,2).count(2) = 2</code>
Index	<code>.index()</code>	<code>(1,2,3).index(2) = 1</code>

CHAPTER 9.4: Packing and Unpacking – Tuple Assignment

What You'll Learn

- Tuple packing (creating tuples automatically)
- Tuple unpacking (extracting values)

- Extended unpacking with 
- Swapping variables

Tuple Packing

Packing is when multiple values are automatically grouped into a tuple:

```
# Packing (no parentheses needed)
point = 10, 20
print(point)          # (10, 20)
print(type(point))    # <class 'tuple'>

# Multiple values
data = 1, "hello", 3.14, True
print(data)           # (1, 'hello', 3.14, True)
```

Tuple Unpacking

Unpacking extracts tuple values into individual variables:

```
# Basic unpacking
point = (10, 20)
x, y = point
print(x)  # 10
print(y)  # 20

# Unpacking with parentheses (optional)
(x, y) = point

# Unpacking list also works
numbers = [1, 2, 3]
a, b, c = numbers
print(a, b, c)  # 1 2 3
```

Number of Variables Must Match

```
point = (10, 20, 30)

#  Correct
x, y, z = point

#  Too few variables
# x, y = point # ValueError: too many values to unpack

#  Too many variables
# x, y, z, w = point # ValueError: not enough values
```

Extended Unpacking with

Use  to capture remaining values as a list:

```
numbers = (1, 2, 3, 4, 5)

# Capture first, then rest
first, *rest = numbers
print(first) # 1
print(rest) # [2, 3, 4, 5]

# Capture first two, then rest
first, second, *rest = numbers
print(first) # 1
print(second) # 2
print(rest) # [3, 4, 5]

# Capture rest, then last
*rest, last = numbers
print(rest) # [1, 2, 3, 4]
print(last) # 5

# Capture first, middle, last
first, *middle, last = numbers
print(first) # 1
print(middle) # [2, 3, 4]
print(last) # 5
```

Swapping Variables

Tuple unpacking makes swapping elegant:

```
# Traditional swap (needs temporary variable)
a = 5
b = 10
temp = a
a = b
b = temp

# Python swap (no temporary variable)
a = 5
b = 10
a, b = b, a
print(a, b)  # 10, 5
```

Returning Multiple Values

```
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers) / len(numbers)

scores = [85, 92, 78, 96, 88]
lowest, highest, average = get_stats(scores)

print(f"Lowest: {lowest}")      # Lowest: 78
print(f"Highest: {highest}")    # Highest: 96
print(f"Average: {average:.2f}") # Average: 87.80
```

Real-World Examples

Parsing Data

```
# Parse CSV line
line = "Ryu,11,Tokyo"
name, age, city = line.split(",")
print(f"Name: {name}, Age: {age}, City: {city}")
# Name: Ryu, Age: 11, City: Tokyo

# Parse with different formats
data = "2024-01-15"
year, month, day = data.split("-")
print(f"Year: {year}, Month: {month}, Day: {day}")
```

Ignoring Values

```
# Use _ for values you don't need
coordinates = (10, 20, 30)
x, _, z = coordinates
print(x, z) # 10 30

# Ignore first and last
numbers = (1, 2, 3, 4, 5)
_, *middle, _ = numbers
print(middle) # [2, 3, 4]
```

Function with Multiple Returns

```
def divide_with_remainder(a, b):
    return a // b, a % b

quotient, remainder = divide_with_remainder(17, 5)
print(f"17 ÷ 5 = {quotient} remainder {remainder}")
# 17 ÷ 5 = 3 remainder 2
```

Enumerate with Unpacking

```
fruits = ["apple", "banana", "orange"]

for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# 0: apple
# 1: banana
# 2: orange
```

Try It Yourself

Challenge 1: Pack `10, "hello", 3.14` into a tuple, then unpack into three variables.

Challenge 2: Swap two variables using tuple unpacking.

Challenge 3: Use `*` to get the first, last, and middle elements of a tuple.

Challenge 4: Create a function that returns both the minimum and maximum of a list.

Chapter Summary

Concept	Syntax	Example
Packing	<code>a, b = value1, value2</code>	<code>x, y = 10, 20</code>
Unpacking	<code>a, b = tuple</code>	<code>x, y = (10, 20)</code>
Extended unpacking	<code>*rest</code>	<code>first, *rest = (1,2,3,4)</code>
Ignore values	<code>_</code>	<code>x, _, z = (1,2,3)</code>
Swap	<code>a, b = b, a</code>	<code>x, y = y, x</code>

CHAPTER 9.5: Tuple Methods – count() and index()

What You'll Learn

- The `count()` method
- The `index()` method

- Why tuples have only two methods
- Practical examples

The `count()` Method

Returns the number of times a value appears in the tuple:

```
numbers = (1, 2, 3, 2, 4, 2, 5)

print(numbers.count(2))    # 3
print(numbers.count(1))    # 1
print(numbers.count(9))    # 0

fruits = ("apple", "banana", "orange", "banana", "apple")
print(fruits.count("apple"))    # 2
print(fruits.count("banana"))   # 2
print(fruits.count("grape"))    # 0
```

The `index()` Method

Returns the index of the first occurrence of a value:

```
fruits = ("apple", "banana", "orange", "banana")

print(fruits.index("apple"))    # 0
print(fruits.index("banana"))   # 1 (first occurrence)
print(fruits.index("orange"))   # 2

# ValueError if not found
# fruits.index("grape")         # ValueError
```

`index()` with Start and End Parameters

```
fruits = ("apple", "banana", "orange", "banana", "mango")

# Start searching from index 2
print(fruits.index("banana", 2)) # 3

# Search between index 1 and 4
print(fruits.index("orange", 1, 4)) # 2

# Value not found in range
# fruits.index("mango", 0, 4) # ValueError (mango at index 4, not in 0-4)
```

Why Only Two Methods?

Tuples are **immutable** — they cannot be changed. Therefore, they don't have methods like `append()`, `insert()`, `remove()`, `pop()`, `sort()`, or `reverse()`.

The only methods they have are:

- `count()` — doesn't modify
- `index()` — doesn't modify

Real-World Examples

Finding All Occurrences

```
def find_all(tuple_data, value):
    """Find all indices where value appears"""
    indices = []
    start = 0
    while True:
        try:
            idx = tuple_data.index(value, start)
            indices.append(idx)
            start = idx + 1
        except ValueError:
            break
    return indices

numbers = (1, 2, 3, 2, 4, 2, 5)
print(find_all(numbers, 2)) # [1, 3, 5]
```

Frequency Counter

```
def frequency(tuple_data):
    """Count frequency of each unique element"""
    result = {}
    for item in tuple_data:
        result[item] = tuple_data.count(item)
    return result

numbers = (1, 2, 3, 2, 4, 2, 5, 3)
print(frequency(numbers)) # {1: 1, 2: 3, 3: 2, 4: 1, 5: 1}
```

Most Common Element

```
def most_common(tuple_data):
    if not tuple_data:
        return None
    return max(set(tuple_data), key=tuple_data.count)

colors = ("red", "blue", "red", "green", "red", "blue")
print(most_common(colors)) # red
```

Validating Data

```
VALID_STATUSES = ("pending", "processing", "completed", "failed")

def is_valid_status(status):
    return status in VALID_STATUSES

print(is_valid_status("completed")) # True
print(is_valid_status("cancelled")) # False
```

Try It Yourself

Challenge 1: Count how many times 5 appears in `(1, 5, 2, 5, 3, 5, 4, 5)`.

Challenge 2: Find the index of "hello" in `("hi", "hello", "hey", "hello")`.

Challenge 3: Use `count()` to find which number appears most frequently in a tuple.

Challenge 4: Use `index()` to find the position of the first negative number in a tuple.

Chapter Summary

Method	Description	Example
<code>count(x)</code>	Returns number of times x appears	<code>(1,2,2).count(2) = 2</code>
<code>index(x)</code>	Returns index of first x	<code>(1,2,3).index(2) = 1</code>
<code>index(x, start)</code>	Search from start index	<code>(1,2,3,2).index(2,2) = 3</code>
<code>index(x, start, end)</code>	Search in range	<code>(1,2,3).index(2,0,2) = 1</code>

PART 9 SUMMARY

You now know all about tuples:

Chapter	Topic
9.1	Creating tuples
9.2	Accessing elements
9.3	Tuple operations
9.4	Packing and unpacking
9.5	Tuple methods (count, index)

What's Next?

Next up: **Part 10: Sets** – Unordered collections of unique elements!

PART 10: SETS

CHAPTER 10.1: Creating Sets – Unordered Unique Collections

What You'll Learn

- What sets are (unordered, unique collections)
- Creating sets with curly braces
- Creating sets with `set()`
- Sets vs lists vs tuples

What Is a Set?

A **set** is an unordered collection of **unique** elements. Duplicates are automatically removed.

```
# Set with curly braces
fruits = {"apple", "banana", "orange", "apple"}
print(fruits)  # {'apple', 'banana', 'orange'} (apple appears once)

# Order is NOT guaranteed
numbers = {3, 1, 4, 1, 5, 9, 2}
print(numbers)  # {1, 2, 3, 4, 5, 9} (order may vary)
```

Creating Sets

```
# Empty set (must use set(), not {})  
empty_set = set()  
print(type(empty_set))  # <class 'set'>  
  
# Empty dictionary (not a set!)  
empty_dict = {}  
print(type(empty_dict))  # <class 'dict'>  
  
# Set from list  
numbers = set([1, 2, 2, 3, 3, 3])  
print(numbers)  # {1, 2, 3}  
  
# Set from string  
letters = set("hello")  
print(letters)  # {'h', 'e', 'l', 'o'} (each letter once)  
  
# Set with values  
colors = {"red", "green", "blue"}
```

Sets vs Lists vs Tuples

Feature	Set	List	Tuple
Order	✗ No	✓ Yes	✓ Yes
Duplicates	✗ No	✓ Yes	✓ Yes
Mutable	✓ Yes	✓ Yes	✗ No
Indexing	✗ No	✓ Yes	✓ Yes
Syntax	{1, 2}	[1, 2]	(1, 2)

```
# List (ordered, allows duplicates)
my_list = [1, 2, 2, 3]
print(my_list) # [1, 2, 2, 3]

# Set (unordered, no duplicates)
my_set = {1, 2, 2, 3}
print(my_set) # {1, 2, 3}

# Tuple (ordered, immutable)
my_tuple = (1, 2, 2, 3)
print(my_tuple) # (1, 2, 2, 3)
```

Set Properties

```
fruits = {"apple", "banana", "orange"}

# Length (number of unique items)
print(len(fruits)) # 3

# Check membership (very fast!)
print("apple" in fruits) # True
print("grape" in fruits) # False
print("apple" not in fruits) # False

# Iteration (order not guaranteed)
for fruit in fruits:
    print(fruit)
```

Removing Duplicates from a List

```
# Remove duplicates using set
numbers = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
unique = list(set(numbers))
print(unique) # [1, 2, 3, 4] (order may vary)

# Preserve original order (Python 3.7+)
def unique_ordered(lst):
    return list(dict.fromkeys(lst))

numbers = [3, 1, 2, 2, 3, 1, 4]
print(unique_ordered(numbers)) # [3, 1, 2, 4]
```

Real-World Examples

Unique Visitors

```
visitors = set()

def add_visitor(user_id):
    visitors.add(user_id)
    print(f"Total unique visitors: {len(visitors)}")

add_visitor("user123") # Total unique visitors: 1
add_visitor("user456") # Total unique visitors: 2
add_visitor("user123") # Total unique visitors: 2 (duplicate ignored)
```


Unique Tags

```
def extract_unique_tags(posts):
    tags = set()
    for post in posts:
        for tag in post.get("tags", []):
            tags.add(tag)
    return tags

posts = [
    {"title": "Post 1", "tags": ["python", "coding"]},
    {"title": "Post 2", "tags": ["python", "tutorial"]},
    {"title": "Post 3", "tags": ["coding", "beginner"]},
]

unique_tags = extract_unique_tags(posts)
print(unique_tags)  # {'python', 'coding', 'tutorial', 'beginner'}
```

Student Enrollment

```
# Students enrolled in different courses
math_students = {"Ryu", "Anna", "Max"}
science_students = {"Anna", "Lily", "Max"}
art_students = {"Ryu", "Lily"}

print(f"Math: {len(math_students)} students")
print(f"Science: {len(science_students)} students")
print(f"Art: {len(art_students)} students")
```

Try It Yourself

Challenge 1: Create a set of your favorite colors (try adding a duplicate).

Challenge 2: Convert a list with duplicates to a set and back to a list.

Challenge 3: Check if "Python" is in a set of programming languages.

Challenge 4: What happens if you try to create an empty set with `{}`? Try it.

Chapter Summary

Operation	Syntax	Example
Create set	<code>{a, b, c}</code>	<code>fruits = {"apple", "banana"}</code>
Empty set	<code>set()</code>	<code>empty = set()</code>
From list	<code>set(list)</code>	<code>set([1,2,2,3])</code> → <code>{1,2,3}</code>
Length	<code>len(set)</code>	<code>len(fruits)</code>
Add item	<code>set.add(item)</code>	<code>fruits.add("grape")</code>
Check membership	<code>item in set</code>	<code>"apple" in fruits</code>

CHAPTER 10.2: Set Operations – Adding and Removing Elements

What You'll Learn

- Adding elements with `add()`
- Removing elements (`remove()`, `discard()`, `pop()`)
- Clearing sets
- Copying sets

Adding Elements

`add()` – Add Single Element

```
fruits = {"apple", "banana"}
fruits.add("orange")
print(fruits)  # {'apple', 'banana', 'orange'}

# Adding duplicate has no effect
fruits.add("apple")
print(fruits)  # {'apple', 'banana', 'orange'} (unchanged)
```

`update()` – Add Multiple Elements

```
fruits = {"apple", "banana"}

# Add from another set
fruits.update({"orange", "mango"})
print(fruits) # {'apple', 'banana', 'orange', 'mango'}

# Add from list
fruits.update(["grape", "kiwi"])
print(fruits) # {'apple', 'banana', 'orange', 'mango', 'grape', 'kiwi'}

# Add from string (adds each character!)
letters = set()
letters.update("hello")
print(letters) # {'h', 'e', 'l', 'o'}
```

Removing Elements

`remove()` – Remove (Error if not found)

```
fruits = {"apple", "banana", "orange"}

fruits.remove("banana")
print(fruits) # {'apple', 'orange'}

# fruits.remove("grape") # KeyError! (element not found)
```

`discard()` – Remove (No error if not found)

```
fruits = {"apple", "banana", "orange"}

fruits.discard("banana")
print(fruits) # {'apple', 'orange'}

# No error if element not found
fruits.discard("grape") # Does nothing
print(fruits) # {'apple', 'orange'}
```

`pop()` – Remove and Return Arbitrary Element

```
fruits = {"apple", "banana", "orange"}

# Removes an arbitrary element (order not guaranteed)
removed = fruits.pop()
print(f"Removed: {removed}")
print(f"Remaining: {fruits}")

# Can't pop from empty set
# empty = set()
# empty.pop() # KeyError!
```

`clear()` – Remove All Elements

```
fruits = {"apple", "banana", "orange"}

fruits.clear()
print(fruits) # set()
```

Copying Sets

```
original = {1, 2, 3}

# Method 1: copy() method
copy1 = original.copy()

# Method 2: set() constructor
copy2 = set(original)

# Method 3: set unpacking
copy3 = {*original}

copy1.add(4)
print(original) # {1, 2, 3} (unchanged)
print(copy1)    # {1, 2, 3, 4}
```

Real-World Examples

Tracking Unique IP Addresses

```
unique_ips = set()

def log_request(ip):
    unique_ips.add(ip)
    print(f"Unique visitors today: {len(unique_ips)}")

log_request("192.168.1.1")
log_request("192.168.1.2")
log_request("192.168.1.1") # Duplicate, ignored
log_request("192.168.1.3")
# Unique visitors today: 3
```

Managing Shopping Cart (Unique Items)

```
cart = set()

def add_to_cart(item):
    cart.add(item)
    print(f"Added: {item}. Cart has {len(cart)} unique items.")

def remove_from_cart(item):
    cart.discard(item) # No error if not found
    print(f"Removed: {item} (if present)")

def show_cart():
    print(f"Cart: {cart}")

add_to_cart("Book") # Added: Book. Cart has 1 unique items.
add_to_cart("Pen") # Added: Pen. Cart has 2 unique items.
add_to_cart("Book") # Added: Book. Cart has 2 unique items. (duplicate)
remove_from_cart("Pen")
show_cart() # Cart: {'Book'}
```

Allowed Values Set

```
VALID_COLORS = {"red", "green", "blue", "yellow"}

def validate_color(color):
    if color in VALID_COLORS:
        print(f"{color} is valid")
    else:
        print(f"{color} is not valid")

validate_color("red")      # red is valid
validate_color("purple")  # purple is not valid

# Add new valid color at runtime
VALID_COLORS.add("purple")
validate_color("purple")  # purple is valid
```

Try It Yourself

- Challenge 1:** Create a set and add three numbers to it.
- Challenge 2:** Try to add a duplicate. What happens?
- Challenge 3:** Use `remove()` and `discard()` to remove an element. What's the difference?
- Challenge 4:** Create a copy of a set and verify that modifying the copy doesn't affect the original.

Chapter Summary

Operation	Method	Behavior if Not Found
Add single	<code>add(x)</code>	Adds element
Add multiple	<code>update(iter)</code>	Adds all elements
Remove	<code>remove(x)</code>	✗ <code>KeyError</code>
Discard	<code>discard(x)</code>	✓ No error
Pop	<code>pop()</code>	✗ <code>KeyError</code> if empty
Clear	<code>clear()</code>	Removes all
Copy	<code>copy()</code>	Returns new set

CHAPTER 10.3: Union and Intersection – Combining Sets

What You'll Learn

- Union (`|`) – all elements from both sets
- Intersection (`&`) – elements in both sets
- Using methods (`union()` , `intersection()`)
- Practical applications

Union (`|`)

Union returns a new set containing **all elements** from both sets (no duplicates).

```
set_a = {1, 2, 3}
set_b = {3, 4, 5}

# Using operator
union = set_a | set_b
print(union)  # {1, 2, 3, 4, 5}

# Using method
union = set_a.union(set_b)
print(union)  # {1, 2, 3, 4, 5}

# Multiple sets
set_c = {5, 6, 7}
union = set_a | set_b | set_c
print(union)  # {1, 2, 3, 4, 5, 6, 7}
```

Intersection (`&`)

Intersection returns a new set containing **only elements in both** sets.

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Using operator
intersection = set_a & set_b
print(intersection)  # {3, 4}

# Using method
intersection = set_a.intersection(set_b)
print(intersection)  # {3, 4}

# Multiple sets
set_c = {4, 5, 6, 7}
intersection = set_a & set_b & set_c
print(intersection)  # {4}
```

Visual Examples

```
# Students in different courses
math = {"Ryu", "Anna", "Max", "Lily"}
science = {"Anna", "Max", "John", "Emma"}
art = {"Ryu", "Lily", "Emma", "Tom"}

# Students in math OR science
math_or_science = math | science
print(math_or_science)  # {'Ryu', 'Anna', 'Max', 'Lily', 'John', 'Emma'}

# Students in math AND science
math_and_science = math & science
print(math_and_science)  # {'Anna', 'Max'}

# Students in all three courses
all_three = math & science & art
print(all_three)  # set() (no one in all three)
```


Real-World Examples

Friend Recommendations

```
my_friends = {"Anna", "Bob", "Charlie", "David"}
anna_friends = {"Bob", "Eve", "Frank", "Grace"}
bob_friends = {"Charlie", "Hannah", "Ivy", "Anna"}

# Friends not following me back
not_following_back = my_friends - anna_friends
print(not_following_back)  # {'Anna', 'Charlie', 'David'}

# Mutual friends with Anna
mutual = my_friends & anna_friends
print(mutual)  # {'Bob'}

# Friend suggestions (friends of friends not already my friends)
suggestions = (anna_friends | bob_friends) - my_friends
print(suggestions)  # {'Eve', 'Frank', 'Grace', 'Hannah', 'Ivy'}
```

Tag Matching

```
post_tags = {"python", "tutorial", "coding", "beginner"}
user_interests = {"python", "data science", "machine learning", "coding"}

# Matching tags
matched = post_tags & user_interests
print(f"Matched tags: {matched}")  # {'python', 'coding'}

# All tags from both
all_tags = post_tags | user_interests
print(f"All tags: {all_tags}")
```

Filtering Data

```
# Products that are in stock AND on sale
in_stock = {"laptop", "mouse", "keyboard", "monitor"}
on_sale = {"mouse", "keyboard", "headphones"}
discounted = in_stock & on_sale
print(f"Discounted items in stock: {discounted}")  # {'mouse', 'keyboard'}
```

User Permissions

```
admin_permissions = {"read", "write", "delete", "manage_users"}
editor_permissions = {"read", "write", "edit"}
viewer_permissions = {"read"}

# A user with multiple roles
user_roles = {"editor", "viewer"}
user_permissions = set()

if "admin" in user_roles:
    user_permissions |= admin_permissions
if "editor" in user_roles:
    user_permissions |= editor_permissions
if "viewer" in user_roles:
    user_permissions |= viewer_permissions

print(f"User permissions: {user_permissions}")
# {'read', 'write', 'edit'}
```

Try It Yourself

Challenge 1: Find the union and intersection of {1, 2, 3, 4} and {3, 4, 5, 6}.

Challenge 2: Use the | operator and the union() method. Are they the same?

Challenge 3: Find the common elements between three sets.

Challenge 4: Which students are enrolled in math OR science but NOT art?

Chapter Summary

Operation	Operator	Method	Result
Union	a b	a.union(b)	All elements from both
Intersection	a & b	a.intersection(b)	Elements in both

CHAPTER 10.4: Difference and Symmetric Difference – Set Differences

What You'll Learn

- Difference (`-`) – elements in first set but not second
- Symmetric difference (`^`) – elements in either set, but not both
- Methods (`difference()` , `symmetric_difference()`)
- Practical applications

Difference (`-`)

Difference returns elements that are in the **first set but NOT in the second**.

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Elements in A but not B
difference = set_a - set_b
print(difference)  # {1, 2}

# Elements in B but not A
difference = set_b - set_a
print(difference)  # {5, 6}

# Using method
difference = set_a.difference(set_b)
print(difference)  # {1, 2}
```

Visual Example

```
# Students in different courses
math = {"Ryu", "Anna", "Max", "Lily"}
science = {"Anna", "Max", "John", "Emma"}

# Students in math ONLY (not in science)
only_math = math - science
print(only_math)  # {'Ryu', 'Lily'}

# Students in science ONLY (not in math)
only_science = science - math
print(only_science)  # {'John', 'Emma'}
```

Symmetric Difference ()

Symmetric difference returns elements that are in **either set, but not both**.

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Using operator
sym_diff = set_a ^ set_b
print(sym_diff)  # {1, 2, 5, 6}

# Using method
sym_diff = set_a.symmetric_difference(set_b)
print(sym_diff)  # {1, 2, 5, 6}
```

Visual Comparison

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Union: all elements
print(set_a | set_b)  # {1, 2, 3, 4, 5, 6}

# Intersection: common elements
print(set_a & set_b)  # {3, 4}

# Difference: A only
print(set_a - set_b)  # {1, 2}

# Symmetric difference: one or the other (not both)
print(set_a ^ set_b)  # {1, 2, 5, 6}
```

Venn Diagram Summary

Set A

1

2

3

4

Set B

5

6

A | B

= {1,2,3,4,5,6}

(everything)

A & B

= {3,4}

(middle)

A - B

= {1,2}

(left only)

B - A

= {5,6}

(right only)

A ^ B

= {1,2,5,6}

(everything but middle)

Real-World Examples

Exclusive Offers

```
# Customers who bought product A or B, but not both
bought_a = {"customer1", "customer2", "customer3", "customer4"}
bought_b = {"customer3", "customer4", "customer5", "customer6"}

exclusive_buyers = bought_a ^ bought_b
print(f"Bought exactly one product: {exclusive_buyers}")
# {'customer1', 'customer2', 'customer5', 'customer6'}
```

Skills Analysis

```
# Employees with certain skills
python_skills = {"Ryu", "Anna", "Max"}
javascript_skills = {"Anna", "Lily", "John"}

# Employees who know ONLY Python
only_python = python_skills - javascript_skills
print(f"Python only: {only_python}") # {'Ryu', 'Max'}

# Employees who know ONLY JavaScript
only_js = javascript_skills - python_skills
print(f"JavaScript only: {only_js}") # {'Lily', 'John'}

# Employees who know exactly one (not both)
exactly_one = python_skills ^ javascript_skills
print(f"Exactly one language: {exactly_one}") # {'Ryu', 'Max', 'Lily', 'John'}

# Employees who know both
both = python_skills & javascript_skills
print(f"Both languages: {both}") # {'Anna'}
```

New vs Returning Visitors

```
yesterday_visitors = {"user1", "user2", "user3", "user4"}
today_visitors = {"user3", "user4", "user5", "user6"}

# New visitors (today only)
new_visitors = today_visitors - yesterday_visitors
print(f"New: {new_visitors}") # {'user5', 'user6'}

# Returning visitors (both days)
returning = today_visitors & yesterday_visitors
print(f"Returning: {returning}") # {'user3', 'user4'}

# Lost visitors (yesterday only)
lost = yesterday_visitors - today_visitors
print(f"Lost: {lost}") # {'user1', 'user2'}

# Unique visitors across both days
unique = yesterday_visitors | today_visitors
print(f"Unique total: {unique}")
```

Missing Permissions

```
required_permissions = {"read", "write", "delete", "admin"}
user_permissions = {"read", "write"}

# Missing permissions
missing = required_permissions - user_permissions
print(f"Missing: {missing}") # {'delete', 'admin'}

# Extra permissions (not needed)
extra = user_permissions - required_permissions
print(f"Extra: {extra}") # set()
```

Try It Yourself

Challenge 1: Find the difference between `{1, 2, 3, 4}` and `{3, 4, 5, 6}`.

Challenge 2: Find the symmetric difference between the same sets.

Challenge 3: Which customers bought product A but not product B?

Challenge 4: Find employees who know Python OR JavaScript, but not both.

Chapter Summary

Operation	Operator	Method	Result
Difference	<code>a - b</code>	<code>a.difference(b)</code>	Elements in A not in B
Symmetric difference	<code>a ^ b</code>	<code>a.symmetric_difference(b)</code>	Elements in either, not both

CHAPTER 10.5: Frozen Sets – Immutable Sets

What You'll Learn

- What frozen sets are (immutable sets)
- Creating frozen sets with `frozenset()`
- When to use frozen sets
- Frozen sets as dictionary keys

What Is a Frozen Set?

A **frozen set** is an immutable version of a set. Once created, you cannot add, remove, or modify elements.

```
# Regular set (mutable)
normal_set = {1, 2, 3}
normal_set.add(4)  # Works

# Frozen set (immutable)
frozen = frozenset([1, 2, 3])
# frozen.add(4)  # AttributeError! 'frozenset' object has no attribute 'add'
```


Creating Frozen Sets

```
# From list
frozen1 = frozenset([1, 2, 3, 3, 4])
print(frozen1)  # frozenset({1, 2, 3, 4})

# From set
normal = {1, 2, 3}
frozen2 = frozenset(normal)
print(frozen2)  # frozenset({1, 2, 3})

# From string
frozen3 = frozenset("hello")
print(frozen3)  # frozenset({'h', 'e', 'l', 'o'})

# Empty frozen set
empty = frozenset()
```

Operations on Frozen Sets

Frozen sets support **read-only** operations (same as regular sets):

```
a = frozenset([1, 2, 3, 4])
b = frozenset([3, 4, 5, 6])

# Union
print(a | b)  # frozenset({1, 2, 3, 4, 5, 6})

# Intersection
print(a & b)  # frozenset({3, 4})

# Difference
print(a - b)  # frozenset({1, 2})

# Symmetric difference
print(a ^ b)  # frozenset({1, 2, 5, 6})

# Membership
print(1 in a)  # True
print(5 in a)  # False

# Length
print(len(a))  # 4
```

Frozen Sets as Dictionary Keys

Regular sets cannot be dictionary keys because they are mutable. Frozen sets can!

```
# ❌ Regular set as key (doesn't work)
# my_dict = {[1, 2]: "value"} # TypeError: unhashable type: 'set'

# ✅ Frozen set as key (works)
my_dict = {frozenset([1, 2]): "value"}
print(my_dict[frozenset([1, 2])]) # "value"

# Real example: caching function results by input parameters
def cached_operation(a, b, c):
    key = frozenset([a, b, c])
    if key in cache:
        print("From cache")
        return cache[key]
    result = a + b + c # Expensive operation
    cache[key] = result
    return result

cache = {}
print(cached_operation(1, 2, 3)) # Computes: 6
print(cached_operation(1, 2, 3)) # From cache: 6
```

Frozen Sets vs Regular Sets

Operation	Regular Set	Frozen Set
Add element	✅ <code>add()</code>	❌ Not allowed
Remove element	✅ <code>remove()</code>	❌ Not allowed
Discard element	✅ <code>discard()</code>	❌ Not allowed
Pop element	✅ <code>pop()</code>	❌ Not allowed
Clear	✅ <code>clear()</code>	❌ Not allowed
Union/intersection/etc	✅	✅
Membership test	✅	✅
Length	✅	✅
Can be dictionary key	❌	✅

Real-World Examples

Caching Query Results

```
from functools import lru_cache
import time

# Using frozenset for immutable cache keys
query_cache = {}

def expensive_query(filters):
    # Convert dict to frozenset of tuples for immutability
    key = frozenset(filters.items())

    if key in query_cache:
        print("Cache hit!")
        return query_cache[key]

    print("Executing expensive query...")
    time.sleep(2) # Simulate expensive operation
    result = {"data": "query results", "filters": filters}
    query_cache[key] = result
    return result

query1 = expensive_query({"category": "books", "price": "under 20"})
query2 = expensive_query({"category": "books", "price": "under 20"}) # Cache hit!
```

Set of Sets

```
# Regular sets cannot contain other sets (they're unhashable)
# nested = {{1, 2}, {3, 4}} # TypeError!

# Frozen sets can be in sets
nested = {frozenset([1, 2]), frozenset([3, 4])}
print(nested) # {frozenset({1, 2}), frozenset({3, 4})}

# Finding unique combinations
def unique_combinations(items, k):
    from itertools import combinations
    return {frozenset(combo) for combo in combinations(items, k)}

items = [1, 2, 3, 4]
combos = unique_combinations(items, 2)
print(combos) # {frozenset({1, 2}), frozenset({1, 3}), ...}
```

Configuration as Dictionary Keys

```
# Store settings for different permission sets
settings_by_permissions = {
    frozenset(["read"]): {"theme": "light", "notifications": False},
    frozenset(["read", "write"]): {"theme": "dark", "notifications": True},
    frozenset(["read", "write", "admin"]): {"theme": "dark", "notifications": True,
"debug": True},
}

def get_settings(user_permissions):
    key = frozenset(user_permissions)
    return settings_by_permissions.get(key, {"theme": "light", "notifications": False})
```

Try It Yourself

Challenge 1: Create a frozen set from a list of numbers.

Challenge 2: Try to add an element to a frozen set. What happens?

Challenge 3: Use a frozen set as a dictionary key.

Challenge 4: Create a set of frozen sets (nested sets).

Chapter Summary

Concept	Syntax	Characteristics
Create frozen set	<code>frozenset(iterable)</code>	Immutable
Empty frozen set	<code>frozenset()</code>	No elements
Hashable	☒ Yes	Can be dictionary key
Set operations	<code> </code> , <code>&</code> , <code>-</code> , <code>^</code>	All supported
Modification	☒ Not allowed	No add/remove/update

PART 10 SUMMARY

You now know all about sets:

Chapter	Topic
10.1	Creating sets
10.2	Adding and removing elements
10.3	Union and intersection
10.4	Difference and symmetric difference
10.5	Frozen sets (immutable sets)

What's Next?

Next up: **Part 11: Dictionaries** – Key-value pairs!

PART 12: MODULES AND PACKAGES

CHAPTER 12.1: Importing Modules – Using Built-in Code

What You'll Learn

- What modules are (reusable code files)

- How to import modules
- Different import styles
- Aliasing modules

What Is a Module?

A **module** is a Python file containing functions, classes, and variables. You can import it to use its code.

```
# Import the math module
import math

# Use functions from math
print(math.sqrt(16))      # 4.0
print(math.pi)            # 3.14159
print(math.floor(3.7))    # 3
```

Import Styles

```
# Style 1: Import whole module (needs prefix)
import math
print(math.sqrt(25))      # 5.0

# Style 2: Import specific items (no prefix needed)
from math import sqrt, pi
print(sqrt(36))           # 6.0
print(pi)                 # 3.14159

# Style 3: Import everything (not recommended)
from math import *
print(sin(0))             # 0.0

# Style 4: Import with alias
import math as m
print(m.sqrt(49))         # 7.0
```

Common Built-in Modules

Module	Purpose
<code>math</code>	Mathematical functions
<code>random</code>	Random number generation
<code>datetime</code>	Dates and times
<code>os</code>	Operating system interface
<code>sys</code>	System-specific parameters
<code>json</code>	JSON data handling
<code>re</code>	Regular expressions

Using `random`

```
import random

# Random integer between 1 and 10
num = random.randint(1, 10)
print(num)

# Random float between 0 and 1
r = random.random()
print(r)

# Random choice from list
colors = ["red", "green", "blue"]
choice = random.choice(colors)
print(choice)

# Shuffle a list
cards = ["A", "2", "3", "4"]
random.shuffle(cards)
print(cards)
```


Using `datetime`

```
from datetime import datetime, date, time

# Current date and time
now = datetime.now()
print(now)

# Current date
today = date.today()
print(today)

# Create specific date
birthday = date(2024, 1, 15)
print(birthday)

# Format dates
formatted = now.strftime("%Y-%m-%d %H:%M:%S")
print(formatted)
```

Chapter Summary

Import Style	Syntax	When to Use
Full module	<code>import module</code>	Many functions from module
Specific items	<code>from module import item</code>	Few specific functions
Alias	<code>import module as alias</code>	Long module names

CHAPTER 12.2: Creating Your Own Modules

What You'll Learn

- How to create your own modules
- Organizing code across files
- The `if __name__ == "__main__":` guard

Creating a Module

Create a file called `my_math.py`:

```
# my_math.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    return None

PI = 3.14159
```

Using Your Module

```
# main.py
import my_math

print(my_math.add(5, 3))      # 8
print(my_math.multiply(4, 6)) # 24
print(my_math.PI)            # 3.14159
```

The `if __name__ == "__main__"` Guard

Prevents code from running when imported:

```
# my_math.py

def add(a, b):
    return a + b

# This only runs when script is executed directly
if __name__ == "__main__":
    print("Testing my_math module")
    print(f"5 + 3 = {add(5, 3)}")
```

Now:

- `python my_math.py` → runs tests
- `import my_math` → tests don't run

Chapter Summary

Concept	Syntax
Create module	Save <code>.py</code> file
Import module	<code>import filename</code>
Run guard	<code>if __name__ == "__main__":</code>

CHAPTER 12.3: Packages – Organizing Modules

What You'll Learn

- What packages are (folders of modules)
- Creating packages with `__init__.py`
- Importing from packages

What Is a Package?

A **package** is a folder containing multiple modules. It must have an `__init__.py` file (can be empty).

```
my_package/  
    __init__.py  
    math_ops.py  
    string_ops.py  
    file_ops.py
```

Creating a Package

```
my_package/math_ops.py:
```

```
def add(a, b): return a + b  
def multiply(a, b): return a * b
```

```
my_package/string_ops.py:
```

```
def reverse(s): return s[::-1]  
def to_upper(s): return s.upper()
```

Using a Package

```
# Import specific modules  
from my_package import math_ops, string_ops  
  
print(math_ops.add(5, 3))          # 8  
print(string_ops.reverse("hello")) # olleh  
  
# Or import specific functions  
from my_package.math_ops import multiply  
print(multiply(4, 5))              # 20
```

Chapter Summary

Concept	Description
Package	Folder with modules
<code>__init__.py</code>	Marks folder as package
Import	<code>from package import module</code>

PART 13: FILE HANDLING

CHAPTER 13.1: Reading Files – Working with Text Files

What You'll Learn

- Opening files with `open()`
- Reading entire files
- Reading line by line
- Closing files (with `with`)

Opening Files

```
# Open file for reading ('r' is default)
file = open("data.txt", "r")
content = file.read()
file.close()
print(content)
```

Using `with` (Recommended)

Auto-closes the file even if errors occur:

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File automatically closed here
```

Reading Methods

```
with open("data.txt", "r") as file:
    # Read entire file as string
    all_text = file.read()

    # Read all lines into list
    lines = file.readlines()

    # Read one line at a time
    line = file.readline()
```

Reading Line by Line (Memory Efficient)

```
with open("large_file.txt", "r") as file:
    for line in file:
        print(line.strip()) # strip removes newline
```

Chapter Summary

Mode	Description
"r"	Read (default)
"w"	Write (overwrites)
"a"	Append
"r+"	Read and write

CHAPTER 13.2: Writing Files – Saving Data

What You'll Learn

- Writing to files with "w"
- Appending to files with "a"
- Writing multiple lines

Writing to a File

```
# Write mode (overwrites existing content)
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("Second line\n")
```

Appending to a File

```
# Append mode (adds to end)
with open("output.txt", "a") as file:
    file.write("This line is appended\n")
```

Writing Multiple Lines

```
lines = ["Line 1", "Line 2", "Line 3"]

with open("output.txt", "w") as file:
    for line in lines:
        file.write(line + "\n")

# Or use writelines()
file.writelines(f"{line}\n" for line in lines)
```

Chapter Summary

Mode	Behavior
"w"	Create new/overwrite
"a"	Append to end

CHAPTER 13.3: Working with CSV and JSON

What You'll Learn

- Reading and writing CSV files
- Reading and writing JSON files
- Converting between Python objects and JSON

CSV Files (Comma-Separated Values)

```
import csv

# Writing CSV
data = [
    ["Name", "Age", "City"],
    ["Ryu", 11, "Tokyo"],
    ["Anna", 12, "Osaka"]
]

with open("people.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)

# Reading CSV
with open("people.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```


JSON Files (JavaScript Object Notation)

```
import json

# Python object to JSON string
person = {"name": "Ryuu", "age": 11, "city": "Tokyo"}
json_string = json.dumps(person)
print(json_string)  # '{"name": "Ryuu", "age": 11, "city": "Tokyo"}'

# Write JSON to file
with open("person.json", "w") as file:
    json.dump(person, file)

# Read JSON from file
with open("person.json", "r") as file:
    data = json.load(file)
    print(data["name"])  # Ryuu
```

Chapter Summary

JSON Function	Purpose
<code>json.dumps(obj)</code>	Python → JSON string
<code>json.loads(str)</code>	JSON string → Python
<code>json.dump(obj, file)</code>	Write JSON to file
<code>json.load(file)</code>	Read JSON from file

PART 14: ERROR HANDLING

CHAPTER 14.1: try and except – Catching Errors

What You'll Learn

- What exceptions are (runtime errors)

- Using `try` and `except`
- Catching specific errors
- Preventing crashes

Basic try-except

```
try:
    num = int(input("Enter a number: "))
    print(f"You entered {num}")
except ValueError:
    print("That's not a valid number!")
```

Catching Specific Exceptions

```
try:
    numbers = [1, 2, 3]
    index = int(input("Enter index: "))
    print(numbers[index])
except ValueError:
    print("Please enter a number")
except IndexError:
    print("Index out of range")
```

Multiple Exceptions in One Block

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except (ValueError, ZeroDivisionError):
    print("Invalid input or division by zero")
```

Chapter Summary

Concept	Syntax
Basic	<code>try: except:</code>
Specific	<code>except ErrorType:</code>
Multiple	<code>except (Error1, Error2):</code>

CHAPTER 14.2: else, finally, and raise

What You'll Learn

- `else` – runs if no exception
- `finally` – always runs
- `raise` – manually trigger errors

else Clause

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("Invalid input")
else:
    # Runs only if no exception
    print(f"Square: {num ** 2}")
```

finally Clause

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    # Always runs (for cleanup)
    print("Closing file")
    file.close()
```

raise – Manually Raising Exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    if age > 150:
        raise ValueError("Age too high")
    return age

try:
    age = set_age(-5)
except ValueError as e:
    print(f"Error: {e}")
```

Chapter Summary

Clause	When It Runs
else	If no exception occurred
finally	Always (for cleanup)
raise	Manually trigger exception

PARTS 12-14 SUMMARY

Part	Topic
12	Modules and Packages
13	File Handling
14	Error Handling

PART 15: OBJECT-ORIENTED PROGRAMMING

CHAPTER 15.1: Classes and Objects – Blueprints and Instances

What You'll Learn

- What classes are (blueprints for objects)
- What objects are (instances of classes)
- Creating classes with `class`
- The `__init__` constructor

What Is a Class?

A **class** is a blueprint for creating objects. It defines what data and methods an object will have.

```
# Define a class
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hi, I'm {self.name} and I'm {self.age} years old")

# Create objects (instances)
person1 = Person("Ryu", 11)
person2 = Person("Anna", 12)

person1.introduce() # Hi, I'm Ryu and I'm 11 years old
person2.introduce() # Hi, I'm Anna and I'm 12 years old
```

The `__init__` Constructor

`__init__` runs automatically when an object is created. `self` refers to the current instance.

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand # instance variable
        self.model = model
        self.year = year

    def start(self):
        print(f"{self.brand} {self.model} is starting...")

my_car = Car("Tesla", "Model 3", 2023)
print(my_car.brand) # Tesla
my_car.start() # Tesla Model 3 is starting...
```

Instance Variables vs Class Variables

```
class Student:
    # Class variable (shared by all instances)
    school = "Springfield Elementary"

    def __init__(self, name):
        # Instance variable (unique to each instance)
        self.name = name

s1 = Student("Ryu")
s2 = Student("Anna")

print(s1.school)  # Springfield Elementary
print(s2.school)  # Springfield Elementary
print(s1.name)    # Ryu
print(s2.name)    # Anna
```

Chapter Summary

Concept	Syntax	Example
Class definition	<code>class Name:</code>	<code>class Person:</code>
Constructor	<code>def __init__(self):</code>	Initialize attributes
Instance variable	<code>self.var = value</code>	<code>self.name = name</code>
Method	<code>def method(self):</code>	<code>def introduce(self):</code>
Create instance	<code>obj = Class()</code>	<code>p = Person("Ryu", 11)</code>

CHAPTER 15.2: Inheritance – Child Classes

What You'll Learn

- What inheritance is (child classes inherit from parent)
- Creating subclasses
- Using `super()` to call parent methods
- Method overriding

Basic Inheritance

```
# Parent class (superclass)
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        print(f"{self.name} makes a sound")

# Child class (subclass)
class Dog(Animal):
    def make_sound(self): # Override parent method
        print(f"{self.name} says: Woof!")

class Cat(Animal):
    def make_sound(self):
        print(f"{self.name} says: Meow!")

dog = Dog("Buddy")
cat = Cat("Whiskers")
dog.make_sound() # Buddy says: Woof!
cat.make_sound() # Whiskers says: Meow!
```


Using `super()` – Call Parent Methods

```
class Vehicle:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

    def start(self):
        print(f"{self.brand} vehicle starting...")

class Car(Vehicle):
    def __init__(self, brand, year, doors):
        super().__init__(brand, year) # Call parent constructor
        self.doors = doors

    def start(self):
        super().start() # Call parent method
        print(f"Car has {self.doors} doors")

car = Car("Toyota", 2023, 4)
car.start()
# Toyota vehicle starting...
# Car has 4 doors
```

Multiple Inheritance

```
class Flyer:
    def fly(self):
        print("Flying...")

class Swimmer:
    def swim(self):
        print("Swimming...")

class Duck(Flyer, Swimmer):
    pass # Inherits from both

duck = Duck()
duck.fly() # Flying...
duck.swim() # Swimming...
```

Chapter Summary

Concept	Syntax	Example
Inheritance	<code>class Child(Parent):</code>	<code>class Dog(Animal):</code>
Call parent	<code>super().method()</code>	<code>super().__init__()</code>
Override	Redefine method in child	<code>def make_sound(self):</code>

CHAPTER 15.3: Encapsulation – Private Attributes

What You'll Learn

- What encapsulation is (hiding internal data)
- Private attributes with `__` (double underscore)
- Getter and setter methods
- Name mangling

Private Attributes

Use `__` (double underscore) to make attributes private:

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited ${amount}")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew ${amount}")
        else:
            print("Insufficient funds")

    def get_balance(self):
        return self.__balance

account = BankAccount("Ryu", 1000)
account.deposit(500) # Deposited $500
account.withdraw(200) # Withdrew $200
print(account.get_balance()) # 1300

# Cannot access private attribute directly
# print(account.__balance) # AttributeError!
```

Getters and Setters (Properties)

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius  # Protected (convention)

    @property
    def celsius(self):
        """Getter"""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Setter with validation"""
        if value < -273.15:
            raise ValueError("Temperature below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):
        """Computed property"""
        return (self._celsius * 9/5) + 32

temp = Temperature(25)
print(temp.celsius)      # 25 (uses getter)
temp.celsius = 30        # (uses setter)
print(temp.fahrenheit)  # 86.0
```

Name Mangling

Python "hides" private attributes by changing their name:

```
class MyClass:
    def __init__(self):
        self.__secret = 42

obj = MyClass()
# print(obj.__secret)      # AttributeError
print(obj._MyClass__secret)  # 42 (name mangled, but accessible)
```

Chapter Summary

Convention	Meaning
<code>name</code>	Public (normal)
<code>_name</code>	Protected (internal use, convention)
<code>__name</code>	Private (name mangling)

PART 16: ADVANCED OOP

CHAPTER 16.1: Class Methods and Static Methods

What You'll Learn

- Class methods (`@classmethod`) – work with class, not instance
- Static methods (`@staticmethod`) – utility functions
- Difference between instance, class, and static methods

Instance Methods (Already Know)

```
class MyClass:
    def instance_method(self):
        print(f"Instance method: {self}")
```

Class Methods (`@classmethod`)

Class methods receive the class (`cls`) instead of the instance. Used for factory methods.

```

class Person:
    count = 0 # Class variable

    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def get_count(cls):
        """Class method - returns total persons created"""
        return cls.count

    @classmethod
    def from_string(cls, data):
        """Factory method - creates Person from "name,age" string"""
        name, age = data.split(",")
        return cls(name, int(age))

p1 = Person("Ryu")
p2 = Person("Anna")
print(Person.get_count()) # 2

p3 = Person.from_string("Max,10")
print(p3.name) # Max

```

Static Methods (`@staticmethod`)

Static methods don't receive `self` or `cls`. They're like regular functions inside a class.

```

class MathUtils:
    @staticmethod
    def is_even(number):
        return number % 2 == 0

    @staticmethod
    def add(a, b):
        return a + b

print(MathUtils.is_even(5)) # False
print(MathUtils.add(5, 3)) # 8

```

Comparison

Method Type	First Parameter	Can Access	Use For
Instance	<code>self</code>	Instance attributes	Most methods
Class	<code>cls</code>	Class attributes	Factory methods
Static	None	Nothing	Utility functions

Chapter Summary

Decorator	Purpose
<code>@classmethod</code>	Works with class, receives <code>cls</code>
<code>@staticmethod</code>	Utility, no <code>self</code> or <code>cls</code>

CHAPTER 16.2: Magic Methods (Dunder Methods)

What You'll Learn

- What magic methods are (methods with `__` like `__str__`)
- Common magic methods
- Making objects work with operators

Common Magic Methods

Method	Called When
<code>__init__(self)</code>	Object created
<code>__str__(self)</code>	<code>str(obj)</code> or <code>print(obj)</code>
<code>__repr__(self)</code>	<code>repr(obj)</code>
<code>__len__(self)</code>	<code>len(obj)</code>
<code>__add__(self, other)</code>	<code>obj1 + obj2</code>
<code>__eq__(self, other)</code>	<code>obj1 == obj2</code>

`__str__`**and**`__repr__`

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        """User-friendly string (for print)"""
        return f"Person: {self.name} ({self.age})"

    def __repr__(self):
        """Developer-friendly string (for debugging)"""
        return f"Person('{self.name}', {self.age})"

p = Person("Ryu", 11)
print(p)          # Person: Ryu (11)  (uses __str__)
print(repr(p))    # Person('Ryu', 11) (uses __repr__)
```

`__len__`**and**`__getitem__`

Make your class behave like a list:


```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

    def __len__(self):
        return len(self.items)

    def __getitem__(self, index):
        return self.items[index]

cart = ShoppingCart()
cart.add("Book")
cart.add("Pen")
cart.add("Notebook")

print(len(cart))      # 3 (uses __len__)
print(cart[1])        # Pen (uses __getitem__)

for item in cart:
    print(item)        # Works with iteration!
```

Operator Overloading

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(1, 4)
v3 = v1 + v2          # Uses __add__
print(v3)             # Vector(3, 7)
print(v1 == Vector(2, 3))  # True
```

Chapter Summary

Magic Method	Operator/Function
<code>__str__(self)</code>	<code>print(obj)</code>
<code>__len__(self)</code>	<code>len(obj)</code>
<code>__getitem__(self, i)</code>	<code>obj[i]</code>
<code>__add__(self, other)</code>	<code>obj1 + obj2</code>
<code>__eq__(self, other)</code>	<code>obj1 == obj2</code>

CHAPTER 16.3: Property Decorators – Getters and Setters

What You'll Learn

- Using `@property` for getters
- Using `@attribute.setter` for setters
- Computed properties
- Validation in setters

The Problem Without Properties

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age # No validation!

p = Person("Ryu", -5) # Invalid age accepted!
```

Solution: Getters and Setters

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self._age = age  # "Protected" attribute

    def get_age(self):
        return self._age

    def set_age(self, value):
        if value < 0:
            raise ValueError("Age cannot be negative")
        if value > 150:
            raise ValueError("Age too high")
        self._age = value

p = Person("Ryu", 11)
p.set_age(12)          # Works
# p.set_age(-5)        # ValueError!
```

Better: `@property` Decorator

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self._age = age

    @property
    def age(self):
        """Getter - accessed like an attribute"""
        return self._age

    @age.setter
    def age(self, value):
        """Setter - called when assigning"""
        if value < 0:
            raise ValueError("Age cannot be negative")
        if value > 150:
            raise ValueError("Age too high")
        self._age = value

p = Person("Ryu", 11)
print(p.age)      # 11 (looks like attribute, not method!)
p.age = 12        # Uses setter
# p.age = -5      # ValueError!
```

Computed Properties (Read-Only)

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def area(self):
        """Computed property - no setter"""
        return self.width * self.height

    @property
    def perimeter(self):
        return 2 * (self.width + self.height)

rect = Rectangle(5, 3)
print(rect.area)          # 15
print(rect.perimeter)     # 16

# Cannot set area - no setter
# rect.area = 20          # AttributeError
```

Real-World Example: Validation

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = value

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self.balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdraw amount must be positive")
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount

account = BankAccount(1000)
account.deposit(500)
account.withdraw(200)
print(account.balance)  # 1300
# account.balance = -100  # ValueError!
```

Chapter Summary

Decorator	Purpose
@property	Getter (makes method look like attribute)
@name.setter	Setter (validation)
@name.deleter	Deleter (rarely used)

PARTS 15-16 SUMMARY

Part	Topic
15	OOP Basics (Classes, Inheritance, Encapsulation)
16	Advanced OOP (Class/Static Methods, Magic Methods, Properties)

What's Next?

Next up: **Parts 17-22** – Advanced Python, Functional Programming, Data Structures, Databases, Backend, and Full-Stack!

PART 17: ADVANCED PYTHON

CHAPTER 17.1: Iterators – Custom Looping

What You'll Learn

- What iterators are (objects you can loop over)
- The `__iter__()` and `__next__()` methods
- The `StopIteration` exception
- Creating custom iterators

What Is an Iterator?

An **iterator** is an object that produces a sequence of values, one at a time. You can use it in `for` loops.


```
# Lists, tuples, strings, dicts are all iterable
numbers = [1, 2, 3]
for num in numbers:
    print(num)

# Get iterator manually
iterator = iter(numbers)
print(next(iterator)) # 1
print(next(iterator)) # 2
print(next(iterator)) # 3
# print(next(iterator)) # StopIteration (no more items)
```

How Iterators Work

Every iterator must implement:

- `__iter__()` – returns the iterator object itself
- `__next__()` – returns the next value, raises `StopIteration` when done

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self # Iterator returns itself

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value

# Use in for loop
for num in Countdown(5):
    print(num)
# Output: 5, 4, 3, 2, 1

# Use manually
counter = Countdown(3)
print(next(counter)) # 3
print(next(counter)) # 2
print(next(counter)) # 1
# print(next(counter)) # StopIteration
```

Real-World Example: Fibonacci Iterator

```
class Fibonacci:
    def __init__(self, limit):
        self.limit = limit
        self.a = 0
        self.b = 1
        self.count = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.limit:
            raise StopIteration

        self.count += 1
        if self.count == 1:
            return self.a
        elif self.count == 2:
            return self.b
        else:
            self.a, self.b = self.b, self.a + self.b
            return self.b

for num in Fibonacci(10):
    print(num, end=" ")

# 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Chapter Summary

Method	Purpose
<code>__iter__(self)</code>	Returns iterator object
<code>__next__(self)</code>	Returns next value or raises StopIteration

CHAPTER 17.2: Generators – Lazy Value Production

What You'll Learn

- What generators are (functions that yield values)
- The `yield` keyword
- Generator expressions
- Memory efficiency

What Is a Generator?

A **generator** is a function that uses `yield` instead of `return`. It produces values lazily (one at a time, only when needed).

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

# Generator creates an iterator
for num in countdown(5):
    print(num)
# 5, 4, 3, 2, 1

# Generator remembers state between yields
gen = countdown(3)
print(next(gen)) # 3
print(next(gen)) # 2
print(next(gen)) # 1
```

Generators vs Lists (Memory Efficiency)

```
import sys

# List (stores all in memory)
def squares_list(n):
    result = []
    for i in range(n):
        result.append(i ** 2)
    return result

# Generator (yields one at a time)
def squares_gen(n):
    for i in range(n):
        yield i ** 2

# Memory usage
list_squares = squares_list(1000000)
gen_squares = squares_gen(1000000)

print(sys.getsizeof(list_squares)) # ~8 million bytes
print(sys.getsizeof(gen_squares)) # ~112 bytes (tiny!)
```

Infinite Generators

```
def infinite_numbers():
    n = 0
    while True:
        yield n
        n += 1

# Take only what you need
numbers = infinite_numbers()
for _ in range(10):
    print(next(numbers), end=" ")

# 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Generator Expressions

Like list comprehensions but with parentheses:

```
# List comprehension (eager)
squares_list = [x**2 for x in range(10)] # Creates full list

# Generator expression (lazy)
squares_gen = (x**2 for x in range(10)) # Creates generator

# Same output, but generator uses less memory
for sq in squares_gen:
    print(sq, end=" ")

# 0, 1, 4, 9, 16, 25, 36, 49, 64, 81

# Generator expressions can be used in functions
sum_of_squares = sum(x**2 for x in range(1000)) # No extra list!
```

Real-World Example: Reading Large Files

```
def read_large_file(file_path):
    """Reads file line by line without loading entire file into memory"""
    with open(file_path, 'r') as file:
        for line in file:
            yield line.strip()

# Process one line at a time
for line in read_large_file("huge_file.txt"):
    process_line(line) # Memory efficient!
```

Chapter Summary

Concept	Syntax	Use
Generator function	<code>def f(): yield x</code>	Produces values lazily
Generator expression	<code>(expr for item in iter)</code>	Like list comprehension, lazy
Memory efficiency	Much smaller	Large datasets

CHAPTER 17.3: Decorators – Wrapping Functions

What You'll Learn

- What decorators are (functions that modify other functions)
- The `@decorator` syntax
- Creating custom decorators
- Decorators with arguments

What Is a Decorator?

A **decorator** is a function that takes another function and extends its behavior without modifying it.

```
def timer(func):  
    """Decorator that measures execution time"""  
    import time  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"{func.__name__} took {end - start:.4f} seconds")  
        return result  
    return wrapper  
  
@timer  
def slow_function():  
    import time  
    time.sleep(1)  
    return "Done"  
  
slow_function() # slow_function took 1.0002 seconds
```

Basic Decorator Syntax

```
# Without decorator syntax

def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

def say_hello():
    print("Hello!")

say_hello = logger(say_hello)  # Manual decoration

# With decorator syntax (same thing)
@logger
def say_hello():
    print("Hello!")

say_hello()
```


Common Decorator Examples

Timing Decorator

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"{func.__name__}: {end - start:.4f}s")
        return result
    return wrapper

@timer
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

factorial(1000) # factorial: 0.0005s (approx)
```

Debug Decorator

```
def debug(func):  
    def wrapper(*args, **kwargs):  
        args_repr = [repr(a) for a in args]  
        kwargs_repr = [f"{k}={v!r}" for k, v in kwargs.items()]  
        signature = ", ".join(args_repr + kwargs_repr)  
        print(f"Calling {func.__name__}({signature})")  
        result = func(*args, **kwargs)  
        print(f"{func.__name__} returned {result!r}")  
        return result  
    return wrapper  
  
@debug  
def add(a, b):  
    return a + b  
  
add(3, 5)  
# Calling add(3, 5)  
# add returned 8
```

Retry Decorator

```
def retry(max_attempts=3):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt + 1} failed: {e}")
                    raise Exception(f"Failed after {max_attempts} attempts")
            return wrapper
        return decorator

@retry(max_attempts=3)
def unstable_network_call():
    import random
    if random.random() < 0.7:
        raise ConnectionError("Network error")
    return "Success!"

# Will retry up to 3 times
result = unstable_network_call()
```

Cache (Memoization) Decorator

```
def cache(func):  
    stored = {}  
    def wrapper(*args):  
        if args in stored:  
            print(f"Cache hit for {args}")  
            return stored[args]  
        result = func(*args)  
        stored[args] = result  
        print(f"Cache miss for {args}")  
        return result  
    return wrapper  
  
@cache  
def fibonacci(n):  
    if n < 2:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)  
  
print(fibonacci(10))  # Cache misses for new values, hits for repeats
```

Preserving Function Metadata

Use `functools.wraps` to keep the original function's name and docstring:

```

from functools import wraps

def my_decorator(func):
    @wraps(func)  # Preserves func.__name__, func.__doc__, etc.
    def wrapper(*args, **kwargs):
        """Wrapper docstring"""
        return func(*args, **kwargs)
    return wrapper

@my_decorator
def say_hello():
    """Original docstring"""
    print("Hello!")

print(say_hello.__name__)  # say_hello (not wrapper)
print(say_hello.__doc__)  # Original docstring (not "Wrapper docstring")

```

Chapter Summary

Pattern	Syntax	Use
Basic decorator	<code>@deco def f():</code>	Wrap function
Decorator with args	<code>@deco(arg)</code>	Parameterized decoration
Preserve metadata	<code>@wraps(func)</code>	Keep original name/doc

CHAPTER 17.4: Context Managers – The `with` Statement

What You'll Learn

- What context managers are (set up and clean up)
- The `with` statement
- Creating context managers with classes (`__enter__`, `__exit__`)
- Creating context managers with `contextlib`

What Is a Context Manager?

A **context manager** ensures that setup and cleanup code always runs, even if errors occur.

```
# Without context manager (manual cleanup)
file = open("data.txt", "r")
try:
    content = file.read()
finally:
    file.close() # Must close manually

# With context manager (automatic cleanup)
with open("data.txt", "r") as file:
    content = file.read()
# File automatically closed here
```

Creating a Class-Based Context Manager

Implement `__enter__` and `__exit__`:

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.end = time.perf_counter()
        print(f"Elapsed: {self.end - self.start:.4f}s")
        return False # Don't suppress exceptions

with Timer():
    time.sleep(1)
# Elapsed: 1.0002s
```

Handling Exceptions in `__exit__`

```
class SuppressErrors:
    def __enter__(self):
        print("Entering context")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"Suppressed error: {exc_val}")
            return True # Suppress the exception
        return False

with SuppressErrors():
    print(1 / 0) # Division by zero is suppressed!
print("Continues after error")
```

Using `contextlib` (Simpler)

```
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.perf_counter()
    try:
        yield
    finally:
        end = time.perf_counter()
        print(f"Elapsed: {end - start:.4f}s")

with timer():
    time.sleep(1)
# Elapsed: 1.0002s
```

Real-World Examples

Database Connection

```
class DatabaseConnection:
    def __enter__(self):
        print("Connecting to database...")
        self.connection = "DB_CONNECTION"
        return self.connection

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Closing database connection...")
        if exc_type:
            print(f"Rolling back transaction due to: {exc_val}")
        else:
            print("Committing transaction")

with DatabaseConnection() as conn:
    print(f"Using {conn}")
    # Do database operations
# Connecting to database...
# Using DB_CONNECTION
# Closing database connection...
# Committing transaction
```


Redirecting Output

```
from contextlib import contextmanager
import sys

@contextmanager
def suppress_output():
    """Temporarily suppresses print output"""
    with open('/dev/null', 'w') as devnull:
        old_stdout = sys.stdout
        sys.stdout = devnull
        try:
            yield
        finally:
            sys.stdout = old_stdout

print("This prints normally")
with suppress_output():
    print("This is suppressed")
print("This prints again")
```

File Locking

```
import fcntl

class FileLock:
    def __init__(self, filename):
        self.filename = filename

    def __enter__(self):
        self.file = open(self.filename, 'w')
        fcntl.flock(self.file, fcntl.LOCK_EX)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        fcntl.flock(self.file, fcntl.LOCK_UN)
        self.file.close()

with FileLock("data.txt") as locked_file:
    locked_file.write("Safe write")
```

Chapter Summary

Method	Purpose
<code>__enter__(self)</code>	Setup, runs before <code>with</code> block
<code>__exit__(self, ...)</code>	Cleanup, runs after (even on error)
<code>@contextmanager</code>	Generator-based context manager

CHAPTER 17.5: Closures – Functions with Memory

What You'll Learn

- What closures are (functions that remember their environment)
- Creating closures with nested functions
- Practical uses of closures
- Closures vs classes

What Is a Closure?

A **closure** is a function that "remembers" variables from its enclosing scope, even after that scope has finished executing.

```
def make_counter():
    count = 0 # Enclosing variable

    def counter():
        nonlocal count # Need nonlocal to modify
        count += 1
        return count

    return counter

counter1 = make_counter()
counter2 = make_counter()

print(counter1()) # 1
print(counter1()) # 2
print(counter2()) # 1 (independent)
```

How Closures Work

```
def make_multiplier(factor):
    # factor is "captured" by the closure
    def multiplier(x):
        return x * factor
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5)) # 10
print(triple(5)) # 15
```

Closures vs Classes

Both can achieve similar results. Closures are often simpler for small tasks.

```
# Using closure
def make_adder(amount):
    def adder(x):
        return x + amount
    return adder

add5 = make_adder(5)
print(add5(10))  # 15

# Using class (more verbose)
class Adder:
    def __init__(self, amount):
        self.amount = amount
    def __call__(self, x):
        return x + self.amount

add5 = Adder(5)
print(add5(10))  # 15
```

Real-World Examples

Function Logger

```
def make_logger(prefix):
    def logger(message):
        print(f"[{prefix}] {message}")
    return logger

info_log = make_logger("INFO")
error_log = make_logger("ERROR")

info_log("Application started")    # [INFO] Application started
error_log("Connection failed")    # [ERROR] Connection failed
```

API Call Builder

```
def make_api_client(base_url):
    def call(endpoint, method="GET", data=None):
        url = f"{base_url}/{endpoint}"
        print(f"{method} {url}")
        if data:
            print(f>Data: {data}")
        return {"status": 200}
    return call

github = make_api_client("https://api.github.com")
users = make_api_client("https://api.example.com/users")

github("repos")          # GET https://api.github.com/repos
users("123", "POST", {"name": "Ryu"})
# POST https://api.example.com/users/123
# Data: {'name': 'Ryu'}
```

Button Click Handlers (GUI style)

```
def make_button_handler(button_id):
    def handler():
        print(f"Button {button_id} was clicked!")
    return handler

button1 = make_button_handler(1)
button2 = make_button_handler(2)

# Simulate clicks
button1() # Button 1 was clicked!
button2() # Button 2 was clicked!
```

Memoization (Caching) with Closure

```
def memoize(func):
    cache = {}
    def wrapper(*args):
        if args in cache:
            print(f"Cache hit for {args}")
            return cache[args]
        result = func(*args)
        cache[args] = result
        print(f"Cache miss for {args}")
        return result
    return wrapper

@memoize
def slow_square(n):
    import time
    time.sleep(1)
    return n * n

print(slow_square(5))  # Cache miss, takes 1s
print(slow_square(5))  # Cache hit, instant!
```

The **nonlocal** Keyword

Used to modify variables from the enclosing scope:

```
def outer():
    x = 10

    def inner():
        nonlocal x  # Without this, x would be local
        x += 5
        return x

    return inner

func = outer()
print(func())  # 15
print(func())  # 20
```

Chapter Summary

Concept	Syntax	Use
Closure	Nested function returning inner function	Captures enclosing variables
<code>nonlocal</code>	<code>nonlocal var</code>	Modify enclosing variable
Use cases	Callbacks, loggers, API clients	Small stateful functions

PART 18: FUNCTIONAL PROGRAMMING (Condensed)

CHAPTER 18.1: map() – Transform All Items

```
# map() applies a function to every item in an iterable
numbers = [1, 2, 3, 4, 5]

# Square each number
squared = list(map(lambda x: x ** 2, numbers))
print(squared)  # [1, 4, 9, 16, 25]

# Convert to string
strings = list(map(str, numbers))
print(strings)  # ['1', '2', '3', '4', '5']

# Multiple iterables
list1 = [1, 2, 3]
list2 = [10, 20, 30]
sums = list(map(lambda a, b: a + b, list1, list2))
print(sums)  # [11, 22, 33]
```

CHAPTER 18.2: filter() – Keep Matching Items

```
# filter() keeps items where function returns True
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Keep even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4, 6, 8, 10]

# Keep numbers > 5
greater = list(filter(lambda x: x > 5, numbers))
print(greater)  # [6, 7, 8, 9, 10]

# With strings
words = ["apple", "banana", "cat", "dog", "elephant"]
long_words = list(filter(lambda w: len(w) > 3, words))
print(long_words)  # ['apple', 'banana', 'elephant']
```

CHAPTER 18.3: reduce() – Combine All Items

```
from functools import reduce

# reduce() combines all items into a single value
numbers = [1, 2, 3, 4, 5]

# Sum
total = reduce(lambda a, b: a + b, numbers)
print(total)  # 15

# Product
product = reduce(lambda a, b: a * b, numbers)
print(product)  # 120

# Maximum
max_val = reduce(lambda a, b: a if a > b else b, numbers)
print(max_val)  # 5

# With initial value
total = reduce(lambda a, b: a + b, numbers, 100)
print(total)  # 115
```

CHAPTER 18.4: zip() – Combine Iterables

```
# zip() pairs items from multiple iterables
names = ["Ryu", "Anna", "Max"]
ages = [11, 12, 10]
cities = ["Tokyo", "Osaka", "Kyoto"]

# Pair two lists
pairs = list(zip(names, ages))
print(pairs)  # [('Ryu', 11), ('Anna', 12), ('Max', 10)]

# Pair three lists
combined = list(zip(names, ages, cities))
print(combined)
# [('Ryu', 11, 'Tokyo'), ('Anna', 12, 'Osaka'), ('Max', 10, 'Kyoto')]

# Create dictionary from two lists
person_dict = dict(zip(names, ages))
print(person_dict)  # {'Ryu': 11, 'Anna': 12, 'Max': 10}

# Unzip
names2, ages2 = zip(*combined)
print(names2)  # ('Ryu', 'Anna', 'Max')
print(ages2)   # (11, 12, 10)

# Different lengths (stops at shortest)
short = [1, 2]
long = [10, 20, 30]
pairs = list(zip(short, long))
print(pairs)  # [(1, 10), (2, 20)]
```

CHAPTER 18.5: any() and all() – Check Conditions

```
# any() - True if ANY item is True
numbers = [0, 0, 1, 0]
print(any(numbers)) # True

# all() - True if ALL items are True
print(all(numbers)) # False

# Real examples
def has_even(numbers):
    return any(n % 2 == 0 for n in numbers)

def all_positive(numbers):
    return all(n > 0 for n in numbers)

print(has_even([1, 3, 5, 7])) # False
print(has_even([1, 2, 3, 5])) # True
print(all_positive([1, 2, 3])) # True
print(all_positive([1, -2, 3])) # False

# With strings
words = ["hello", "world", "python"]
has_o = any("o" in w for w in words)
all_long = all(len(w) > 3 for w in words)

print(has_o) # True
print(all_long) # True
```

PART 18 SUMMARY

Function	Purpose
<code>map(func, iter)</code>	Apply function to every item
<code>filter(func, iter)</code>	Keep items where func is True
<code>reduce(func, iter)</code>	Combine into single value
<code>zip(*iters)</code>	Pair items from multiple iterables
<code>any(iter)</code>	True if any item is True
<code>all(iter)</code>	True if all items are True

What's Next?

Next up: **Parts 19-22** – Data Structures & Algorithms, Databases, Backend, Full-Stack!

PART 19: DATA STRUCTURES & ALGORITHMS

CHAPTER 19.1: Big O Notation – Measuring Efficiency

What You'll Learn

- What Big O notation is (measuring algorithm efficiency)
- Common time complexities
- Why it matters for performance

What Is Big O Notation?

Big O notation describes how fast an algorithm runs as the input size grows. It focuses on the worst-case scenario.

```
# O(1) - Constant time (always the same, no matter input size)
def get_first(items):
    return items[0] # Takes same time for 10 or 10,000 items

# O(n) - Linear time (time grows with input size)
def find_max(items):
    max_val = items[0]
    for item in items: # Loops through every item once
        if item > max_val:
            max_val = item
    return max_val

# O(n²) - Quadratic time (nested loops)
def find_duplicates(items):
    duplicates = []
    for i in range(len(items)):
        for j in range(i + 1, len(items)): # Nested loop
            if items[i] == items[j]:
                duplicates.append(items[i])
    return duplicates
```

Common Big O Complexities

Notation	Name	Example
$O(1)$	Constant	Array access, hash table lookup
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Single loop, searching a list
$O(n \log n)$	Linearithmic	Efficient sorting (merge sort, quicksort)
$O(n^2)$	Quadratic	Nested loops, bubble sort
$O(2^n)$	Exponential	Recursive Fibonacci (naive)

Visualizing Growth

```
import time

def measure_time(func, n):
    start = time.perf_counter()
    func(n)
    return time.perf_counter() - start

# O(n) - Linear
def linear_example(n):
    total = 0
    for i in range(n):
        total += i

# O(n^2) - Quadratic
def quadratic_example(n):
    total = 0
    for i in range(n):
        for j in range(n):
            total += i + j

# As n grows, quadratic becomes MUCH slower
# n=1000: linear ~0.0001s, quadratic ~0.1s
# n=10000: linear ~0.001s, quadratic ~10s
```

Chapter Summary

Complexity	Speed	Description
O(1)	Fastest	Constant time
O(log n)	Very fast	Divides problem in half
O(n)	Fast	One loop
O(n log n)	Good	Efficient sorting
O(n ²)	Slow	Nested loops
O(2 ⁿ)	Very slow	Exponential growth

CHAPTER 19.2: Arrays and Lists

What You'll Learn

- Time complexities for list operations
- When to use lists vs other structures
- Efficient list usage

List Operation Complexities

Operation	Time Complexity	Notes
Access by index	$O(1)$	<code>arr[i]</code>
Append to end	$O(1)$ amortized	<code>arr.append(x)</code>
Insert at beginning	$O(n)$	<code>arr.insert(0, x)</code>
Delete at end	$O(1)$	<code>arr.pop()</code>
Delete at beginning	$O(n)$	<code>arr.pop(0)</code>
Search by value	$O(n)$	<code>x in arr</code>
Sort	$O(n \log n)$	<code>arr.sort()</code>

Efficient List Usage

```
# ❌ Inefficient (inserting at beginning shifts all elements)
def build_list_inefficient(n):
    result = []
    for i in range(n):
        result.insert(0, i)  # O(n) each time → O(n²) total
    return result

# ✅ Efficient (appending to end)
def build_list_efficient(n):
    result = []
    for i in range(n):
        result.append(i)  # O(1) each time → O(n) total
    return result

# ❌ Inefficient (searching in list)
def find_inefficient(haystack, needles):
    result = []
    for needle in needles:
        if needle in haystack:  # O(n) each time
            result.append(needle)
    return result

# ✅ Efficient (using set for lookups)
def find_efficient(haystack, needles):
    haystack_set = set(haystack)  # O(n) once
    result = []
    for needle in needles:
        if needle in haystack_set:  # O(1) each time
            result.append(needle)
    return result
```

Chapter Summary

Use List When...	Use Other Structure When...
Order matters	Fast lookups by value (use set)
Frequent appends	Frequent inserts at beginning (use deque)
Index access needed	Key-value mapping needed (use dict)

CHAPTER 19.3: Stacks (LIFO)

What You'll Learn

- What a stack is (Last-In, First-Out)
- Push and pop operations
- Implementing stacks in Python
- Real-world applications

What Is a Stack?

A **stack** is like a stack of plates: you add to the top and remove from the top. Last In, First Out (LIFO).

```
# Python list as a stack
stack = []

# Push (add to top)
stack.append(1)
stack.append(2)
stack.append(3)
print(stack)  # [1, 2, 3]

# Pop (remove from top)
top = stack.pop()
print(top)    # 3
print(stack)  # [1, 2]

# Peek (look at top without removing)
top = stack[-1]
print(top)    # 2
```

Custom Stack Implementation

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._items.pop()

    def peek(self):
        if self.is_empty():
            return None
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
        return len(self._items)

# Usage
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
print(stack.pop())    # 3
print(stack.peek())   # 2
print(stack.size())   # 2
```

Real-World Examples

Undo/Redo Feature

```
class UndoRedo:
    def __init__(self):
        self.undo_stack = []
        self.redo_stack = []

    def perform_action(self, action):
        print(f"Doing: {action}")
        self.undo_stack.append(action)
        self.redo_stack.clear()

    def undo(self):
        if not self.undo_stack:
            print("Nothing to undo")
            return
        action = self.undo_stack.pop()
        print(f"Undo: {action}")
        self.redo_stack.append(action)

    def redo(self):
        if not self.redo_stack:
            print("Nothing to redo")
            return
        action = self.redo_stack.pop()
        print(f"Redo: {action}")
        self.undo_stack.append(action)

history = UndoRedo()
history.perform_action("Type 'Hello'")
history.perform_action("Add image")
history.undo()    # Undo: Add image
history.redo()    # Redo: Add image
```

Balanced Parentheses Checker

```
def is_balanced(text):  
    """Check if parentheses, brackets, braces are balanced"""  
    stack = []  
    pairs = {'(': ')', '[': ']', '{': '}'  
  
    for char in text:  
        if char in '([{':  
            stack.append(char)  
        elif char in ')]}':  
            if not stack or stack.pop() != pairs[char]:  
                return False  
    return len(stack) == 0  
  
print(is_balanced("()"))          # True  
print(is_balanced("() [] {}"))    # True  
print(is_balanced("]"))           # False  
print(is_balanced("(] ]"))        # False  
print(is_balanced("{ [] }"))      # True
```

Browser Back Button

```
class BrowserHistory:
    def __init__(self):
        self.back_stack = []
        self.forward_stack = []
        self.current = None

    def visit(self, url):
        if self.current:
            self.back_stack.append(self.current)
        self.current = url
        self.forward_stack.clear()
        print(f"Visited: {url}")

    def back(self):
        if not self.back_stack:
            print("No page to go back to")
            return
        self.forward_stack.append(self.current)
        self.current = self.back_stack.pop()
        print(f"Back to: {self.current}")

    def forward(self):
        if not self.forward_stack:
            print("No page to go forward to")
            return
        self.back_stack.append(self.current)
        self.current = self.forward_stack.pop()
        print(f"Forward to: {self.current}")

history = BrowserHistory()
history.visit("google.com")
history.visit("youtube.com")
history.visit("github.com")
history.back()      # Back to: youtube.com
history.back()      # Back to: google.com
history.forward()   # Forward to: youtube.com
```

Chapter Summary

Operation	Complexity	Description
Push	O(1)	Add to top
Pop	O(1)	Remove from top
Peek	O(1)	View top
Search	O(n)	Find element

CHAPTER 19.4: Queues (FIFO)

What You'll Learn

- What a queue is (First-In, First-Out)
- Using `collections.deque` for queues
- Real-world applications

What Is a Queue?

A **queue** is like a line at a store: first come, first served. First In, First Out (FIFO).

```
from collections import deque

# Using deque (pronounced "deck") for efficient queue
queue = deque()

# Enqueue (add to end)
queue.append(1)
queue.append(2)
queue.append(3)
print(queue)  # deque([1, 2, 3])

# Dequeue (remove from front)
front = queue.popleft()
print(front)  # 1
print(queue)  # deque([2, 3])
```

List vs Deque for Queues

```
# ❌ List as queue (inefficient)

list_queue = []

list_queue.append(1)      # O(1)
list_queue.append(2)      # O(1)
list_queue.pop(0)         # O(n) - SLOW for large queues!


# ✅ Deque as queue (efficient)

from collections import deque

deque_queue = deque()

deque_queue.append(1)      # O(1)
deque_queue.append(2)      # O(1)
deque_queue.popleft()      # O(1) - FAST!
```

Custom Queue Implementation

```
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, item):
        self._items.append(item)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self._items.popleft()

    def front(self):
        if self.is_empty():
            return None
        return self._items[0]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
        return len(self._items)

# Usage
q = Queue()
q.enqueue("Task 1")
q.enqueue("Task 2")
q.enqueue("Task 3")
print(q.dequeue()) # Task 1
print(q.front())   # Task 2
print(q.size())    # 2
```


Real-World Examples

Print Queue

```
class PrintQueue:
    def __init__(self):
        self.queue = deque()

    def add_job(self, document, pages):
        self.queue.append({"document": document, "pages": pages})
        print(f"Added: {document} ({pages} pages)")

    def print_next(self):
        if not self.queue:
            print("No jobs in queue")
            return
        job = self.queue.popleft()
        print(f"Printing: {job['document']} ({job['pages']} pages)")

    def show_queue(self):
        for job in self.queue:
            print(f"  - {job['document']}")

printer = PrintQueue()
printer.add_job("Report.pdf", 10)
printer.add_job("Photo.jpg", 1)
printer.add_job("Essay.docx", 5)
printer.print_next()  # Printing: Report.pdf (10 pages)
printer.show_queue()
```

Task Scheduler

```
from collections import deque
import time

class TaskScheduler:
    def __init__(self):
        self.tasks = deque()

    def add_task(self, task_name, delay=0):
        execute_at = time.time() + delay
        self.tasks.append((execute_at, task_name))
        self._sort_by_time()

    def _sort_by_time(self):
        self.tasks = deque(sorted(self.tasks, key=lambda x: x[0]))

    def run_pending(self):
        now = time.time()
        while self.tasks and self.tasks[0][0] <= now:
            execute_at, task = self.tasks.popleft()
            print(f"Running: {task} (scheduled at {execute_at})")

scheduler = TaskScheduler()
scheduler.add_task("Backup database", 2)
scheduler.add_task("Send email", 1)
scheduler.add_task("Clean cache", 3)

print("Waiting for tasks...")
time.sleep(3)
scheduler.run_pending()
```

BFS (Breadth-First Search) using Queue

```
from collections import deque

def bfs(graph, start):
    """Breadth-first search using a queue"""
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Graph as adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

bfs(graph, 'A')  # A B C D E F
```

Chapter Summary

Operation	Complexity	Description
Enqueue	O(1)	Add to back
Dequeue	O(1)	Remove from front
Front	O(1)	View front
Search	O(n)	Find element

CHAPTER 19.5: Sorting Algorithms

What You'll Learn

- Bubble sort (simple but slow)
- Selection sort
- Insertion sort
- Python's built-in sort (Timsort)

Bubble Sort ($O(n^2)$)

```
def bubble_sort(arr):  
    """Simplest sorting algorithm. Repeatedly swaps adjacent elements."""  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True  
        if not swapped:  
            break  
    return arr  
  
arr = [64, 34, 25, 12, 22, 11, 90]  
print(bubble_sort(arr))  # [11, 12, 22, 25, 34, 64, 90]
```

Selection Sort ($O(n^2)$)

```
def selection_sort(arr):  
    """Finds minimum element and places it at beginning."""  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
    return arr  
  
arr = [64, 34, 25, 12, 22, 11, 90]  
print(selection_sort(arr)) # [11, 12, 22, 25, 34, 64, 90]
```

Insertion Sort ($O(n^2)$, good for small data)

```
def insertion_sort(arr):  
    """Builds sorted array one element at a time."""  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key  
    return arr  
  
arr = [64, 34, 25, 12, 22, 11, 90]  
print(insertion_sort(arr)) # [11, 12, 22, 25, 34, 64, 90]
```

Python's Built-in Sort (Timsort, $O(n \log n)$)

```
# Most efficient, use this in real projects
arr = [64, 34, 25, 12, 22, 11, 90]

# In-place sort
arr.sort()
print(arr)  # [11, 12, 22, 25, 34, 64, 90]

# Returns new sorted list
sorted_arr = sorted(arr)

# Custom sorting
words = ["apple", "Banana", "cherry", "DATE"]
words.sort(key=str.lower)  # Case-insensitive
print(words)  # ['apple', 'Banana', 'cherry', 'DATE']

words.sort(key=len)  # Sort by length
print(words)  # ['DATE', 'apple', 'Banana', 'cherry']
```

Chapter Summary

Algorithm	Time Complexity	Space	Best For
Bubble sort	$O(n^2)$	$O(1)$	Learning
Selection sort	$O(n^2)$	$O(1)$	Small arrays
Insertion sort	$O(n^2)$	$O(1)$	Nearly sorted data
Timsort (Python)	$O(n \log n)$	$O(n)$	Everything (use this!)

CHAPTER 19.6: Searching Algorithms

What You'll Learn

- Linear search ($O(n)$)
- Binary search ($O(\log n)$)
- When to use each

Linear Search ($O(n)$)

```
def linear_search(arr, target):
    """Checks each element one by one. Works on unsorted data."""
    for i, item in enumerate(arr):
        if item == target:
            return i
    return -1 # Not found

arr = [64, 34, 25, 12, 22, 11, 90]
print(linear_search(arr, 22)) # 4
print(linear_search(arr, 99)) # -1
```

Binary Search ($O(\log n)$)

```
def binary_search(arr, target):
    """
    Divides search space in half each time.
    Requires SORTED array!
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Must be sorted!
sorted_arr = [11, 12, 22, 25, 34, 64, 90]
print(binary_search(sorted_arr, 22)) # 2
print(binary_search(sorted_arr, 99)) # -1
```

Recursive Binary Search

```
def binary_search_recursive(arr, target, left, right):
    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)

sorted_arr = [11, 12, 22, 25, 34, 64, 90]
print(binary_search_recursive(sorted_arr, 22, 0, len(sorted_arr) - 1)) # 2
```

Binary Search vs Linear Search

```
import time

def time_search(search_func, arr, target, *args):
    start = time.perf_counter()
    result = search_func(arr, target, *args)
    end = time.perf_counter()
    return result, end - start

# Create large sorted array
large_arr = list(range(1000000))

# Linear search on large array
linear_result, linear_time = time_search(linear_search, large_arr, 999999)
print(f"Linear: {linear_time:.6f}s") # ~0.01s

# Binary search on large array
binary_result, binary_time = time_search(binary_search, large_arr, 999999, 0, len(large_arr) - 1)
print(f"Binary: {binary_time:.6f}s") # ~0.000001s (much faster!)
```


Chapter Summary

Algorithm	Time Complexity	Requires Sorted	Best For
Linear search	O(n)	No	Small arrays, unsorted data
Binary search	O(log n)	Yes	Large sorted arrays

PART 19 SUMMARY

Chapter	Topic
19.1	Big O Notation
19.2	Arrays and Lists
19.3	Stacks (LIFO)
19.4	Queues (FIFO)
19.5	Sorting Algorithms
19.6	Searching Algorithms

What's Next?

Next up: **Parts 20-22** – Databases, Backend Development, and Full-Stack!

PART 20: DATABASES

CHAPTER 20.1: SQL Fundamentals

What You'll Learn

- What SQL is (Structured Query Language)
- Basic SQL commands
- Database concepts

What Is SQL?

SQL (Structured Query Language) is the standard language for working with databases. It lets you create, read, update, and delete data.

```
-- Basic SQL commands

SELECT * FROM users;           -- Read data

INSERT INTO users VALUES (1, 'Ryuu'); -- Create data

UPDATE users SET name = 'Anna' WHERE id = 1; -- Update data

DELETE FROM users WHERE id = 1; -- Delete data
```

Main SQL Commands

Command	Purpose	Example
SELECT	Read data	SELECT name, age FROM users
INSERT	Add data	INSERT INTO users (name) VALUES ('Ryuu')
UPDATE	Change data	UPDATE users SET age = 12 WHERE name = 'Ryuu'
DELETE	Remove data	DELETE FROM users WHERE age < 13
CREATE	Make tables	CREATE TABLE users (id INT, name TEXT)
DROP	Remove tables	DROP TABLE users

SQL Filtering

```
-- WHERE clause (filtering)

SELECT * FROM users WHERE age >= 18;

SELECT * FROM products WHERE price < 50;

SELECT * FROM orders WHERE status = 'completed';


-- AND / OR

SELECT * FROM users WHERE age >= 18 AND city = 'Tokyo';

SELECT * FROM products WHERE price < 10 OR category = 'sale';


-- ORDER BY (sorting)

SELECT * FROM users ORDER BY age DESC;

SELECT * FROM products ORDER BY price ASC;
```

Chapter Summary

Concept	SQL
Read	<code>SELECT columns FROM table WHERE condition</code>
Insert	<code>INSERT INTO table (cols) VALUES (vals)</code>
Update	<code>UPDATE table SET col = value WHERE condition</code>
Delete	<code>DELETE FROM table WHERE condition</code>

CHAPTER 20.2: SQLite – Lightweight Database

What You'll Learn

- Using SQLite in Python
- Creating databases and tables
- CRUD operations

Connecting to SQLite

```
import sqlite3

# Connect to database (creates file if doesn't exist)
conn = sqlite3.connect("my_database.db")
cursor = conn.cursor()

# Close connection when done
conn.close()
```

Creating Tables

```
import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# Create students table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER,
        grade REAL
    )
''')

conn.commit()
conn.close()
```

INSERT – Adding Data

```
def add_student(name, age, grade):
    conn = sqlite3.connect("school.db")
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
        (name, age, grade)
    )

    conn.commit()
    conn.close()
    print(f"Added: {name}")

add_student("Ryu", 11, 95.5)
add_student("Anna", 12, 87.0)
add_student("Max", 10, 92.5)
```

SELECT – Reading Data

```
def get_all_students():
    conn = sqlite3.connect("school.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM students")
    students = cursor.fetchall()

    conn.close()
    return students

def get_students_above_grade(min_grade):
    conn = sqlite3.connect("school.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM students WHERE grade > ?", (min_grade,))
    students = cursor.fetchall()

    conn.close()
    return students

# Usage
all_students = get_all_students()
for student in all_students:
    print(student)

top_students = get_students_above_grade(90)
print(f"Top students: {top_students}")
```

UPDATE – Modifying Data

```
def update_student_grade(student_id, new_grade):
    conn = sqlite3.connect("school.db")
    cursor = conn.cursor()

    cursor.execute(
        "UPDATE students SET grade = ? WHERE id = ?",
        (new_grade, student_id)
    )

    conn.commit()
    conn.close()
    print(f"Updated student {student_id} to grade {new_grade}")

update_student_grade(2, 89.5)
```

DELETE – Removing Data

```
def delete_student(student_id):
    conn = sqlite3.connect("school.db")
    cursor = conn.cursor()

    cursor.execute("DELETE FROM students WHERE id = ?", (student_id,))

    conn.commit()
    conn.close()
    print(f"Deleted student {student_id}")

# delete_student(3)
```

Complete CRUD Example

```
import sqlite3

class StudentDB:
    def __init__(self, db_name="school.db"):
        self.conn = sqlite3.connect(db_name)
        self.cursor = self.conn.cursor()
        self._create_table()

    def _create_table(self):
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS students (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                age INTEGER,
                grade REAL
            )
        ''')
        self.conn.commit()

    def add(self, name, age, grade):
        self.cursor.execute(
            "INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
            (name, age, grade)
        )
        self.conn.commit()
        print(f"Added: {name}")

    def get_all(self):
        self.cursor.execute("SELECT * FROM students")
        return self.cursor.fetchall()

    def get_by_id(self, student_id):
        self.cursor.execute("SELECT * FROM students WHERE id = ?", (student_id,))
        return self.cursor.fetchone()

    def update_grade(self, student_id, new_grade):
        self.cursor.execute(
            "UPDATE students SET grade = ? WHERE id = ?",
            (new_grade, student_id)
        )
```

```

        self.conn.commit()

        print(f"Updated student {student_id}")

    def delete(self, student_id):
        self.cursor.execute("DELETE FROM students WHERE id = ?", (student_id,))
        self.conn.commit()
        print(f"Deleted student {student_id}")

    def close(self):
        self.conn.close()

# Usage
db = StudentDB()
db.add("Ryu", 11, 95.5)
db.add("Anna", 12, 87.0)

print(db.get_all())
db.update_grade(1, 98.0)
# db.delete(2)

db.close()

```

Chapter Summary

Operation	SQLite Method
Create table	<code>cursor.execute(CREATE TABLE)</code>
Insert	<code>cursor.execute(INSERT, values)</code>
Select	<code>cursor.execute(SELECT); cursor.fetchall()</code>
Update	<code>cursor.execute(UPDATE, values)</code>
Delete	<code>cursor.execute(DELETE, values)</code>

CHAPTER 20.3: SQLAlchemy ORM (Optional)

What You'll Learn

- What ORM is (Object-Relational Mapping)
- Using SQLAlchemy

- Working with Python objects instead of SQL

What Is an ORM?

ORM lets you work with databases using Python classes and objects instead of writing SQL.

```
# Without ORM (raw SQL)

cursor.execute("SELECT * FROM users WHERE name = ?", ("Ryu",))

user = cursor.fetchone()


# With ORM (Python objects)

user = session.query(User).filter_by(name="Ryu").first()

print(user.name, user.age)
```

SQLAlchemy Example

```
from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# Setup
engine = create_engine('sqlite:///school_orm.db', echo=False)
Base = declarative_base()
Session = sessionmaker(bind=engine)
session = Session()

# Define model
class Student(Base):
    __tablename__ = 'students'

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    age = Column(Integer)
    grade = Column(Float)

    def __repr__(self):
        return f"Student(name='{self.name}', age={self.age}, grade={self.grade})"

# Create table
Base.metadata.create_all(engine)

# CRUD operations
# Create
new_student = Student(name="Ryu", age=11, grade=95.5)
session.add(new_student)
session.commit()

# Read
all_students = session.query(Student).all()
students_over_90 = session.query(Student).filter(Student.grade > 90).all()

# Update
student = session.query(Student).filter_by(name="Ryu").first()
student.grade = 98.0
session.commit()
```

```
# Delete
session.delete(student)
session.commit()
```

Chapter Summary

ORM Operation	SQL Equivalent
<code>session.add(obj)</code>	INSERT
<code>session.query(Model).all()</code>	SELECT *
<code>session.query(Model).filter(...)</code>	SELECT ... WHERE
<code>obj.attr = value; session.commit()</code>	UPDATE
<code>session.delete(obj)</code>	DELETE

PART 21: BACKEND DEVELOPMENT (Condensed)

CHAPTER 21.1: HTTP Fundamentals

What You'll Learn

- What HTTP is (Hypertext Transfer Protocol)
- HTTP methods (GET, POST, PUT, DELETE)
- Status codes
- How the web works

What Is HTTP?

HTTP is the protocol used to transfer data on the web. When you visit a website, your browser sends an HTTP request to a server, and the server sends back an HTTP response.

HTTP Methods

Method	Purpose	Example
GET	Read data	GET /users
POST	Create data	POST /users (with data in body)
PUT	Update all fields	PUT /users/1
PATCH	Update some fields	PATCH /users/1
DELETE	Delete data	DELETE /users/1

HTTP Status Codes

Code Range	Meaning	Examples
200-299	Success	200 OK, 201 Created
300-399	Redirection	301 Moved Permanently
400-499	Client Error	400 Bad Request, 404 Not Found
500-599	Server Error	500 Internal Server Error

Chapter Summary

Concept	Description
HTTP	Protocol for web communication
GET	Read data
POST	Create data
PUT/PATCH	Update data
DELETE	Delete data

CHAPTER 21.2: REST APIs

What You'll Learn

- What REST APIs are
- API endpoints
- JSON data format

- Making API requests with Python

What Is a REST API?

REST API (Representational State Transfer Application Programming Interface) is a way for applications to communicate over HTTP.

```
# API endpoints example

GET      /api/users          # Get all users
GET      /api/users/1       # Get user with id 1
POST     /api/users          # Create new user
PUT      /api/users/1       # Update user 1
DELETE   /api/users/1       # Delete user 1
```

Working with JSON

APIs usually send data in JSON format.

```
import json

# Python dict to JSON string
person = {"name": "Ryu", "age": 11}
json_string = json.dumps(person)
print(json_string)  # '{"name": "Ryu", "age": 11}'

# JSON string to Python dict
json_data = '{"name": "Anna", "age": 12}'
person = json.loads(json_data)
print(person["name"])  # Anna
```

Making API Requests

```
import requests

# GET request
response = requests.get("https://api.example.com/users")
if response.status_code == 200:
    users = response.json()
    for user in users:
        print(user["name"])

# POST request
new_user = {"name": "Ryuu", "age": 11}
response = requests.post("https://api.example.com/users", json=new_user)

# PUT request
update_data = {"age": 12}
response = requests.put("https://api.example.com/users/1", json=update_data)

# DELETE request
response = requests.delete("https://api.example.com/users/1")
```

Chapter Summary

Operation	requests method
GET	<code>requests.get(url)</code>
POST	<code>requests.post(url, json=data)</code>
PUT	<code>requests.put(url, json=data)</code>
DELETE	<code>requests.delete(url)</code>

CHAPTER 21.3: Flask – Web Framework

What You'll Learn

- Creating web applications with Flask
- Routes and views
- Running a development server

What Is Flask?

Flask is a lightweight web framework for Python. It makes it easy to create web applications and APIs.

Basic Flask App

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Route with GET
@app.route('/')
def home():
    return "Hello, World!"

# Route with JSON response
@app.route('/api/users')
def get_users():
    users = [
        {"id": 1, "name": "Ryu"},
        {"id": 2, "name": "Anna"}
    ]
    return jsonify(users)

# Route with parameter
@app.route('/api/users/<int:user_id>')
def get_user(user_id):
    return jsonify({"id": user_id, "name": f"User {user_id}"})

# POST route
@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()
    return jsonify({"message": f"Created user {data.get('name')}"}), 201

if __name__ == '__main__':
    app.run(debug=True)
```

Running Flask

```
python app.py
# Server running at http://127.0.0.1:5000
```

Chapter Summary

Concept	Flask Syntax
Route	<code>@app.route('/path')</code>
GET	Default method
POST	<code>methods=['POST']</code>
URL param	<code><type:name></code>
JSON response	<code>jsonify(data)</code>

CHAPTER 21.4: FastAPI – Modern API Framework

What You'll Learn

- Creating APIs with FastAPI
- Automatic documentation
- Type hints in routes

What Is FastAPI?

FastAPI is a modern, fast web framework for building APIs. It's faster than Flask and includes automatic API documentation.

Basic FastAPI App

```
from fastapi import FastAPI

from pydantic import BaseModel
from typing import List

app = FastAPI()

# Data model
class User(BaseModel):
    id: int
    name: str
    age: int

# Fake database
users_db = [
    User(id=1, name="Ryu", age=11),
    User(id=2, name="Anna", age=12)
]

# GET all users
@app.get("/users", response_model=List[User])
def get_users():
    return users_db

# GET single user
@app.get("/users/{user_id}")
def get_user(user_id: int):
    for user in users_db:
        if user.id == user_id:
            return user
    return {"error": "User not found"}

# POST create user
@app.post("/users", response_model=User)
def create_user(user: User):
    users_db.append(user)
    return user

# Run with: uvicorn main:app --reload
```

Interactive Documentation

FastAPI automatically generates documentation at:

- `http://localhost:8000/docs` (Swagger UI)
- `http://localhost:8000/redoc` (ReDoc)

Chapter Summary

Feature	FastAPI
Speed	Very fast
Docs	Automatic (Swagger)
Type hints	Full support
Run command	<code>uvicorn main:app --reload</code>

CHAPTER 21.5: Authentication & JWT

What You'll Learn

- What authentication is (verifying user identity)
- What JWT is (JSON Web Tokens)
- Protecting API endpoints

What Is JWT?

JWT (JSON Web Token) is a secure way to authenticate users. After login, the server gives the client a token. The client sends this token with each request.

```
# Simplified JWT example

import jwt
from datetime import datetime, timedelta

SECRET_KEY = "your-secret-key"

# Create token (login)
def create_token(user_id, username):
    payload = {
        'user_id': user_id,
        'username': username,
        'exp': datetime.utcnow() + timedelta(hours=24)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm='HS256')

# Verify token (protected routes)
def verify_token(token):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        return payload
    except jwt.ExpiredSignatureError:
        return None # Token expired
    except jwt.InvalidTokenError:
        return None # Invalid token
```

Protected Endpoint Example (FastAPI)

```
from fastapi import FastAPI, HTTPException, Depends, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

app = FastAPI()
security = HTTPBearer()

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)):
    token = credentials.credentials
    payload = verify_token(token) # Your verify function
    if not payload:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired token"
        )
    return payload

@app.get("/protected")
def protected_route(user=Depends(verify_token)):
    return {"message": f"Welcome user {user['username']}"}

```

Chapter Summary

Concept	Purpose
Authentication	Verify user identity
Authorization	Check user permissions
JWT	Stateless token for API auth
Bearer token	Token format for API requests

PARTS 20-21 SUMMARY

Part	Topic
20	Databases (SQLite, SQLAlchemy)
21	Backend (HTTP, REST, Flask, FastAPI, Auth)

What's Next?

Next up: **Part 22: Super App Development** – Full-stack applications!

PART 22: SUPER APP DEVELOPMENT

CHAPTER 22.1: Frontend Basics (HTML/CSS/JS)

What You'll Learn

- HTML structure
- CSS styling
- JavaScript interactivity
- Connecting frontend to backend

HTML – Structure

```
<!DOCTYPE html>

<html>
  <head>
    <title>My App</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Welcome to My App</h1>
    <div id="app">
      <button onclick="fetchUsers()">Load Users</button>
      <ul id="user-list"></ul>
    </div>
    <script src="script.js"></script>
  </body>
</html>
```

CSS – Styling

```
/* style.css */

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 20px;
    background-color: #f0f0f0;
}

h1 {
    color: #333;
    text-align: center;
}

button {
    background-color: #007bff;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background-color: #0056b3;
}

.user-card {
    background: white;
    padding: 10px;
    margin: 10px 0;
    border-radius: 5px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}
```

JavaScript – Interactivity

```
// script.js
async function fetchUsers() {
  try {
    const response = await fetch('/api/users');
    const users = await response.json();

    const userList = document.getElementById('user-list');
    userList.innerHTML = '';

    users.forEach(user => {
      const li = document.createElement('li');
      li.className = 'user-card';
      li.innerHTML = `<strong>${user.name}</strong> - Age: ${user.age}`;
      userList.appendChild(li);
    });
  } catch (error) {
    console.error('Error:', error);
  }
}
```

Chapter Summary

Technology	Purpose
HTML	Structure of webpage
CSS	Styling and layout
JavaScript	Interactivity and API calls

CHAPTER 22.2: React Integration

What You'll Learn

- What React is
- Components and state
- Connecting React to Python backend

What Is React?

React is a JavaScript library for building user interfaces. It's component-based and very popular for modern web apps.

```
// App.js

import React, { useState, useEffect } from 'react';

function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <div>Loading...</div>;

  return (
    <div>
      <h1>User List</h1>
      {users.map(user => (
        <div key={user.id} className="user-card">
          <h3>{user.name}</h3>
          <p>Age: {user.age}</p>
        </div>
      ))}
    </div>
  );
}

export default App;
```


Full-Stack Example: FastAPI + React

Backend (FastAPI)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI()

# Enable CORS for React
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)

class User(BaseModel):
    id: int
    name: str
    age: int

users_db = [
    User(id=1, name="Ryu", age=11),
    User(id=2, name="Anna", age=12),
]

@app.get("/api/users")
def get_users():
    return users_db

@app.post("/api/users")
def create_user(user: User):
    users_db.append(user)
    return user

# Run: uvicorn main:app --reload --port 8000
```

Frontend (React) – API Call

```
// Add new user

function AddUser() {
  const [name, setName] = useState('');
  const [age, setAge] = useState('');

  const addUser = async () => {
    const response = await fetch('http://localhost:8000/api/users', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({id: Date.now(), name, age: parseInt(age)})
    });
    if (response.ok) {
      setName('');
      setAge('');
      // Refresh list
    }
  };

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.value)} placeholder="Name"/>
      <input value={age} onChange={e => setAge(e.target.value)} placeholder="Age" type="number"/>
      <button onClick={addUser}>Add User</button>
    </div>
  );
}
```

Chapter Summary

Concept	Description
Component	Reusable UI piece
State	Data that changes
Effect	Run code on mount/update
CORS	Cross-origin resource sharing

CHAPTER 22.3: Docker & Deployment

What You'll Learn

- What Docker is (containerization)
- Creating Dockerfiles
- Deploying to cloud services

What Is Docker?

Docker packages your app and its dependencies into a container that runs anywhere.

Dockerfile for Python Backend

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

requirements.txt

```
fastapi==0.104.0
uvicorn==0.24.0
sqlalchemy==2.0.0
```

Docker Commands

```
# Build image
docker build -t my-python-app .

# Run container
docker run -p 8000:8000 my-python-app

# Run with environment variables
docker run -p 8000:8000 -e DATABASE_URL=postgresql://... my-python-app
```

Deployment Options

Service	Best For
Render	Easy, free tier
Railway	Simple deployment
Heroku	Traditional PaaS
AWS EC2	Full control
Google Cloud Run	Serverless containers
Fly.io	Global deployment

Deploying to Render

1. Push code to GitHub
2. Create account on render.com
3. Connect GitHub repository
4. Set build command: `pip install -r requirements.txt`
5. Set start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
6. Deploy!

Chapter Summary

Concept	Purpose
Docker	Package app in container
Dockerfile	Instructions for building image
Deployment	Making app live on internet

CHAPTER 22.4: Cloud Services

What You'll Learn

- Cloud providers (AWS, GCP, Azure)
- Databases in the cloud
- Environment variables

Cloud Database Options

Service	Provider	Free Tier
Supabase	Self-hosted	Yes (500MB)
PlanetScale	MySQL	Yes
MongoDB Atlas	MongoDB	Yes (512MB)
Railway	PostgreSQL	Yes (limited)
Neon	PostgreSQL	Yes

Environment Variables

Never hardcode secrets! Use environment variables.

```
import os

# Get from environment
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///local.db")
SECRET_KEY = os.getenv("SECRET_KEY")
DEBUG = os.getenv("DEBUG", "False").lower() == "true"
```

Setting Environment Variables

```
# Local
export DATABASE_URL="postgresql://user:pass@localhost/db"

# Render dashboard
Environment Variables → Add variable

# Railway
Add variable in dashboard

# Docker
docker run -e DATABASE_URL="..." my-app
```

Full Deployment Checklist

- ☐ Code pushed to GitHub
- ☐ requirements.txt created
- ☐ Dockerfile ready (optional)
- ☐ Environment variables set
- ☐ Database provisioned
- ☐ Deployed and tested

Chapter Summary

Concept	Use
Cloud DB	Persistent data storage
Environment variables	Secrets and config
Deployment platform	Host your app

CHAPTER 22.5: Building Full-Scale Apps

What You'll Learn

- Project structure for full-stack apps
- Putting everything together

- Next steps in your coding journey

Full-Stack Project Structure

```
my_project/
├── backend/
│   ├── main.py
│   ├── models.py
│   ├── database.py
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.js
│   │   └── components/
│   │       └── index.js
│   └── package.json
├── docker-compose.yml
└── README.md
```

Putting It All Together

```
# Complete backend example (main.py)

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List
import sqlite3
from contextlib import contextmanager

app = FastAPI()

@contextmanager
def get_db():
    conn = sqlite3.connect("app.db")
    conn.row_factory = sqlite3.Row
    try:
        yield conn
    finally:
        conn.close()

# Initialize database
def init_db():
    with get_db() as conn:
        conn.execute('''
            CREATE TABLE IF NOT EXISTS items (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                description TEXT
            )
        ''')
        conn.commit()

init_db()

class Item(BaseModel):
    id: int = None
    name: str
    description: str = ""

@app.get("/api/items", response_model=List[Item])
def get_items():
    with get_db() as conn:
```



```

        cursor = conn.execute("SELECT * FROM items")
        return [dict(row) for row in cursor.fetchall()]

@app.post("/api/items", response_model=Item)
def create_item(item: Item):
    with get_db() as conn:
        cursor = conn.execute(
            "INSERT INTO items (name, description) VALUES (?, ?)",
            (item.name, item.description)
        )
        conn.commit()
        item.id = cursor.lastrowid
    return item

@app.delete("/api/items/{item_id}")
def delete_item(item_id: int):
    with get_db() as conn:
        conn.execute("DELETE FROM items WHERE id = ?", (item_id,))
        conn.commit()
    return {"message": "Deleted"}

# Run with: uvicorn main:app --reload

```

Your Journey Continues

Now that you've completed this book, you can build:

Type	Examples
Web Apps	Todo lists, blogs, dashboards
APIs	REST backends for mobile/web
Data Apps	Visualizations, analytics
Automation	Scripts, bots, scrapers
Desktop Apps	Tkinter, PyQt
Games	Pygame

Next Steps

1. **Build projects** – Start small, then scale up
2. **Read documentation** – Official docs are your best friend

- 3. **Join communities** – r/Python, Discord servers
- 4. **Contribute to open source** – Start with small fixes
- 5. **Keep learning** – Explore Django, FastAPI, data science, AI/ML

Chapter Summary

Skill	Where to Learn More
Web Apps	Django, Flask, FastAPI docs
Data Science	Pandas, NumPy, Matplotlib
Machine Learning	Scikit-learn, TensorFlow
Mobile (Kivy)	Kivy documentation
Game Dev	Pygame tutorials

PART 22 SUMMARY

Chapter	Topic
22.1	Frontend Basics (HTML/CSS/JS)
22.2	React Integration
22.3	Docker & Deployment
22.4	Cloud Services
22.5	Building Full-Scale Apps

CONGRATULATIONS, RYUU! 🎉

You completed the entire Python book!

What You've Learned

Part	Topic
1	Getting Started with Python
2	Variables and Data Types
3	Input, Output, and Operators
4	Control Flow
5	Loops
6	Functions
7	Strings
8	Lists
9	Tuples
10	Sets
11	Dictionaries
12	Modules and Packages
13	File Handling
14	Error Handling
15	Object-Oriented Programming
16	Advanced OOP
17	Advanced Python
18	Functional Programming
19	Data Structures & Algorithms
20	Databases
21	Backend Development
22	Super App Development

Total Chapters: ~100

GO SWIFT WITH THE FLOW ☁ NEXT WAY TO PYTHON 🐍

— RYUU

Open source. Open knowledge. Open future.